水泥烧成系统电除尘器的协同优化控制

摘　要

水泥烧成系统电除尘器需在排放达标与运行电耗之间协同取舍。本文以七天分钟级运行数据为基础，先逐列作可识别性审计，把“可由数据验证的”与“只能靠机理推演的”分开，再分别建立功率实证模型与排放灰箱模型，最后在历史运行范围内求解约束优化问题，依次回答四问。

针对问题一，本文建立了由可识别性审计、物理形式功率模型与 Deutsch–Anderson 灰箱构成的分层响应模型。首先逐列审计：出口浓度 10030 条有效记录全落在 48.74～50.00 mg/Nm³，5491 条恰为 50，与全部 12 个变量的相关系数绝对值均小于 0.017；截尾极大似然进一步给出可证伪的确认——仅用未封顶记录估得潜变量 $N(50.036, 0.313^2)$，预测封顶比例 0.5461 对实测 0.5475，其真实变异仅为均值的 0.63%。故判定该列量程截顶、不可识别，排放层只能按机理推演并逐处标注其性质。其次按电场功率 $\propto U^2$、振打功率 $\propto 1/T$ 的物理形式拟合总功率，得 $P = 73.1 + 0.0850\sum_i U_i^2 + 5.39\times10^4\sum_i T_i^{-1}$，在未参与拟合的测试日上 $R^2 = 0.993$、RMSE=5.98 kW、平均偏差 –0.02 kW，据此算出**随振打频率同步变化的电耗占总功率的 38.8%**（该项量级远超振打机构，不作机械功率解释），而入口温度、浓度、流量对总功率无额外解释力。再由相关分析提取历史操作规律：入口浓度升高时四场振打周期一律缩短（$r = -0.21 \sim -0.28$，四时段方向一致）。最后由每周期积灰量 $\propto L\cdot T$ 的质量守恒推导峰值标度，不必假定指数即得：单次峰值随周期近似线性放大，而其对小时均值的贡献 $\propto L\cdot T\cdot(1/T) = L$，**与周期无关**；相位折叠检验给出与振打同步的排放波动上界约 0.03 mg/Nm³。

针对问题二，本文建立了 K-means 工况聚类模型与支持域约束下的非线性规划模型。先以入口温度、入口浓度、烟气流量三个外生变量作 K-means 聚类，由轮廓系数（$k = 4$ 取最大值 0.4084）与温度两档 × 流量两档的 2×2 结构，定为 4 类典型工况。再以上述功率模型为目标函数，以记录值 50 mg/Nm³ 为锚点、不作任何缩放而建立的 Deutsch–Anderson 灰箱排放式为约束，以历史运行范围与工程排序关系为边界，构成 8 变量非线性规划，用多起点 SLSQP（53 个起点）求解，并以 4×10^5 点随机搜索对照。求得四类工况达到 10 mg/Nm³ 的最低功率为 1851.1～2104.3 kW，加权 1973.0 kW，比历史实际的 1783.2 kW **高 10.6%**（日电耗 42.8→47.4 MWh）。收紧排放必然增加电耗，同一排放水平下不存在额外的节能空间。八个解均把四个振打周期顶至支持域上界，属**角点解**；上界改取 P95 后功率升 1.8%～2.4%，第四问增幅仍达 6.13%。

针对问题三，本文建立了单位电功率去除收益的解析判据模型。把去除指数 K 与电场功率 P_{field} 同时对 U_i^2 求导，得判据 $\alpha_i/(b_iU_i^{ref\,2})$。第三电场参考电压最低，该收益在四类工况中均居首；原“前级优先”的结论实由排序约束 $U_1 \geq U_3 + 3$ 导出，而非权重 α_i 本身。振打侧同理按 c_i/T_i^2 排序，应优先延长周期较短的前级。据此给出低负荷四场同步升压、高负荷择优升压的策略，并整理为七条决策规则。

针对问题四，本文建立了四场电压同比缩放假设下的解析估计模型，并以八变量数值重解校验。由 $K \propto U^p$ 解得 $\Delta P = P_{field}[(1+\Delta K/K_{10})^{2/p} - 1]$，即 $\Delta P = P_{field}\ln[(10-B)/(5-B)]/K_{10}$，给出 123.1～142.4 kW，与数值解差不足 1%；加权功率由 1973.0 升至 2105.5 kW，**增幅 6.72%**，日电耗多 3.2 MWh。增幅与 $2/p$ 成正比，就功率增量与目标可达性而言，p 是主导性的待标定参数，一次电压阶跃试验即可确定，其余灰箱参数仍须一并标定。更关键的约束不是电耗而是**电源容量**：10 mg/Nm³ 仍在历史电压范围内，5 mg/Nm³ 则需各场电压超出历史最大值最多 5%。若要求解严格落在历史经验内，八组中七组仍可行，多耗 1.5%～6.1%。

最后以 81 组结构情景、权重消融与幂次参考表检验稳健性，并明确排放层各系数均属工程先验，所给控制参数只能作为现场试验起点。

关键词：电除尘器　可识别性审计　Deutsch–Anderson 灰箱　多起点 SLSQP

1　问题重述与问题分析

1.1 问题背景

电除尘器通过高压电场使粉尘荷电并向收尘极迁移。提高电场电压通常能够增强荷电与迁移，但会提高运行功率并受到火花、绝缘和联锁条件限制；振打过于频繁会增加再飞扬次数，周期过长又可能导致积灰加重和单次脱落量增大。水泥工业颗粒物排放受到国家标准约束 [1]，题目进一步给出 10 mg/Nm³ 和 5 mg/Nm³ 两个控制目标，要求同时考虑排放、总功率和振打峰值。

在建立模型之前，须先明确本题的变量在设备上各处于什么位置，否则后文的“可调量”与“结果量”容易混淆。图 1 给出研究对象的结构与变量定义：含尘烟气依次流经四个电场，每个电场由独立高压电源施加电压 U_i，并由独立振打装置按周期 T_i 清灰。

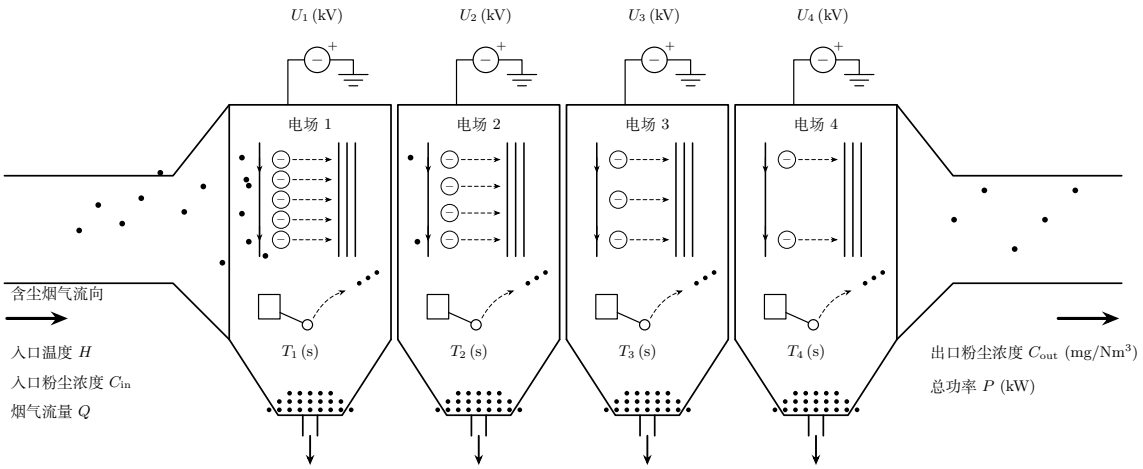

图 1　四电场电除尘器结构与变量定义

由图 1 可读出本题的变量结构：入口侧的入口温度 H、入口粉尘浓度 C_{in} 与烟气流量 Q 由上游工艺决定，属**不可调的工况量**；四电场电压 U_i 与振打周期 T_i 共 8 个量属**可调的控制量**；出口侧的出口粉尘浓度 C_{out} 与总功率 P 则是需要同时兼顾的**两个结果量**。四问都可归结为：在工况量给定的条件下如何配置这 8 个控制量，使一个结果量达标而另一个尽可能小。本文不涉及设备本体的机械结构设计。

1.2 问题要求

附件给出 2024 年 5 月 1 日 0 时至 5 月 7 日 23 时 59 分的分钟级运行记录，共 10080 条、14 个字段。据此需回答四个逐层递进的问题：

问题一：建立入口温度、入口浓度、烟气流量、四电场电压与四电场振打周期同出口粉尘浓度之间的关系，并分析振打动作引起的瞬时排放峰值。

问题二：依据入口工况划分典型运行区间，在出口浓度不超过 10 mg/Nm³ 的约束下联合优化四电场电压与四电场振打周期，使总电耗最低。

问题三：选取两类差异显著的工况，分别给出最优操作参数，并解释电压与振打周期在其间的调节优先级变化。

问题四：将排放约束由 10 mg/Nm³ 收紧至 5 mg/Nm³，计算相应的电耗增幅，并对高浓度工况给出运行建议。

1.3 总体思路

附件中有两列结果变量，信息量却相差悬殊：总功率几乎完全由控制量决定，而出口浓度被压缩在量程上限附近，几乎没有变化。若不加区分地将二者同时作为建模目标，模型在出口浓度上学到的只会是传感器的截顶读数。本文因此采取先分层、后优化的路线：

1. 逐列检查哪些关系能被数据辨认出来，把“可由附件验证的”与“只能靠机理推演的”分开；
2. 对可识别的总功率，按物理形式（电场功率 $\propto U^2$、振打功率 $\propto 1/T$）建立并检验模型；
3. 对不可识别的排放层，以**记录值本身**为锚点建立 Deutsch–Anderson 灰箱，不作任何未经证实的量纲改写；
4. 在历史运行范围内求解优化问题，并把“结果在哪些方面超出了历史经验”当作结论的一部分如实报告。

第 3 点是本文与常见做法的关键差别。附件记录的出口浓度约 50 mg/Nm³，题目要求的却是 10 与 5 mg/Nm³，相差 5～10 倍。承认这个差距，题目就自然变为：控制参数应如何由现状调整至超低排放目标，相应的电耗代价是多少。反过来，若为了让数据靠近目标区间而假设记录值点错了小数点，第二问就会变成“把排放放宽到 10 mg/Nm³ 能省多少电”——与题意正好相反。四问之间的衔接关系与两条证据路径见 §3.3 的全文技术路线图（图 4），以下先分述各问的分析思路。

1.4 问题一的分析

问题一要求建立各变量与出口浓度的关系。附件的两列结果变量必须区别对待：总功率由控制量近乎确定，可直接建模；出口浓度则须先检验其是否携带可学习的信号，否则任何回归都只是在复现传感器读数。因此本文首先对 14 列数据逐列作可识别性审计，以同期相关、滞后相关、封顶分类与未封顶子样本回归四项证据判定各列的可用性。然后对判定为可识别的总功率，按电场功率与电压平方成正比、振打功率与振打频率成正比的物理形式建模，并按时间顺序滚动检验；对判定为不可识别的出口浓度，改以 Deutsch–Anderson 关系为骨架构建灰箱，其系数取自机理与文献而非附件拟合，并逐处标注证据状态。最后处理振打峰值：附件没有实际振打时刻与秒级浓度，逐次峰值无法辨认，但峰值随周期的变化规律可由每周期积灰量的质量守恒直接推出，无须假定指数。

1.5 问题二的分析

问题二要求划分典型工况并在排放约束下求最低电耗。工况划分只应使用由上游工艺决定的外生变量，否则会出现用结果定义工况的循环，故本文只以入口温度、入口浓度与烟气流量作 K-means 聚类，并由轮廓系数与 Calinski–Harabasz 指标共同判断簇数。求解层面，目标函数取问题一辨识的功率模型，排放约束取问题一的灰箱排放式，可行域取各工况历史实际出现过的控制量范围并附加工程排序关系，由此构成 8 变量非线性规划。该问题在任一坐标下均无法化为凸问题，故不宜宣称凸性，本文改用多起点确定性求解并以大规模随机搜索独立对照。此外，附件记录的出口浓度约 50 mg/Nm³，而题目要求 10 mg/Nm³，二者相差五倍，因此所得最优解是否已越出历史运行经验，必须作为结论的一部分一并审计。

1.6 问题三的分析

问题三要求解释控制优先级。若只从某一次数值求解的结果归纳优先级，结论会随解的具体取值而变；因此本文改由模型结构直接求导，得到各电场单位电功率的去除指数增益这一解析判据，再以数值解印证。振打侧的优先级同理由边际节电率导出。至于附件中的历史操作规律，只作经验对照而不作因果依据——因为排放侧的响应本身不可识别。

1.7 问题四的分析

问题四要求估计限值收紧一档的电耗代价。直接比较两档的数值最优解只能得到数字，无法说明该数字由什么决定；因此本文先在四场电压同比缩放的简化下导出功率增量的解析式，再以八变量数值最优解检验该简化的偏差，使结论既有数值又有机理解释。由解析式可知，增幅的一阶依赖集中于去除指数对电压的幂次，这也指明敏感性分析应围绕该参数组织。此外，达到 5 mg/Nm³ 所需的电压是否超出历史范围，直接关系到目标能否实现，其重要性高于电耗本身，故须单独核算。

2　数据审计与关系可识别性分析

2.1 数据完整性与派生变量

附件覆盖 2024 年 5 月 1 日 0 时至 5 月 7 日 23 时 59 分，共 10080 条记录，采样间隔 1 min，时间戳无重复、无缺口。原始字段包括入口温度 H_t、入口浓度 $C_{in,t}$、烟气流量 Q_t、四电场电压 $U_{i,t}$、四电场振打周期 $T_{i,t}$、出口浓度 $C_{out,t}$ 和总功率 P_t。出口浓度缺失 50 条（0.50%），其余字段完整。

为表达过程负荷和振打风险，构造粉尘质量负荷

$$L_t = C_{in,t} Q_t / 1000 \tag{1}$$

其中 C_{in} 单位为 g/Nm³，Q 单位为 Nm³/h，故 L 单位为 kg/h。另构造电压平方和、振打频率 $60/T_i$ 与每周期积尘代理 $L_t T_i/3600$。所有聚类与回归参数仅在训练期（5 月 1—5 日）估计，验证期（5 月 6 日）与回顾性测试期（5 月 7 日）不参与拟合，划分按时间顺序而非随机打乱。

2.2 逐列可识别性结论

对 14 列逐一审计的结果见表 1：**出口浓度一列无法用于识别目标区间内的排放响应，其余各列则能为功率建模、工况分层或历史操作规律分析提供有效信息**。

表 1　附件逐列可识别性审计结论

数据块	审计证据	可用性
C_{out} 出口浓度	有效值 10030 条全部位于 48.74～50.00 mg/Nm³，5491 条恰等于 50；与全部 12 个变量的同期相关系数绝对值均 < 0.017，1～120 min 滞后同样无稳定关系；封顶分类 AUC=0.5098，未封顶子样本回归验证 $R^2 = -0.019$	**不可识别，不可修复**
P 总功率	由控制量近乎确定，残差标准差约 6 kW（见 §5.2）	精确可辨识
H, C_{in}, Q 工况变量	连续波动并含两段持续异常事件（§4.3）	工况分层与扰动分析
U_i, T_i 控制设定	与工况变量存在稳定相关结构（§4.2）	可提取历史操作规律

在对出口浓度作任何拟合之前，必须先确认一件事：这一列记录是否覆盖了题目要求的目标区间，以及它与控制量之间是否存在可学习的关系。若不先做这一步，任何回归模型都可能在事实上只是复现传感器读数。图 2 给出该列的分布与截顶诊断。已有研究曾用宽负荷下的机组负荷与各电场二次电流训练出口浓度模型 [8]；相比之下，本附件既没有二次电流，出口记录本身也几乎不变化，不具备同样的条件。附录 B 给出直接以 C_{out} 为目标建模的对照结果，可与其他方案交叉比较。

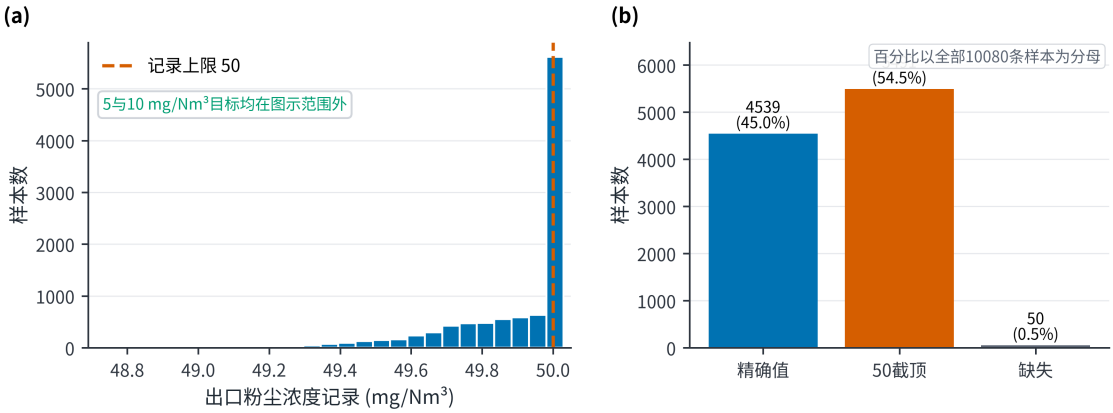

图 2　出口浓度数据诊断

图 2(a) 显示有效记录高度集中于 50 mg/Nm³ 附近，5 与 10 mg/Nm³ 的目标区间内**没有任何训练样本**；图 2(b) 显示恰好等于 50 的记录占 54.5%。结合表 1 中相关系数绝对值均小于 0.017 这一结果，可以得到一条完整的推论链：目标区间无样本，且控制量与出口记录之间不存在稳定关系，因此**无**

法用黑箱回归直接建立目标区间内的排放模型。这一判断直接决定了后文的分层结构——排放层只能改用显式标注的条件式灰箱模型（§4.4），而不能与功率层一样按数据拟合。

“截顶”是推断，须由可证伪的检验支持。上述证据的形式都是“没有找到关系”，而“没找到”与“不可能找到”并不等价。为把这一判断由经验陈述提升为可检验的结论，本文对截顶假设作如下检验：设真实浓度为潜变量 $C^{*} \sim N(\mu, \sigma^{2})$，记录值为 $\min(C^{*}, 50)$，仅用**未封顶的 4539 条**记录按截尾极大似然估计 (μ, σ)，再用估计值**预测**应有的封顶比例，并与实际封顶比例比较——后者未参与估计，因而是一次真正的外推检验。结果见表 2。

表 2　截顶假设与振打周期性的可证伪检验

检验	对象	统计量	参照值	结论
截尾正态极大似然	C_{out}	预测封顶比例 0.5461	实测 0.5475	潜变量 $N(50.036, 0.313^{2})$，预测与实测相差 0.0014
相位折叠（残差）	T_1, T_2（3.88 min）	折叠振幅 0.028 mg/Nm³	残差标准差 0.170	无与振打周期同步的成分
相位折叠（残差）	T_3, T_4（7.42 min）	折叠振幅 0.011 mg/Nm³	残差标准差 0.170	无与振打周期同步的成分
相位折叠（封顶率）	T_1, T_2 ／ T_3, T_4	χ^{2} 检验 $p = 0.142$ ／ 0.707	显著性 0.05	封顶率不随振打相位变化

由表 2 首行可得一个此前缺失的量化事实：**潜变量的标准差仅 0.313 mg/Nm³，为其均值的 0.63%**。也就是说，即便完全剥离截顶效应，这一列的真实变异也只有 ±0.6%，其中还包含仪表噪声；控制量在附件全部取值范围内引起的排放变化，无论多大，都不可能在这样的变异中被分辨出来。据此，§2.2 的判断由“本文未能找到关系”收紧为**“在该列的信息量下不可能找到关系”**，这是一个由数据本身支持的结论，而非建模选择。表 2 后三行属振打峰值的检验，将在 §4.5 一并讨论。

需要强调的是，以上结果只说明该记录不能提供可供学习的信号，不能据此反推物理变量对真实排放没有影响。作为对照，图 3 给出首 24 h 各过程量的时序，用以说明“信息量不足”是出口浓度一列的特有问题，而非整份数据的普遍状况。

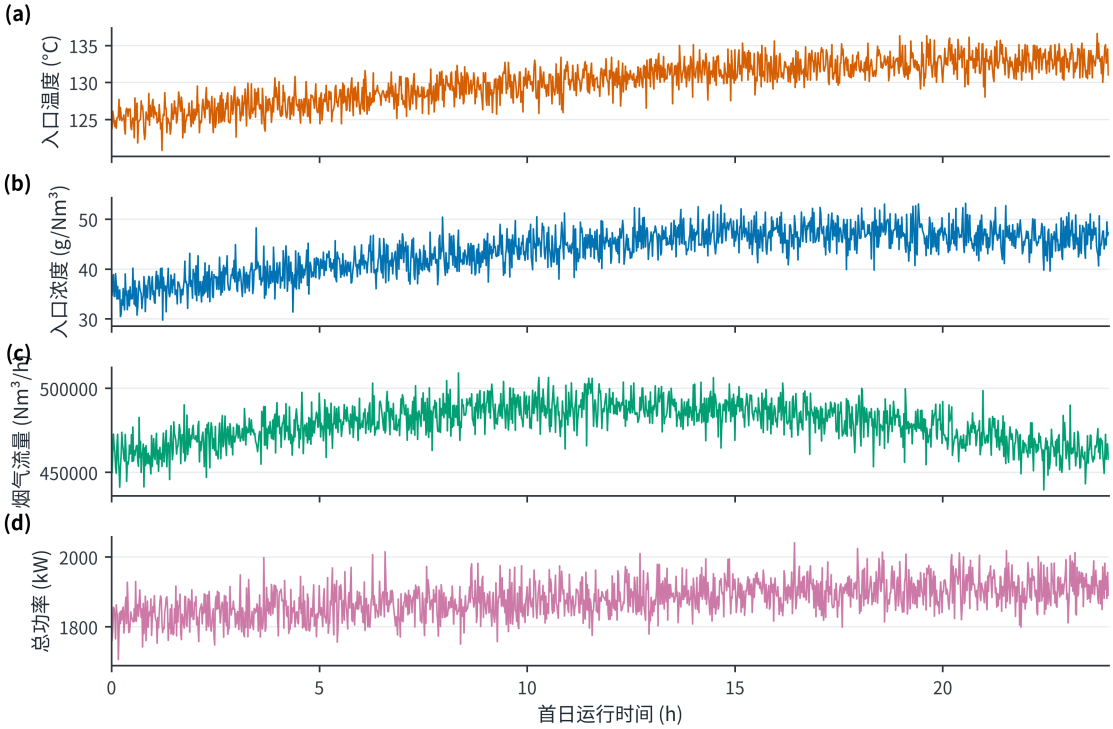

图 3　首 24 h 过程量时序

图 3 中入口温度、入口浓度、流量与总功率均呈现连续变化和工况迁移特征，波动幅度足以支

撑建模；与之相比，出口浓度记录几乎被压缩在单点附近。两者的反差说明：附件整体质量良好，问题仅出在出口浓度这一列，故后文对其余 13 列仍按实证方式使用。

2.3 排放锚点的取法

本文**不对出口记录作任何缩放**，直接把各工况出口浓度的记录中位数（都是 50 mg/Nm³）当作机理模型的起算点（下称锚点）。由此确立的问题定位是：设备当前距超低排放水平尚远，本文求解的是由现状降至 10 与 5 mg/Nm³ 的控制路径及其代价。这一取法带来三个后果，后文均予明示：最优点必然落在历史电压范围的上端（§5.5）；10→5 mg/Nm³ 的功率增幅明显小于从更低起点出发时的估计（§7）；排放层的数值仍属机理推演，须在补采未截顶的排放数据后重新标定（附录 C）。

3 模型假设、符号与总体框架

3.1 模型假设

本文将假设分为数据事实、情景假设、优化边界与运行约束四类，并逐条标注其证据状态，如表 3 所示。

表 3　模型假设及证据状态

类别	假设	证据状态
数据事实	7 天记录按分钟连续，除 50 条出口浓度缺失外不填造目标值	附件可核验
数据事实	总功率按 U_i^2 与 $1/T_i$ 的物理形式建模，按时间滚动验证	可由验证与测试结果检查
数据事实	排放锚点取出口浓度记录中位数 50 mg/Nm³，不作缩放	附件记录值
情景假设	去除指数按 $K = K_0\,S(U)$ 随电压幂次 p 缩放，中心取 $p = 2$	Deutsch 迁移速度形式，未经现场标定
情景假设	电场贡献权重 α_i 取等权、弱前级与中心前级三种形状	工程先验与消融
情景假设	振打峰值指数 $\gamma = 1$ 由积灰量正比关系推导，扫描 1.00/1.30/1.60	本文推导＋结构情景
优化边界	控制量位于各工况训练期实测最小—最大范围内	历史运行包络，不等同设备安全边界
优化边界	5 mg/Nm³ 档允许电压上界外推至历史最大值的 1.05 倍	显式声明的外推，须经容量核算
运行约束	前级电压高于后级至少 3 kV，后级振打周期长于前级至少 100 s	工程排序约束

3.2 主要符号

表 4　主要符号与单位

符号	含义	单位
U_i	第 i 电场电压	kV
T_i	第 i 电场振打周期	s
C_{in}, C_{out}	入口、出口粉尘浓度	g/Nm³、mg/Nm³
Q	烟气流量	Nm³/h
L	粉尘质量负荷	kg/h
$P, \widehat{P}$	实际、预测总功率	kW
b_i, c_i	电场功率系数、振打功率系数	kW/kV²、kW·s
K, K_0	去除指数及其历史锚点值	—
$S(U)$	电压强度因子，$U = U^{ref}$ 时为 1	—
p	去除指数对电压的幂次，$K \propto U^p$	—
$B(T)$	振打引起的峰值增量代理	mg/Nm³
ρ, γ	振打相位叠加尺度、峰值—周期指数	—

3.3 总体技术路线

前两节分别给出了假设与符号，但四问之间如何衔接、哪些结论可由数据复算、哪些只能条件式采信，仍需一张图整体说明。图 4 给出全文技术路线。

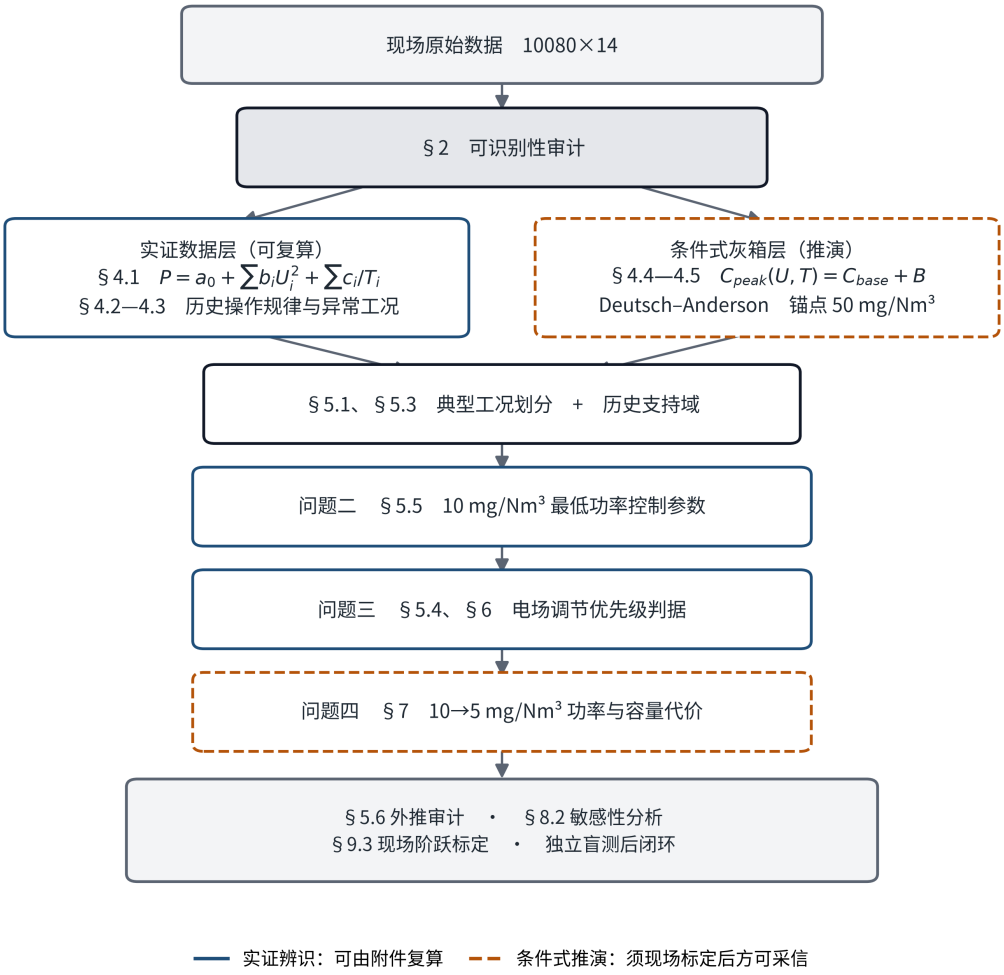

图 4　全文技术路线

图 4 的关键在于 **§2 的可识别性审计把论证分成了两条证据强度不同的路径**：实线一侧（实证数据层）的功率律、历史操作规律与异常工况均可由附件直接复算；虚线一侧（条件式灰箱层）的排放模型只能由机理骨架加显式先验构成，须经现场标定后方可采信。两条路径仅在“典型工况＋支持域”处合并，此后四问的结论按其所属路径分别标注：问题二、三的功率与优先级结论以实证层为主，问题四的排放代价则属条件式推演。图末的外推审计、敏感性分析与现场标定构成结论的适用边界，说明本文给出的是可检验的试验起点而非可直接下发的设定值。

4　问题一的模型建立与求解：影响关系与振打峰值

第一问要求解释各变量与出口浓度的关系，并分析振打峰值。**本节所用模型与方法为**：逐列可识别性审计（相关分析、滞后相关、截顶诊断与对照建模）、物理形式最小二乘功率模型、历史操作规律的相关分析、Deutsch–Anderson 灰箱骨架，以及由积灰质量守恒推导的振打峰值标度关系。按 §2 的审计结论，本节分两层作答：先给出附件**能够**支持的三类实证结论（§4.1—§4.3），再给出只能靠机理推演的排放部分（§4.4—§4.5）。

4.1 总功率的生成律（实证）

在训练期以物理形式拟合，得到

$$P = 73.09 + \sum_{i=1}^{4} b_i U_i^2 + \sum_{i=1}^{4} \frac{c_i}{T_i} \tag{2}$$

其中 $b = (0.0849, 0.0851, 0.0847, 0.0853)$ kW/kV²、$c = (5.43, 5.37, 5.49, 5.26) \times 10^4$ kW·s。四个电场的 b_i 彼此相差不到 0.8%，说明四台高压电源的电气特性一致；写成合并形式即 $P \approx 73.0 + 0.0850 \sum U_i^2 + 5.39 \times 10^4 \sum T_i^{-1}$。该式的物理含义明确：**电场功率与电压平方成正比，振打功率与振打频率 $60/T_i$ 成正比**。模型的检验结果见 §5.2。

由该式还可直接得到一项此前未被注意的结果。在训练期的中位设定下，$\sum b_i U_i^2$ 为 1020.3 kW，$\sum c_i/T_i$ 为 693.7 kW，即**与振打频率同步变化的那部分电耗占总功率的 38.8%**。振打周期虽非题面强调的主要电气变量，但其对电耗的贡献与电压同一量级，不可忽略。相应地，把四个电场的周期从中位延长到历史 P95 可省下 56.1 kW，延长到历史最大值可省下 109.1 kW，代价是单次积灰量分别增加 8.4% 和 17.9%（图 6b）。§7.3 将说明，该可回收功率与排放限值收紧一档所增加的功率代价处于同一量级——这正是振打周期不宜为节电而无限延长的定量依据。

这里须对 $\sum c_i/T_i$ 一项的物理归属作出限定。**该项的量级远超振打机构本身**：电除尘器的振打锤由小功率电机或电磁装置驱动，单台额定通常不足 1 kW，四电场合计与 693.7 kW 相差两到三个数量级。同一量纲上，附件的总功率相当于 3.84 kWh/千 Nm³，也显著高于电除尘器 0.1～0.4 kWh/千 Nm³ 的常见水平。因此本文只把 c_i/T_i 读作**“随振打频率同步变化的那部分厂用电”**（可能包含清灰关联的辅机负荷，或数据本身的生成结构），而**不**把它解释为振打锤的机械功率。这一限定不影响后文的任何数值：优化只用到 $\partial P/\partial T_i = -c_i/T_i^2$ 这一经验关系，其在附件上的外推精度已由 §5.2 的测试日结果检验；受影响的仅是该项的机理命名。

此外，把入口温度、浓度、流量也加入该式后，验证集与测试集的 R^2 变化都不到 10^{-3}（见表 10 末行）。**在现有数据与模型精度下，入口温度、入口浓度与烟气流量对总功率未显示出额外解释力**。后文按工况分别优化所得的功率差异，全部来自各工况不同的排放锚点与取值范围，而非工况本身直接作用于功率。

4.2 历史操作规律（实证）

出口浓度不可识别，并不意味着附件不含操作策略方面的信息。图 5(c) 给出整周记录中工况变量与控制设定的相关系数，其中两类规律值得报告：

规律一（稳定）：入口浓度升高时，四个电场的振打周期一律缩短（$r = -0.208$、-0.211、-0.277、-0.278）。该方向在训练期、验证期与测试期三个时窗内符号一致，是附件中可复现的操作规律，与“高负荷下不宜延长周期”的工程直觉相符。

规律二（不稳定）：入口浓度升高时，前两个电场的电压随之升高（整周 $r = +0.172, +0.166$），后两个电场基本不变（$r = +0.019$）。但该规律并不贯穿整周：$r(C_{in}, U_1)$ 在前 5 天为 -0.052，第 6、7 天分别为 $+0.507$ 与 $+0.492$。即入口浓度升高时抬升前级电压这一做法，仅出现于最后两天。本文如实报告该非平稳性：前后级响应的不对称在整周与最后两天均成立，却不能视为贯穿全周的稳定规律。这也表明历史数据中至少发生过一次操作策略调整，故第三问只将其作为参照，而非定论。

表 5　工况变量与控制设定的历史相关系数（全记录期）

控制量	与入口温度 H	与入口浓度 C_{in}	与烟气流量 Q
U_1	−0.130	+0.172	+0.074
U_2	−0.127	+0.166	+0.070
U_3	−0.112	+0.019	−0.068
U_4	−0.113	+0.019	−0.063
T_1	−0.075	−0.208	−0.180
T_2	−0.064	−0.211	−0.175
T_3	−0.159	−0.277	−0.130
T_4	−0.151	−0.278	−0.135

4.3 两段持续异常工况（实证）

出口浓度虽不可识别，但附件仍可能保留控制策略、异常工况响应与功率响应等信息，因此有必要对其余变量继续分析；§4.2 的相关结构与本节的两段异常工况合并呈现于图 5。附件含两段持续

数十分钟以上的异常工况，其控制端响应方向相反，构成理解设备行为的两个直接入口（图 5a、5b）。

表 6　两段持续异常工况及控制端响应

事件	时间窗	特征	控制端响应
入口浓度尖峰	05-02 14:02—15:49，其中 41 min 超过 60 g/Nm³	入口浓度峰值 72.47 g/Nm³（基线中位 37.41）	U_1 均值 63.7 kV（峰值 67.8），振打周期同步缩短，总功率均值 1924 kW、峰值 2020 kW
入口温度尖峰	05-06 13:00—14:59，持续 120 min	入口温度峰值 158.2 °C（基线中位 126.5），同期入口浓度降至均值 25.9 g/Nm³	U_1 均值降至 55.5 kV，总功率均值降至 1668 kW

第二段事件在工程上更值得注意：**140～160 °C 正是水泥窑尾粉尘比电阻最高的温度区间**。比电阻过高会引发反电晕，使粉尘向极板迁移的速度下降、火花率上升，是电除尘最难运行的一段工况[3-4]。附件在这段时间里的记录显示，操作端选择了降压而不是升压，这与“高比电阻区应限制电流密度以抑制反电晕”的常规处置一致；但附件没有二次电流和火花率记录，无法判断这是否为最优处置。由此可得两点：第一，第二问的工况划分把温度作为主轴之一是必要的；第二，本文四类工况的温度中心约为 120.7 与 130.4 °C，**都没有覆盖这段高比电阻区间**，因此本文结论不能外推到该区间，须单独补采数据重新标定（§9.3）。

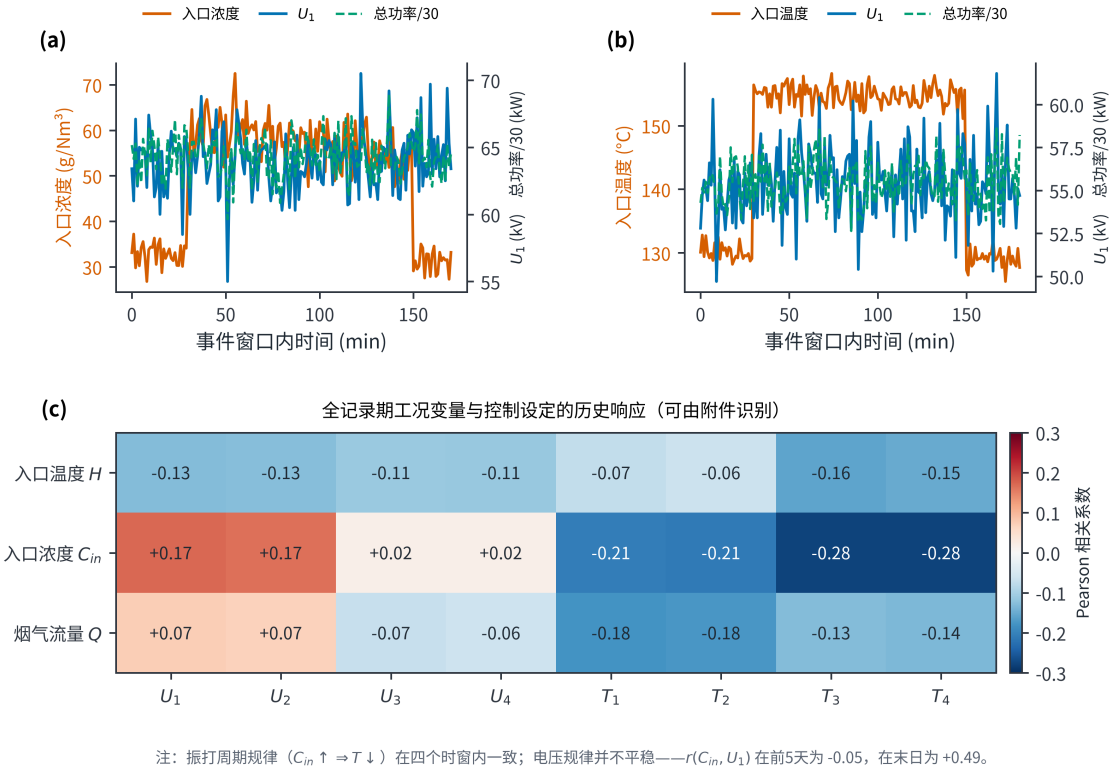

图 5　附件中可识别的运行证据

综合图 5 可得三点。其一，**稳定的规律**是图 5(c) 中入口浓度与四场振打周期的负相关（$r = -0.208 \sim -0.278$），该方向在四个时窗内一致。其二，**不稳定的规律**是入口浓度与前两级电压的正相关，它只在最后两天成立（§4.2）。其三，图 5(a)(b) 的两段事件说明控制端对不同扰动的处置方向相反：入口浓度尖峰时抬压增功率，入口温度尖峰时反而降压降功率。

必须指出，以上全部是**历史操作留下的相关关系**，只能说明操作者当时如何响应工况，不能直接翻译为控制量如何影响排放的因果结论——因为排放侧的响应本身不可识别（§2.2）。故第三问只把它作为经验对照，判据本身仍由模型结构导出（§5.4）。

4.4 排放响应的灰箱骨架（条件式推演）

Deutsch–Anderson 关系用指数形式描述电除尘效率。它的理想化形式忽略了再飞扬、气流分布和粒径变化，因此只能当作**骨架**：形式取自机理，系数则须另行给定（本文称“灰箱”）[2-4]。White 的经典专著 [12] 与 Parker 的电除尘器电气运行专著 [13] 都强调，电气运行、粉尘性质与振打三者是相互耦合的。定义去除指数

$$K = \ln \frac{1000 C_{in}}{C_{out}}, \qquad C_{out} = 1000 C_{in} e^{-K} \tag{3}$$

K 越大表示除尘越彻底：K 每增加 $\ln 2$，出口浓度就减半。对工况 k，以历史中位电压 $U_{i,k}^{ref}$ 和出口浓度记录中位数 $C_{0,k} = 50$ mg/Nm³ 为起算点，得 $K_{0,k} = \ln(1000 C_{in,k}/C_{0,k})$，四类工况依次为 6.257、6.514、6.702、6.797。粉尘向极板迁移的速度与电场强度的平方近似成正比，故取

$$K_k(U,T) = K_{0,k}\, S_k(U) - \{G_k(T) - G_k(T^{ref})\}, \qquad S_k(U) = \frac{\sum_i \alpha_i \left(U_i / U_{i,k}^{ref}\right)^p}{\sum_i \alpha_i} \tag{4}$$

$$G_k(T) = \sum_{i=1}^{4} \beta_{i,k} \left(\frac{T_i}{T_{i,k}^*} + \frac{T_{i,k}^*}{T_i} - 2 \right), \qquad \beta_{i,k} = \beta_i \left(\frac{L_k}{L_{med}} \right)^{1/2} \tag{5}$$

这样构造有一个好处：当 $U = U^{ref}$、$T = T^{ref}$ 时式子严格还原为 $K = K_{0,k}$，即**模型天然复现历史运行点**，因此不再需要一个额外的整体比例参数 η，全部电压敏感性都归结到一个幂次 p 上（中心取 $p = 2$）。$G_k(T)$ 在 $T_i = T_{i,k}^*$ 处取最小值，表示周期过短或过长都不利：过短会频繁引起再飞扬，过长则积灰加重、单次脱落量变大。试验研究也确实观察到再飞扬与最大允许周期之间存在这种权衡 [6]。再加一点安全裕度 s，连续排放的基础值为 $C_{base,k} = 1000 C_{in,k} \exp[-K_k(U,T) + s]$。

本节全部排放系数均为工程先验，不是由附件出口浓度估计得到的参数。

表 7　灰箱排放模型参数来源与证据状态

参数	中心取值	含义	来源	是否由附件识别
p	2.0	去除指数对电压的幂次	Deutsch 迁移速度形式	否
α_i	(1.15,1.10,0.95,0.80)	电场去除贡献权重	工程先验与消融	否
β_i	(0.30,0.27,0.20,0.17)	周期偏离损失	工程情景	否
b_0	1.2	峰值尺度	工程情景	否
γ	1.0	周期—峰值指数	本文由积灰量推导	否
ρ	1.0	相位叠加尺度	同相保守情景	否
s	0.08	安全裕度	工程情景	否

4.5 振打峰值：由积灰量推导而非假定

附件只给出振打周期的设定值，没有实际振打时刻、相位和秒级浓度，所以逐次峰值本身无法辨认。但峰值**随周期怎样变化**可以由质量守恒直接推出，不必假定指数：设单位时间的积灰速率正比于粉尘负荷 L，那么一个周期内积下的灰饼质量就正比于 $L \cdot T$；一次振打抛落的粉尘量又正比于灰饼质量，于是

$$B_k(T;\rho,\gamma) = b_0 \rho \left(\frac{L_k}{L_{med}} \right) \sum_{i=1}^{4} w_i \left(\frac{T_i}{T_{i,k}^*} \right)^{\gamma}, \qquad \gamma = 1 \tag{6}$$

振打周期是本题中唯一同时牵动两个结果量的控制量：缩短周期提高振打频率、直接增加电耗，延长周期虽省电却使每次抛落的灰量增大。由上式可得两条互补的结论，也正是第一问关于振打峰值的完整回答：

- **单次瞬时峰值随周期近似线性放大**：周期延长 10% 与 20% 时，单次峰值代理相应放大 10% 与 20%；
- **对小时均值的贡献在一阶近似下与周期无关**：在单位面积灰量所对应的再飞扬比例近似不随振打周期变化这一前提下，单位时间的总再飞扬量正比于 $L \cdot T \times (1/T) = L$，只取决于粉尘负荷。该结论是上述前提下的一阶近似，而非无条件成立的物理定律；若周期显著延长使灰饼结构改变、再飞扬比例本身随之变化，该近似即不再适用。

即延长振打周期并不改变再飞扬对小时均值的贡献，只是把等量粉尘集中于次数更少、单次幅度更高的峰值中释放。这条结论有两个直接用途：一是解释了为什么“延长周期省电”必须和瞬时峰值一起考虑（§7.3）；二是由于现行标准按小时均值考核，本文把瞬时峰值直接卡到限值的做法偏**保守**（§7.4）。中心情景取 $\rho = 1$，即假设四个电场同时振打、峰值完全叠加，这是最不利的上界；若错开相位则取 $\rho = 0.80$ 或 0.65。敏感性分析仍扫描 $\gamma \in \{1.00, 1.30, 1.60\}$，以检验推导值附近是否稳健。

峰值幅度虽不能逐次辨认，却可以给出上界。“无法辨认逐次事件”与“无法检验峰值是否存在”是两回事：前者成立，后者不成立。四个电场的设定周期中位数为 3.88 min（T_1, T_2）与 7.42 min（T_3, T_4），**都大于 1 min 采样对应的 2 min 奈奎斯特周期**，因此与振打同步的周期性成分是可检出的。据此本文补作两项检验（结果已列入表 2）：

- **对出口浓度残差作相位折叠**：按各电场设定周期把残差折叠为 10 个相位区间，T_1, T_2 的折叠振幅为 0.028 mg/Nm³，T_3, T_4 为 0.011 mg/Nm³，而残差标准差为 0.170 mg/Nm³，均无高出噪声的相位结构；
- **对封顶率作相位折叠**：振打峰值使浓度升高，而升高恰是 50 mg/Nm³ 截顶所抹去的方向，因此单看记录值会低估峰值。改检验“哪些分钟被封顶”是否随相位变化，χ^2 检验 $p = 0.142$ 与 0.707，同样无相位依赖。

由此可得一条由附件直接支持的定量结论：**在分钟级分辨率下，与振打周期同步的排放波动幅度不超过约 0.03 mg/Nm³**。须同时说明该上界的适用范围：它约束的是能在分钟平均后残留下来的周期性成分，对持续数秒、在分钟平均中被摊薄、且其升高方向又被截顶抹去的单次尖峰并不构成约束。因此该检验**不能否定**峰值的存在，只能给出其可检出部分的上界；§4.5 的标度关系仍由质量守恒推导，b_0 与 ρ 仍待现场以秒级未截顶数据标定。但这一检验至少排除了一种情形——附件中并不存在一个大到可由分钟数据直接辨认的振打峰值信号。

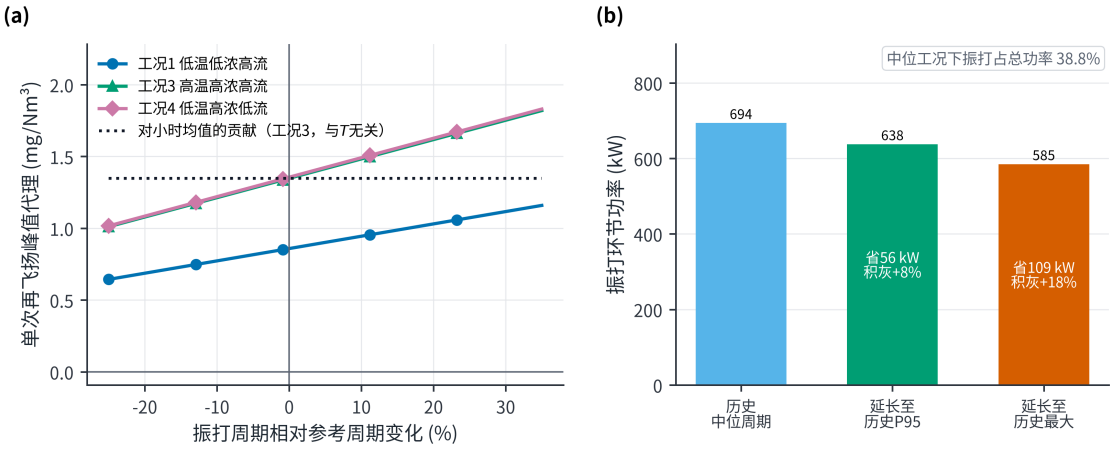

图 6　振打峰值机理与振打功率权衡

图 6(a) 给出周期延长时单次峰值的线性放大，图 6(b) 给出同一动作在功率侧的收益：周期由中位延长至历史极值可回收约 109 kW。二者合起来给出本题的一条关键判断——**振打周期并非越长越**

省电越好，而是在电耗与瞬时峰值风险之间取舍的权衡变量。该权衡在第四问中被定量使用：§7.3 将说明振打侧可回收的功率与排放收紧一档的代价处于同一量级，但其额度受峰值裕度限制。

4.6 各因素方向性小结

在上述骨架中，其他条件不变时各因素的作用方向如下。提高 U_i 会增大 K、降低连续排放，但总功率按 U^2 上升。K 不变时，入口浓度升高会使出口浓度同比升高。流量增大会缩短烟气在电场中的停留时间、同时抬高粉尘负荷，因而需要更强的捕集能力。温度则通过气体性质、粉尘比电阻和放电状态影响除尘效果，其中 140～160 °C 的高比电阻区间最为不利（§4.3）。这些方向来自机理与文献 [2-4]，不是从附件回归出来的。为便于对照，表 8 把第一问涉及的五类因素、它们对排放与电耗的作用及各自的证据性质汇总如下。

表 8　各因素对排放与电耗的作用及证据性质

因素	对排放的主要作用	对电耗的作用	证据性质
入口浓度 C_{in}	抬高排放压力：固定 K 时出口浓度同比升高	不直接进入功率模型	机理＋历史操作数据（§4.2）
烟气流量 Q	缩短有效停留时间、抬高粉尘负荷，需更强捕集	未显示额外解释力	工程机理
入口温度 H	经粉尘比电阻与放电状态影响除尘，140～160 °C 最不利	无直接功率项	文献 [3-4] ＋异常事件（§4.3）
电场电压 U_i	提高去除指数 K、降低连续排放	功率按 U_i^2 上升	排放灰箱（推演）＋功率实证
振打周期 T_i	过长抬高单次峰值，偏离最优区间增大排放风险	功率按 $1/T_i$ 下降	机理推导＋功率实证

该表同时显示了本文分层的必要性：电耗一列全部有实证支撑，排放一列则以机理与文献为主，二者的证据强度并不对等。

5　问题二的模型建立与求解：工况划分与最低功率

第二问要求划分典型工况，并在出口浓度不超过 10 mg/Nm³ 的约束下使总电耗最低。**本节所用模型与方法为**：以入口三变量作 K-means 聚类并由轮廓系数与 Calinski–Harabasz 指标选择簇数（§5.1）；以四种设定的滚动时间外推比较确定功率模型形式（§5.2）；以功率模型为目标函数、以灰箱排放式与历史支持域为约束建立 8 变量非线性规划，用多起点 SLSQP 确定性求解并以随机搜索对照（§5.3）；由模型结构求导得出电场优先级的解析判据（§5.4）；最后以马氏距离作外推审计（§5.6），并逐档收紧限值得到排放—功率权衡前沿（§5.7）。

5.1 工况聚类

聚类只用入口温度、入口浓度和烟气流量三个变量——它们由上游工艺决定，不受控制策略影响，因此不会出现“用结果去定义工况”的问题。三者标准化后作 K-means 聚类 [9]。取 $k = 2, 3, 4, 5, 6$ 比较，轮廓系数 [10]（衡量类内紧凑程度与类间分离程度，取值越大越好）依次为 0.2963、0.3682、0.4084、0.3958、0.3934，在 $k = 4$ 最高。另一项 Calinski–Harabasz 指标依次为 3204.2、4399.0、4721.2、4835.5、4712.0，最大值却出现在 $k = 5$，两项判据并不一致。

两项指标的分歧是本节必须交代的问题，图 7 把它并列呈现。

图 7(a) 显示轮廓系数在 $k = 4$ 取最大值，图 7(b) 显示 Calinski–Harabasz 指标却在 $k = 5$ 取最大值。**两项统计判据结论不一致，因此 $k = 4$ 并非唯一的“数学最优”聚类数**。本文最终取 $k = 4$，依据有四条：其一，轮廓系数在 $k = 4$ 最高；其二，四类中心恰好构成温度两档（约 120.7 与 130.4 °C）与流量两档（约 44.0 与 48.1 万 Nm³/h）的 2×2 组合，物理含义清楚；其三，四类比五类更便于控制解释与后续按工况优化；其四，由此本文把这四类理解为**统计意义上的工况分层，而非唯一的物理状态划分**。此外如 §4.3 所述，两段异常工况都落在四类中心之外，不被该分层覆盖。

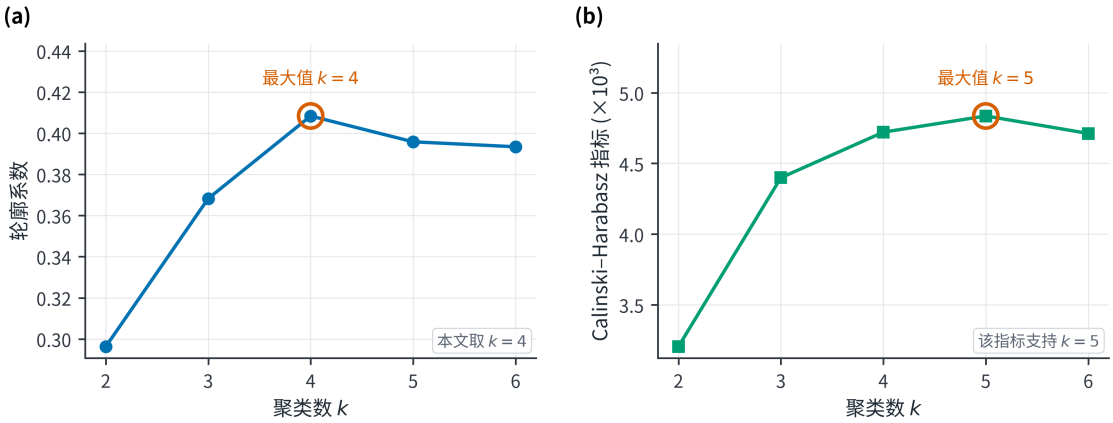

图 7　聚类数选择结果

在确定 $k = 4$ 之后，图 8 给出四类工况在入口浓度—温度平面上的实际分布，用以核对分层结果是否与工况的物理含义相符。

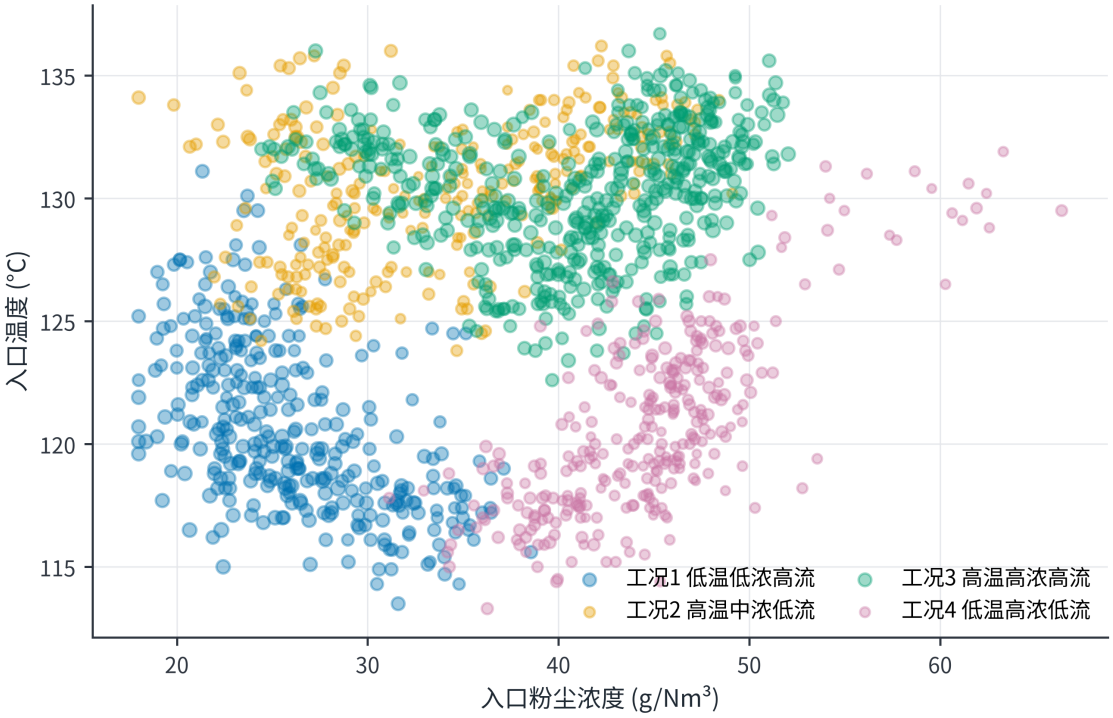

图 8　训练期典型工况分布

图 8 中四类样本在浓度—温度平面上分区清晰，点的大小（流量）进一步区分了同温度带内的两类工况，与上述 2×2 结构一致。四类工况的统计画像见表 10，其中的历史功率与锚点 K_0 将分别用于 §5.5 的对照基准与 §4.4 的排放起算点。

5.2 功率模型的设定与检验

§4.1 采用的形式并不是唯一可选的。为说明“选什么形式”本身有多重要，本文在同一时间划分下比较四种设定：只用均值的基线、电压一次项＋周期一次项的岭回归 [11]、电压二次项＋周期一次项的岭回归，以及本文的 $U_i^2 + 1/T_i$ 形式。岭回归的正则强度按训练期 3 个逐步加长的窗口滚动选择，本文形式则没有可调超参数。结果见表 10。

上表只回答了“哪种设定更好”，接下来的三幅图分别回答三个递进的问题：图 9 回答最终模型预测得有多准，图 10 回答为什么必须把振打周期写成 $1/T$，图 11 回答改正之后残差是否还残留系统性结构。

表 9　四类典型工况画像

工况	占比	温度/°C	入口浓度/(g/Nm³)	流量/(Nm³/h)	粉尘负荷/(kg/h)	历史总功率/kW	锚点 K_0
1 低温低浓度高流量	23.35%	120.65	26.08	478257	12460	1818.35	6.257
2 高温中浓度低流量	20.06%	130.32	33.72	443591	14922	1870.89	6.514
3 高温高浓度高流量	34.93%	130.41	40.71	480851	19560	1763.08	6.702
4 低温高浓度低流量	21.67%	120.69	44.75	439990	19673	1696.55	6.797

表 10　功率模型设定比较与时间外推结果

设定	验证 R^2	验证 RMSE/kW	测试 R^2	测试 RMSE/kW	测试平均偏差/kW
均值基线	−5.3259	140.18	−0.3733	83.51	−43.54
电压一次＋周期一次（岭）	0.9798	7.93	0.9180	20.41	−16.65
电压二次＋周期一次（岭）	0.9837	7.12	0.9571	14.76	−10.78
本文 $U_i^2 + 1/T_i$	**0.9885**	**5.97**	**0.9930**	**5.98**	**−0.02**
本文设定＋工况变量	0.9885	5.97	0.9929	5.98	+0.08

由表 10 可得三点结论。

第一，**该系统性偏差来自模型设定误差，而非数据漂移**。线性 T 的设定在测试日留下 −10.78 kW 的平均偏差，而且偏差在高功率区还会扩大；改成 $1/T$ 后，偏差降到 −0.02 kW，测试 RMSE 从 14.76 kW 降到 5.98 kW，R^2 从 0.957 升到 0.993（图 10）。若沿用线性形式，该偏差很容易被误判为数据本身的问题而额外计入误差预算；实际上，它主要由错误的函数形式引入。

第二，T 是优化中的决策变量。线性形式意味着周期每延长单位时间所节省的功率恒定，而真实的边际节电按 c_i/T_i^2 衰减，即周期越长、再延长的收益越小。因此线性周期设定会使最优振打周期系统性地偏离真实值。

第三，再加入工况变量后各项指标不再改善，印证了 §4.1 的判断：在现有数据精度下，工况变量对总功率没有额外解释力。

验证集上绝对误差的 90% 分位数为 9.78 kW，本文把它作为做功率预算时的余量。它不改变各方案的优劣排序，也不保证在今后的日期里仍能覆盖 90% 的情形。

首先确认最终模型的整体预测水平。

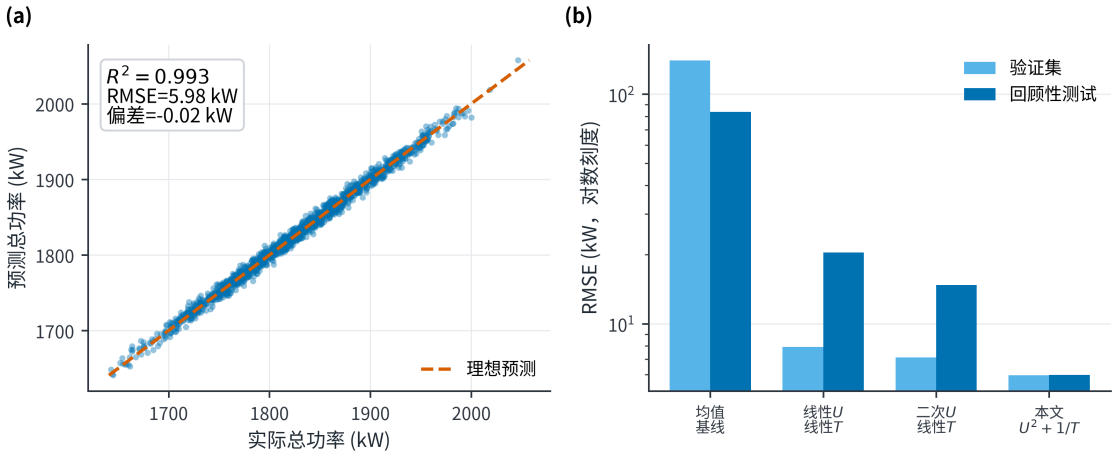

图 9　功率模型预测结果

图 9(a) 中测试日的预测值紧贴理想线，图 9(b) 以对数纵轴比较四种设定的 RMSE：本文设定在

验证与测试两段均最低，且是唯一在测试段不发生劣化的设定。这确认了 $P = a_0 + \sum b_i U_i^2 + \sum c_i / T_i$ 可以作为后续优化的目标函数使用。

预测精度高并不等于设定形式正确。图 10 进一步给出本文修正设定形式的依据。

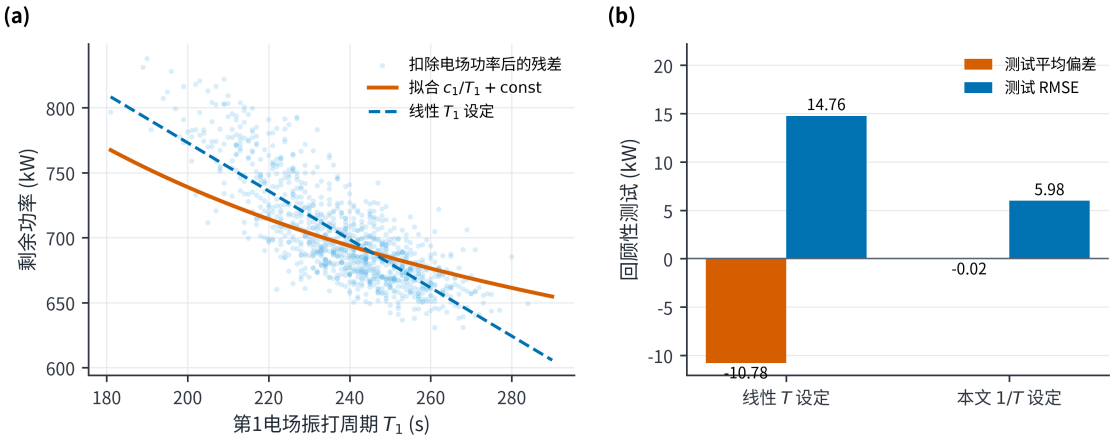

图 10　振打周期的设定形式对比

图 10(a) 把扣除电场功率后的剩余功率对 T_1 展开：数据呈明显的双曲形状，线性设定（虚线）在周期的两端系统性偏离。图 10(b) 则给出后果——线性 T 设定在测试段留下约 −10.8 kW 的平均偏差，改为 $1/T$ 后降至约 0。**这是本文修正模型设定的关键证据：最初的系统性负偏差并非数据漂移，而主要由 T 的函数形式设定错误引起**。若不作此修正，该偏差会被误计入误差预算，且会使 §5.3 优化中的最优周期系统性偏移。

修正是否彻底，需由残差结构本身确认。

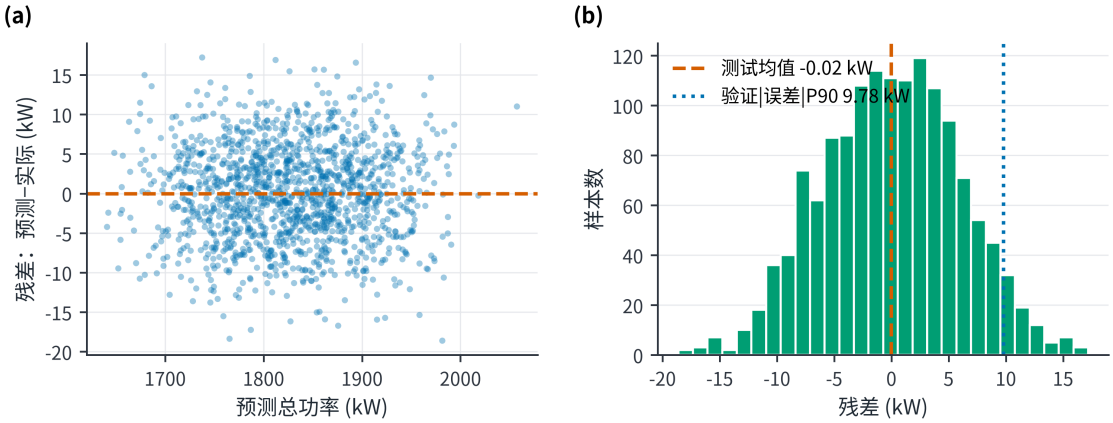

图 11　回顾性测试残差诊断

图 11(a) 显示残差围绕零线对称散布，原先在高功率区扩大的负偏差已消失；图 11(b) 的残差分布以零为中心，测试均值 −0.02 kW。至此三幅图形成完整链条：设定比较（表 10）→ 整体精度（图 9）→ 形式修正的依据（图 10）→ 修正后无残留结构（图 11），据此确定本文的功率模型为 $P = a_0 + \sum b_i U_i^2 + \sum c_i / T_i$。

5.3 优化模型

设控制向量 $z = (U_1, \ldots, U_4, T_1, \ldots, T_4)$。对每类工况 k 与限值 C_{lim} 求解

$$\min_{z} \ \widehat{P}(z) = a_0 + \sum_i b_i U_i^2 + \sum_i \frac{c_i}{T_i} \tag{7}$$

$$\text{s.t.}\quad C_{base,k}(z)+B_k(z)\le C_{lim},\quad U_1\ge U_3+3,\ U_2\ge U_4+3,\ T_3\ge T_1+100,\ T_4\ge T_2+100,\quad z\in\Omega_k \tag{8}$$

取值范围 Ω_k 取该工况训练期内各控制量实际出现过的最小值到最大值。5 mg/Nm³ 一档另外允许电压上界放宽到历史最大值的 1.05 倍，放宽了多少在表 13 中逐条列出。已有的多电场节能研究同样采用“排放受限、分场优化”的思路 [5,7]，但它们的排放模型都由现场实测标定，条件与本文不同，方法对照见附录 A。

关于求解方法，此处说明本文未采用凸优化求解器的原因。以电压 U 为变量时，目标函数 $\sum b_iU_i^2$ 本身是凸的，但排放约束要求这样一个凸函数取值**足够大**——这类约束把可行域挖成了非凸的形状，整体不是凸规划。若改用 $v_i=U_i^2$ 作变量，目标和排放约束都变成线性，可惜排序约束 $U_1\ge U_3+3$ 在 v 下不再是凸集。即在任一坐标下均无法化为凸问题，故不宜宣称凸性。

本文因此采用**多起点 SLSQP 确定性求解**：每个“工况—限值”组合从 53 个起点分别求解（5 个有结构的起点：历史中位、上界、下界、中点、T^* 点；再加 48 个随机起点），以检验各起点是否收敛至同一解。此外，在同一可行域内均匀随机抽取 4×10^5 个候选点作随机搜索，用作独立对照（§8.1）。这一做法取代了早期版本的百万级随机采样：问题结构清楚之后，确定性求解既更快，也更容易复算。

5.4 电场优先级的解析判据

在给出数值解之前，先由模型结构推导各电场的调节优先级。把去除指数与电场功率

$$K_k=K_{0,k}\,\frac{\sum_i \alpha_i\left(U_i/U_{i,k}^{ref}\right)^2}{\sum_i \alpha_i},\qquad P_{field}=\sum_i b_iU_i^2 \tag{9}$$

同时对 U_i^2 求导，得到**单位电功率的去除指数增益**

$$\left.\frac{\mathrm{d}K}{\mathrm{d}P}\right|_i=\frac{K_{0,k}\,\alpha_i}{\left(\sum_j \alpha_j\right)b_i\left(U_{i,k}^{ref}\right)^2} \tag{10}$$

该比值越大，表示该电场单位功率带来的除尘收益越高。须说明，这是**灰箱模型内部的条件式边际判据**：功率系数 b_i 由附件辨识，而去除贡献权重 α_i 取自工程先验，其稳健性由 §8.2 的场权重消融进一步检验；该排序并非由实测出口浓度直接证实的物理事实。由于四个电场的 b_i 几乎相同，排序完全取决于 $\alpha_i/(U_{i,k}^{ref})^2$，而**不是**取决于 α_i。原因在于功率按电压平方增长，参考电压较低的电场按相同比例抬升时，所需电功率显著更少。代入中心权重与各工况的历史中位电压，得到表 11。可以看到，第三电场的去除权重 $\alpha_3=0.95$ 虽然低于前两级，但它的参考电压低约 10 kV，平方之后使它的单位功率收益在四类工况中**都排第一**。

表 11　电场优先级解析判据（$\times10^{-3}$ kW^{-1}）

工况	电场 1	电场 2	电场 3	电场 4	优先次序
1 低温低浓度高流量	5.768	5.504	**6.668**	5.573	E3 > E1 > E4 > E2
2 高温中浓度低流量	5.556	5.269	**7.023**	5.870	E3 > E4 > E1 > E2
3 高温高浓度高流量	6.630	6.327	**8.024**	6.706	E3 > E4 > E1 > E2
4 低温高浓度低流量	7.551	7.207	**7.717**	6.502	E3 > E1 > E2 > E4

这也解释了本文早期版本“前级优先”这一结论的真实来源：**它是排序约束 $U_1\ge U_3+3$ 带出来的，而不是权重 α_i 决定的**。由于抬升 U_3 必须同时抬升 U_1，该排序约束使二者的调节之间存在耦合，故最优解中 U_1 与 U_3 总是同步走高。若排除该约束，单位功率收益最高的仍是第三电场。

振打侧同理：$\partial P/\partial T_i=-c_i/T_i^2$。前级周期短（中位 240 s），后级周期长（中位 441 s），故**在**

相同的周期延长量下，前级的节电量约为后级的 3.4 倍。这与电压侧由前级承担主要捕集并不矛盾：电压用于提高捕集能力，周期用于降低电耗，二者的最优着力点分别位于不同电场。

5.5 10 mg/Nm³ 下的最优控制参数

四类工况的中心情景解如表 12（5 mg/Nm³ 行为第四问提供参数）。

表 12　四工况两档目标的最优控制参数

工况	目标/(mg/Nm³)	U_1/kV	U_2/kV	U_3/kV	U_4/kV	T_1/s	T_2/s	T_3/s	T_4/s	峰值排放/(mg/Nm³)	预测总功率/kW
1	10	70.2	68.6	57.6	57.4	290	289	492	493	9.948	2045.55
2	10	70.9	69.8	59.2	58.2	280	278	494	498	9.962	2104.30
3	10	68.0	60.3	56.3	57.3	275	280	506	510	9.971	1924.67
4	10	70.5	61.8	57.2	42.9	285	283	495	498	9.991	1851.15
1	5	73.7	71.6	60.5	60.3	290	289	492	493	4.993	2182.25
2	5	74.4	72.9	62.2	61.1	280	278	494	498	4.998	2245.51
3	5	71.4	63.4	59.1	60.4	275	280	506	510	4.991	2056.06
4	5	74.0	66.4	60.1	42.9	285	283	495	498	4.988	1973.10

第二问的答案：在 10 mg/Nm³ 下，四类工况的最低总功率为 1851.15～2104.30 kW，按训练期各工况占比加权为 **1973.0 kW**，折合日电耗 47.35 MWh。作为对照，历史实际加权功率是 1783.2 kW（日电耗 42.80 MWh）。也就是说，达到 10 mg/Nm³ 意味着**功率上升 10.6%、日电耗多用 4.55 MWh**。

这一方向是必然的：历史排放约 50 mg/Nm³，将出口浓度压至其五分之一必须提高捕集能力，而在线可调的手段只有电压，电压又按平方计入功率。因此，**本文所称“节能”是指在同一排放目标下相对其他可行控制方案的功率降低，而非相对历史运行工况的降低**。

解的结构与表 11 的判据一致。振打周期 T 在两档下都被推至历史区间上端——事实上是**恰好取在上界**，属角点解，其成因与影响见 §5.6.1；电压则按单位功率收益由高到低分配——工况 3 与工况 4 中 U_2 明显低于 U_1，工况 4 的 U_4 更被压至历史下界 42.9 kV，正是由于该电场在此工况下单位功率收益最低（表 11 末行）。可见四场同步抬升并非最优。

表中数值为点预测值，编制功率预算时可另计 9.78 kW 的误差余量。电压无须按 0.1 kV 精度执行：多起点求解所得功率完全一致，但控制量本身存在若干功率近乎等价的组合（§8.1）。

5.6 目标点已越出历史运行经验

上表的解不能直接下发给设备。**各电场电压均未超过历史最大值，并不意味着该控制组合落在历史正常运行的范围内**，理由可由数据定量给出。

本文引入马氏距离，以定量评价推荐控制组合偏离历史运行经验的程度。仅逐个检查各电压是否越限并不足够，因为八个控制量之间彼此相关：历史上从未出现过 U_1 很高而 U_3 很低的组合，即便二者各自都落在允许范围内。马氏距离平方 $d_M^2 = (z-\mu_k)^T \Sigma_k^{-1} (z-\mu_k)$ 正是把这种相关性一并计入的距离，其中 μ_k 和 Σ_k 是该工况训练期控制量的均值与协方差。将其与历史上 97.5% 的时间均不超过的水平 $q_{k,0.975}$（四类工况依次为 20.55、18.72、16.85、19.09）相比，即可量化推荐解偏离历史经验的程度，结果见表 13。

表 13　推荐解相对历史运行经验的外推审计

工况与限值	d_M^2	$q_{0.975}$	倍数	最高电压超出历史最大值	判定
工况 1，10 mg	49.39	20.55	2.40×	0.0%	外推
工况 2，10 mg	46.46	18.72	2.48×	0.0%	外推
工况 3，10 mg	77.69	16.85	4.61×	0.0%	外推
工况 4，10 mg	102.79	19.09	5.38×	0.0%	外推
工况 1，5 mg	65.84	20.55	3.20×	+5.05%	外推
工况 2，5 mg	60.31	18.72	3.22×	+5.07%	外推
工况 3，5 mg	106.26	16.85	6.31×	+5.00%	外推
工况 4，5 mg	133.43	19.09	6.99×	+5.07%	外推

结论有三层。

其一，最低功率解全都落在历史经验之外，d_M^2 达到该水平的 2.40～6.99 倍。因此本文把 d_M^2 **作为外推风险指标报告**，而不把它当作必须满足的约束——否则求出来的就不是“最低功率”，而是“最低功率且最像以前的操作”，这是两个不同的问题。

其二，如果把 $d_M^2 \le q_{k,0.975}$ 真的**加成硬约束**，八个“工况—限值”组合里仍有七个有解（`results/mahalanobis_constrained_feasibility.csv`），代价是功率上升：10 mg/Nm³ 档四类工况分别多耗 1.46%～4.38%（加权 2026.5 kW，比最低功率解高 2.7%），5 mg/Nm³ 档三个有解的工况多耗 2.22%～6.07%，只有工况 1 在 5 mg/Nm³ 下无解。这就给出了一个可选的**保守方案**：若现场要求控制组合严格落在历史经验支持的范围内，可以略增电耗为代价采用该方案。本文以最低功率解为主报告结果，同时保留该保守方案，供现场按风险偏好选用。

其三，从电源容量看，10 mg/Nm³ 的四个解**在单变量电压幅值上未显著外推**：各电场电压均不超过该工况历史出现过的最大值。但如上所述，其控制组合仍偏离历史联合运行经验，故实际容量、绝缘与联锁方面的可行性仍须现场核验，不能仅凭幅值合规判定该目标可达。而 5 mg/Nm³ 需要各电场电压比历史最大值再高，最多高出 5.07%；四类工况所需的最小裕度分别是 5%、4%、1%、0%。**这说明 5 mg/Nm³ 首先是容量问题，其次才是电耗问题**：若现场高压电源不具备该裕度，仅靠优化运行参数无法达标，须通过增设电场、改造电源或前端减尘等措施解决。

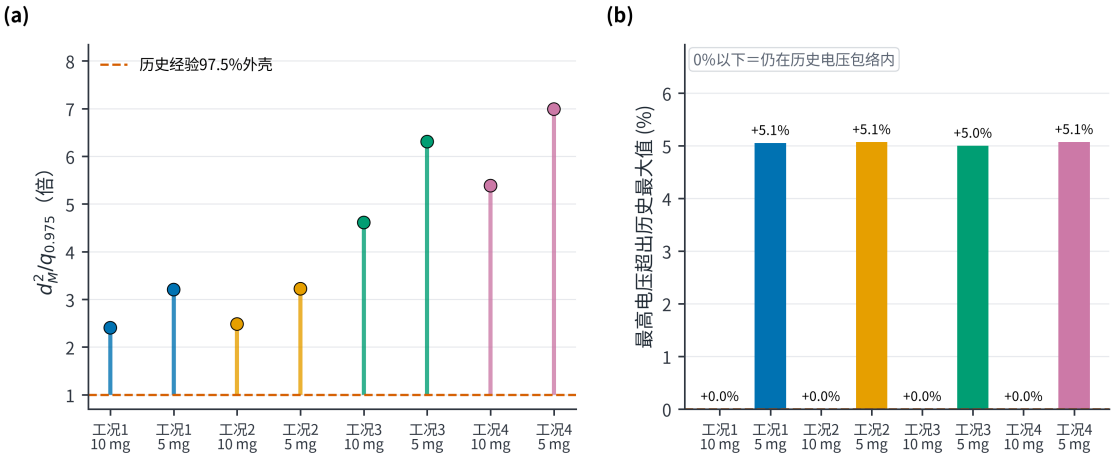

图 12　推荐解相对历史运行经验的外推审计

图 12 把两件不同的事并列：图 12(b) 显示 10 mg/Nm³ 解的各场电压均未超出历史最大值（单变量幅值合规），图 12(a) 却显示同一组解的 d_M^2 已达经验外壳的 2.40～5.38 倍（联合组合越界）。这正说明三个概念必须区分：

$$\text{单变量历史区间} \neq \text{历史联合运行支持域} \neq \text{设备安全域} \tag{11}$$

前两者的差别由图 12 量化；而第三者——设备安全域——本文无法从附件判断，因为附件不含二次电流、火花率、绝缘状态与联锁信息。因此，“历史上出现过该电压”不能推出“该状态对设备安全可达”。

5.6.1 解的活跃约束：振打周期是角点解

上节说明推荐解偏离历史联合经验的程度，本节进一步说明**解本身停在哪些约束上**。这一信息决定了各控制量的取值由什么确定：停在可行域内部的分量由权衡决定，停在边界上的分量则由边界本身决定。逐解统计见表 14。

由表 14 可得三点必须如实报告的结论。

其一，**八个解无一例外地把四个振打周期全部顶到支持域上界**，即表 12 中的 T 取值就是各工况训练期内出现过的周期最大值。原因可由两侧的边际量直接算出：把周期由中位延长至上界可回收约 60 kW（§4.1），而由此增加的峰值仅占用 0.1～0.3 mg/Nm³ 的限值余量，按 §7.1 的解析式折算，

表 14　最优解的活跃约束与振打周期上界的敏感性

工况与限值	贴上界的控制量	贴下界	排放约束余量/(mg/Nm³)	T 上界改取 P95 后的功率	代价
工况 1，10 mg	$U_1, U_3, U_4, T_1, T_2, T_3, T_4$	—	0.052	2084.9	+1.92%
工况 2，10 mg	$U_1, U_3, U_4, T_1, T_2, T_3, T_4$	—	0.038	2146.7	+2.01%
工况 3，10 mg	$U_1, U_3, T_1, T_2, T_3, T_4$	—	0.029	1991.3	+3.46%
工况 4，10 mg	$U_1, U_3, T_1, T_2, T_3, T_4$	U_4	0.009	1881.0	+1.61%
工况 1，5 mg	$U_1, U_3, U_4, T_1, T_2, T_3, T_4$	—	0.007	2221.4	+1.80%
工况 2，5 mg	$U_1, U_3, U_4, T_1, T_2, T_3, T_4$	—	0.002	2286.5	+1.82%
工况 3，5 mg	$U_1, U_3, T_1, T_2, T_3, T_4$	—	0.009	2100.2	+2.15%
工况 4，5 mg	$U_1, U_3, T_1, T_2, T_3, T_4$	U_4	0.012	2000.4	+1.39%

补偿这部分余量所需的电压代价不过 1～3 kW。两者相差一个数量级以上，$G(T)$ 与 $B(T)$ 两项在中心先验下**根本不足以形成内点最优**。因此本文必须说明：**振打周期这四个决策量的解是角点解，其数值由支持域上界给定，而非由电耗与峰值的权衡给定**；§7.3 所述的权衡在中心情景下并未真正约束优化器，它刻画的是若峰值代价被现场标定为更高时将会出现的情形。

其二，该上界取自历史最大值，而最大值是训练周内的单个极端分钟，作为边界并不稳健。为量化这一选择的影响，本文把 T 的上界改取 P95 后重解全部八个组合：全部仍可行，功率上升 1.39%～3.46%，10 mg/Nm³ 档加权由 1973.0 升至 2020.4 kW、5 mg/Nm³ 档由 2105.5 升至 2144.3 kW。**关键在于第四问的结论并不因此改变**：10→5 mg/Nm³ 的加权增幅由 6.72% 变为 6.13%，仍落在 §8.2 给出的 5.13%～7.55% 情景区间内。即振打上界的取法影响绝对功率水平，不影响限值收紧的相对代价。

其三，电压侧只有 U_2 始终位于可行域内部，U_1 与 U_3 在全部八个解中均贴上界，工况 4 的 U_4 则贴下界；工况 3 的排序约束 $U_1 \geq U_3 + 3$ 取等（余量 0.0），这正是 §5.4 所述“抬升 U_3 必须同步抬升 U_1”的直接体现。排放约束在八个解中余量均小于 0.06 mg/Nm³，即全部取等。综合来看，八个控制量中通常有六到七个由边界确定，**真正由内部权衡决定的自由度只有一到两个**。这一事实与 §8.1“多起点收敛一致”并不矛盾：正因为解被约束面钉住，各起点才会收敛到同一点。

5.7 排放—功率权衡前沿

只给出 10 与 5 mg/Nm³ 两个孤立点，无法判断二者之间的关系是单调的还是存在拐点，也无法看出限值每收紧一档的边际代价如何变化。为此把排放限值由 10 逐档收紧到 5 mg/Nm³ 并重复求解，得到“排放—功率”权衡曲线（图 13）。四类工况的最优功率随限值收紧单调上升，工况加权功率由 1973.0 kW（10 mg/Nm³）升至 2105.5 kW（5 mg/Nm³），中间各档依次为 1991.3、2012.7、2046.3、2067.7 kW；所需电压裕度同步由 0% 增至 5%（每收紧 1 mg/Nm³ 约增 1 个百分点）。全部档位均高于历史实际加权功率 1783.2 kW。该图把限值与功率的关系整体呈现出来：**沿曲线向左（限值更严）必然增加电耗，向右（限值更松）才降低电耗**；历史运行点则位于该图右侧更远处。

图 13 给出两点结论。第一，**限值越严，最优功率越高，且四类工况与加权曲线均单调**，中间不存在“更严反而更省电”的区段。第二，全部档位都位于历史实际加权功率 1783.2 kW 之上。由此可再次明确本文“节能”一词的口径：**它只指在同一排放目标下相对其他可行控制方案的功率降低，不能理解为在维持历史排放水平的同时还能额外省电**。历史运行点对应的排放水平约为 50 mg/Nm³，位于该图右侧更远处，与图中任一档位都不可直接比较。

6　问题三的模型建立与求解：控制策略比较

第三问要求选取差异显著的两类工况，比较其最优操作参数并解释电压与振打周期的优先级变化。**本节所用模型与方法为**：以 §5.4 的单位电功率去除收益判据 $\alpha_i/(b_i U_i^{ref\,2})$ 与振打侧的边际节电判据 c_i/T_i^2 为解析依据，对照 §5.5 的数值解读出两类工况的策略差异，并结合 §4.2 的历史操作规律作经验校核，最后整理为可现场执行的控制优先级决策表与三状态运行方案。

选取工况 1 和工况 3。二者流量都较高，但入口浓度和粉尘负荷差异明显：工况 1 平均负荷 12460 kg/h，工况 3 为 19560 kg/h，可用于比较低负荷与高负荷策略。

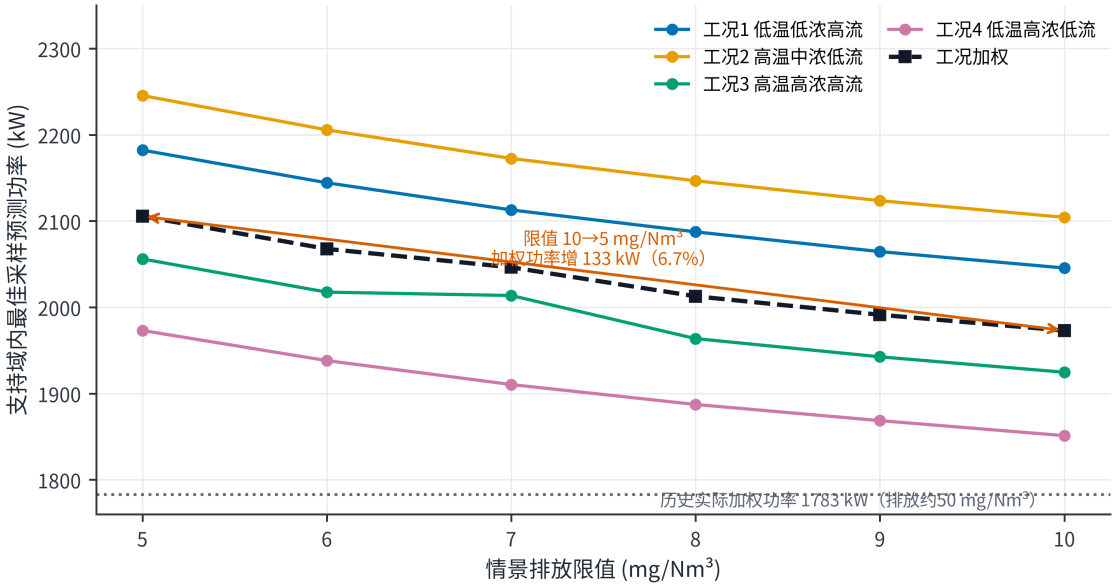

图 13　排放—功率权衡前沿

低负荷工况 1 采用四场同步升压策略（即四个电场按相近幅度同步抬升）：10 mg/Nm³ 下 $U = (70.2, 68.6, 57.6, 57.4)$ kV，四个电场相对各自参考值的抬升幅度接近。原因是该工况锚点 $K_0 = 6.257$ 最低，达标所需的强度因子 $S = 1.288$ 最大，四个电场均须出力，无从取舍。

高负荷工况 3 采用择优升压策略：$U = (68.0, 60.3, 56.3, 57.3)$ kV，其中 U_1、U_3、U_4 都接近历史最大值，唯独 U_2 停在 60.3 kV（它的历史最大值是 69.3 kV）。这正是表 11 判据的直接体现——工况 3 的优先次序为 E3 > E4 > E1 > E2，第二电场单位功率收益最低，故被留在低位。限值由 10 收紧到 5 mg/Nm³ 时，各场增量为 $\Delta U = (+3.4, +3.1, +2.8, +3.1)$ kV，这一格局保持不变。

因此，第三问不能简单归结为固定的“前级优先”，而应按单位功率的除尘收益确定调节顺序。以下三条判据可直接计算：

- **电压侧**：按 $\alpha_i/(b_i U_i^{ref\,2})$ 降序抬升，受排序约束 $U_1 \geq U_3 + 3$ 牵引时同步抬升 U_1；
- **振打侧**：按 c_i/T_i^2 降序延长周期，即优先延长周期较短的前级；
- **两者分工**：电压用于提高捕集能力，周期用于降低电耗，二者的最优着力点分别位于不同电场，不应混为一谈。

历史数据对上述判据只提供了部分支持。§4.2 的规律一（入口浓度升高时缩短各场周期）在全部时段方向一致，与高负荷下不宜为节电而延长周期相符；规律二（入口浓度升高时抬升前两级电压）仅在最后两天成立，故本文只将其列作参照。

为便于现场执行，按排放构成与容量状态把上述比较整理成一张决策表，见表 15。

上述两类策略的差异可由图 14 直接读出。

图 14 显示四个电场并非机械同步抬升：工况 1 四场抬升幅度接近，工况 3、4 则出现明显的高低分化。这一分配由两个因素共同决定——单位功率去除收益 $\alpha_i/(b_i U_i^{ref\,2})$ 决定优先次序，排序约束 $U_1 \geq U_3 + 3$、$U_2 \geq U_4 + 3$ 则把部分电场的取值绑定在一起。两者叠加的结果是：性价比最高的第三电场被抬到接近上限，与之绑定的 U_1 随之走高，而性价比最低的电场（如工况 4 的 U_4）被压至历史下界。各档的振打周期取值见表 12，均被推至历史区间上端。

在线运行设 **TARGET-10、TARGET-5、FALLBACK 三种状态**。排放目标由上级指令或管理规定选定，不设自动切换阈值——本文排放模型未经现场标定，尚不足以在线自动判定当前排放水平。TARGET-10 与 TARGET-5 分别读取表 12 中对应工况、对应目标的控制参数。控制量按斜坡逐步逼近目标值，试运行阶段暂定电压变化不超过 0.5 kV/min、振打周期不超过 5 s/min；若现场 DCS、联锁或设备制造商规定了更严的限制，以更严者为准。四场振打先按 $\rho = 1$（同相振打、峰值完全叠加）

表 15　控制优先级决策规则

状态特征	优先动作	原因
连续排放基值接近限值，振打峰值占比较低	按 $\alpha_i/(b_i U_i^{ref\,2})$ 降序小步提高电压	主要矛盾是连续捕集强度不足，应优先动用性价比最高的电场
峰值增量占比较高，周期明显长于参考周期	优先缩短过长周期并实施错相振打	主要矛盾是单次积灰脱落和峰值叠加
周期过短、振打频繁，且电耗压力大	优先延长前级周期	前级 c_i/T_i^2 最大，单位延长的节电最多
高粉尘负荷且基值和峰值均高	先错相、再调整周期，最后小步提升电压	先控制峰值，再补充连续捕集能力
某电场电压已达历史最大值	转向次优电场；若各场均达上限则停止优化	数学可行不等于电源容量可行
入口温度进入 140～160 °C 高比电阻带	退出优化，按现场反电晕处置预案运行	该带未被本文工况覆盖，模型外推无效
超出历史支持域或出现火花、联锁风险	停止优化，回退现场安全设定	数学可行不等于设备安全

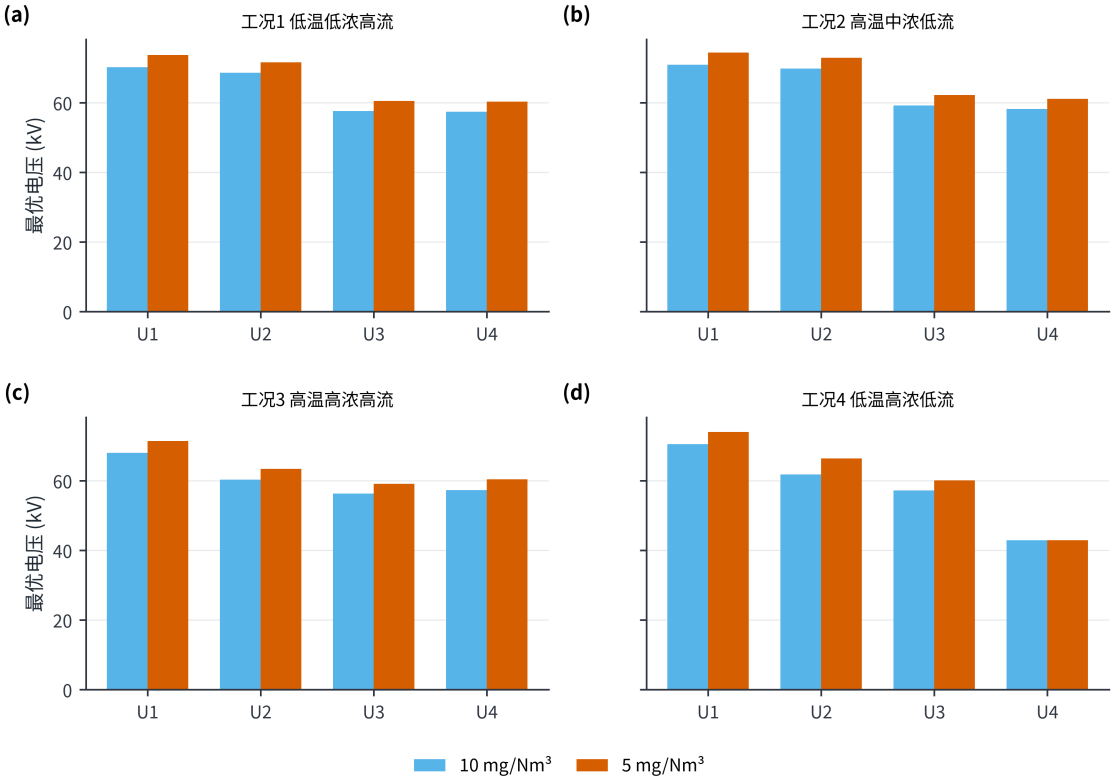

图 14　两档排放限值下的电压配置

这一最不利情形核验，再以 $\rho = 0.80$ 和 0.65 评估错相运行可带来的裕度。传感器失效、火花率越限、工况超出取值范围或联锁触发时，即切换至 FALLBACK。

7　问题四的模型建立与求解：限值收紧的功率代价

第四问要求把排放约束由 10 mg/Nm³ 收紧到 5 mg/Nm³，估计功率代价并给出高浓度工况建议。**本节所用模型与方法为**：在四场电压同比缩放的简化下导出功率增量的解析式（§7.1），与 §5.3 的八变量数值最优解互为验证（§7.2），再把该代价与 §4.1 的振打功率结构并列，给出振打侧可回收功率的定量对照（§7.3），最后说明峰值约束口径的选取及其偏保守的方向（§7.4）。

7.1 同比缩放下的解析估计

第四问可在一个简化假设下导出解析式，其解释力强于纯数值优化结果。该假设是：四个电场的电压同比缩放。它并非原八变量非凸优化问题的精确闭式解，但 §7.2 将验证二者相差不足 1%。设

在 10 mg/Nm³ 的最优点上，电场功率为 P_{field}、去除指数为 K_{10}。把四个电场的电压同时放大 f 倍，则去除指数变成 $K_{10}f^{p}$，电场功率变成 $P_{field}f^{2}$。要让排放减半（计入峰值项后，所需的去除指数增量是 $\Delta K=\ln\frac{10-B}{5-B}$），解得 $f=(1+\Delta K/K_{10})^{1/p}$，于是

$$\Delta P=P_{field}\left[\left(1+\frac{\Delta K}{K_{10}}\right)^{2/p}-1\right]\approx\frac{2}{p}\cdot\frac{P_{field}}{K_{10}}\Delta K \tag{12}$$

该式含义清晰：**限值每减半所增加的电耗，等于电场功率乘以所需的去除指数增量、除以当前去除指数，再乘以** $2/p$。由此可见，设备当前排放水平越低（K 越大），进一步减半的相对代价越小。

7.2 分工况结果

表 16　中心情景下 10 收紧至 5 mg/Nm³ 的分工况功率代价

工况	10 mg 功率/kW	5 mg 功率/kW	K_{10}	ΔK	解析 ΔP/kW	数值 ΔP/kW	功率增幅
1 低温低浓度高流量	2045.55	2182.25	8.059	0.804	137.7	136.7	6.68%
2 高温中浓度低流量	2104.30	2245.51	8.338	0.832	142.4	141.2	6.71%
3 高温高浓度高流量	1924.67	2056.06	8.594	0.903	131.3	131.4	6.83%
4 低温高浓度低流量	1851.15	1973.10	8.690	0.906	123.1	122.0	6.59%

解析估计与数值最优解的相对差异不超过 1%（图 15），二者互为验证：数值优化给出结果，解析式给出机理解释。按训练期工况占比加权，

$$\bar{P}_{10}=1973.0\ \text{kW},\qquad \bar{P}_{5}=2105.5\ \text{kW},\qquad \Delta P/\bar{P}_{10}=6.72\% \tag{13}$$

按相同运行时长折算，两档日电耗分别约为 47.35 与 50.53 MWh/d，收紧限值的日增电耗约 3.18 MWh/d；按年运行 8760 h 计，年增电耗约 1.16×10³ MWh。加上 9.78 kW 验证期误差附加量后，两档加权功率分别为 1982.8 与 2115.3 kW；由于两者使用同一附加量，该比例不作为严格上界，只用于功率预算。

四类工况的增幅高度一致（6.59%～6.83%）。这不是巧合，而是上述解析式的直接推论：增幅 $\approx(2/p)(P_{field}/P)(\Delta K/K)$，而四类工况的电场功率占比和当前去除指数都相差不大，所以增幅几乎不随工况变化。

上述闭式估计建立在四场电压同比缩放这一简化之上，而完整优化允许八个控制量各自独立调整。因此有必要检验该简化相对八变量数值最优解的偏差。

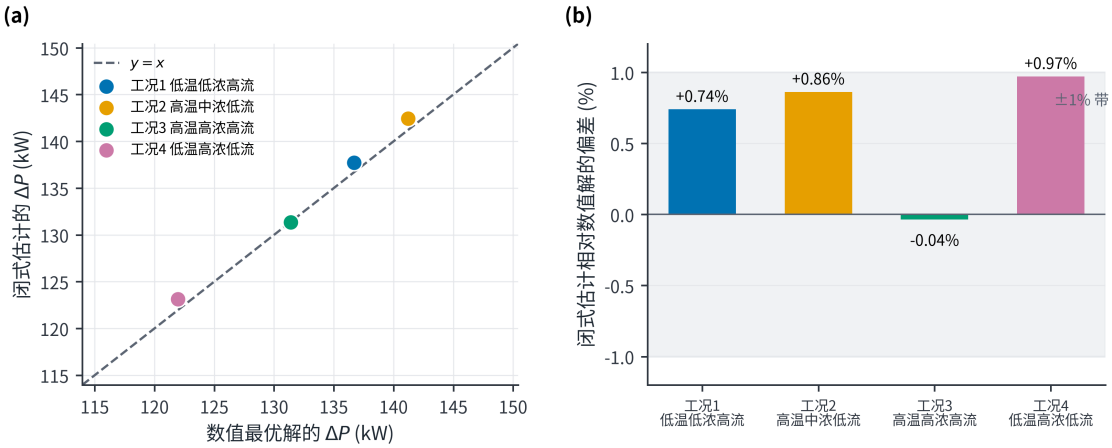

图 15　闭式估计与数值最优解的一致性

图 15(a) 中四类工况的点均紧贴 $y = x$ 线，图 15(b) 给出相对偏差为 −0.04%～+0.97%，全部落在 ±1% 带内。**这说明按同比缩放导出的解析式虽是近似，却能以不足 1% 的误差复现完整的八变量数值优化结果**，因而可以作为第四问功率代价的解析解释使用：数值优化给出数值，解析式给出机理。

7.3 真正的权衡：振打周期能买回多少

将第四问的结果与 §4.1 的振打功率结构并列，可得到本文最具操作价值的一组对照。按工况加权计算，限值由 10 收紧到 5 mg/Nm³ 要多耗约 132.6 kW；而把四个电场的振打周期从历史中位延长到历史 P95 可省下 56.1 kW，延长到历史最大值可省下 109.1 kW。**即振打侧可挖掘的全部潜力（约 109 kW），相当于排放收紧一档所增电耗（约 133 kW）的 82%，二者处于同一量级。**

但上述节电收益必须与振打峰值排放的代价一并权衡。按 §4.5 的推导，把周期由中位延长至历史最大值（平均延长 17.9%）时，单次振打的灰饼质量与瞬时峰值同比放大约 17.9%。而在表 12 的解中，峰值项本就已经占掉限值的 0.95～1.61 mg/Nm³，相当于 5 mg/Nm³ 这一档约五分之一到三分之一的余量。峰值再放大 17.9%，将进一步占用 0.17～0.29 mg/Nm³ 的排放裕度，该部分必须由电压侧补偿，而补偿电压本身又要付出功率代价。这正是振打周期不宜为节电而无限延长的定量表述：**振打周期是本设备上成本最低的节电手段，但其额度有限，且以瞬时排放峰值为代价。**

7.4 峰值约束口径的说明

在中心情景的解中，10 mg/Nm³ 方案的连续排放基础值为 8.38～9.00 mg/Nm³，振打带来的峰值增量为 0.95～1.61 mg/Nm³；5 mg/Nm³ 方案的基础值则降到 3.38～4.05 mg/Nm³。可见仅令平均值或基础值低于限值并不充分，否则振打时刻仍可能超限。

需要说明的是，本文把**瞬时峰值**直接卡到限值，而现行排放标准对颗粒物通常按小时均值考核。按 §4.5 的推导，一次持续数秒到数十秒的再飞扬峰值，对小时均值的贡献正比于 L 而与周期无关。因此本文口径系统性偏保守，会高估达标所需功率。若改用小时均值达标、峰值仅作软性惩罚的口径，所得功率增幅将低于本文报告值。在缺少逐次振打数据的条件下，本文选取保守口径，但不应将其视为唯一的合规解释。

仅约束连续排放并不充分，因为振打瞬间的峰值同样占用限值裕度。图 16 给出两档解中限值裕度的实际构成。

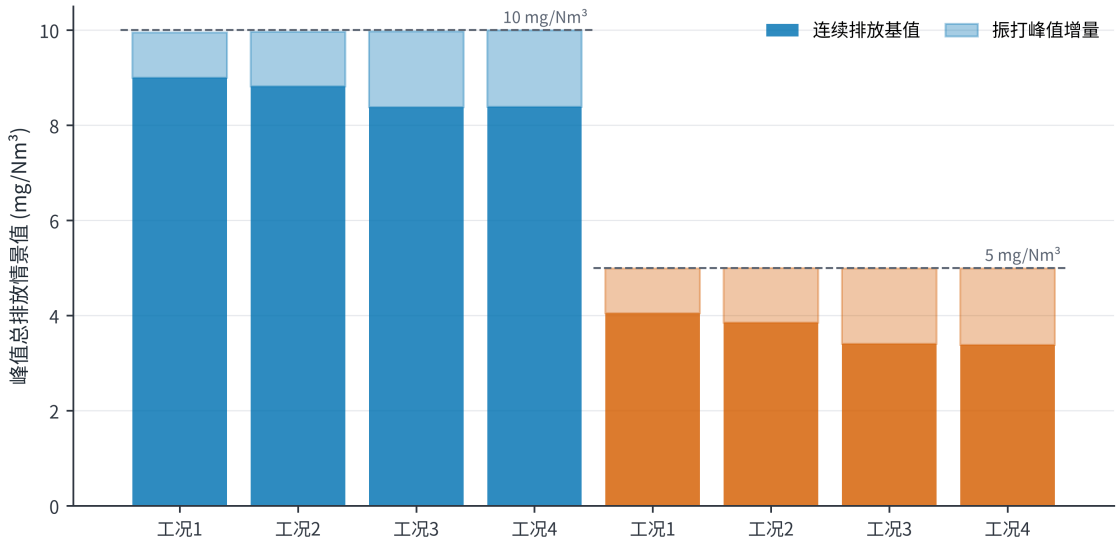

图 16　排放约束分解

图 16 显示峰值项在 10 mg/Nm³ 档占用 0.95～1.61 mg/Nm³，在 5 mg/Nm³ 档更相当于限值的五分之一至三分之一。这正是本文把约束写成 $C_{peak} = C_{base} + B$ 并整体施加、而非只约束 C_{base} 的原因：若只令连续基值达标，振打时刻仍可能超限。

8　模型检验、敏感性与稳健性

8.1 求解质量

§5.3 已说明本问题不是凸规划，因此单次求解可能只收敛到局部解，必须检验不同初值是否给出不同结果。图 17 给出两项独立检验。八个“工况—限值”组合中，多起点 SLSQP 有 47～53 个起点成功收敛，**所有收敛起点给出的最优功率一致到 10^{-7} kW 以内**，表明各起点均收敛至同一解。作为独立对照，在同一可行域内均匀随机抽取 4×10^5 个候选点的随机搜索，在全部八个组合上均未找到更优解，其最优值反而高出 0.97%～5.17%；其中工况 1 的 10 mg/Nm³ 组合甚至未采到任何可行样本。原因在于可行域被排放约束与排序约束夹成一条窄带，均匀随机采样的命中效率很低。

该对照说明：在问题结构明确之后，确定性求解优于大规模随机搜索。但仍须指出，对非凸问题而言，多起点收敛至同一点是**很强的证据，而非数学证明**。因此本文只称“最优解”，不宣称“全局最优”。

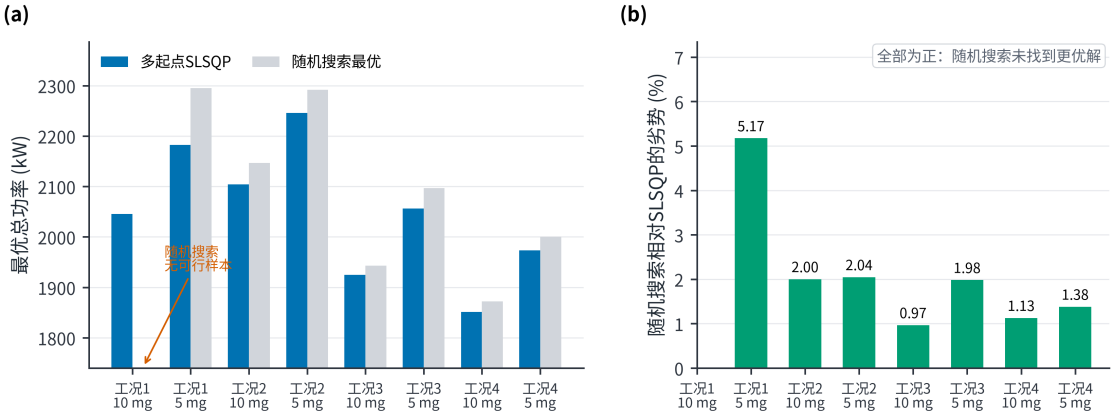

图 17　求解方法对照

图 17(a) 显示两种方法在八个组合上的最优功率，图 17(b) 给出随机搜索相对多起点 SLSQP 的劣势。三点结论：47～53 个起点全部收敛到同一功率；4×10^5 点随机搜索未找到任何更优解；因此该解在数值上是稳定的。但仍须重申，**这是数值稳定性的强证据，不构成全局最优的数学证明**——对非凸问题，多起点一致只能降低遗漏更优解的可能性，不能排除它。

8.2 先验敏感性

排放层含 $p, \alpha_i, \beta_i, b_0, \gamma, \rho, s$ 共七类先验参数，逐一扫描既无必要也不便阅读。本节按三个层次组织：先看结构设定变化后核心结论是否稳定（层次一），再定位真正主导功率代价与可达性的参数（层次二），最后检验电场优先级是否依赖权重设定（层次三）。

层次一：结构情景下核心结论是否稳定。扫描电场权重形状（3 种）× 峰值指数 $\gamma \in \{1.00, 1.30, 1.60\}$× 相位尺度 $\rho \in \{0.65, 0.80, 1.00\}$× 参考周期负荷指数（3 种），共 81 组。

图 18 显示 81 组情景全部可行，加权增幅为 5.13%～7.55%、中位数 6.40%，中心值 6.72% 位于该区间内。其中峰值指数在推导值 1.00 附近变化时增幅几乎不变，这与 §4.5 一致——峰值项占用的是限值余量，本身不是主导项。**即核心结论对灰箱的形状设定并不敏感。**

层次二：什么参数真正主导功率代价与可达性。既然形状设定影响有限，就需要找出真正起决定作用的参数。由 §7.1 的解析式，功率增幅近似正比于 $2/p$，即幂次 p 以显式方式进入结论；其余先验只通过峰值裕度间接影响结果。这正是本文把 p 单独提出的原因。表 17 给出 p 的参考表。

表中增幅为闭式估计值；对 $p \geq 2$ 的三档另作数值重解校验（图 19 叉号），与解析值相差不超过 0.5 个百分点。

图 19 表明 p 同时决定两件事：**功率增幅**（由 14.2% 降至 4.4%，跨度超过三倍）与**目标可达性**（$p = 1$ 与 $p = 1.5$ 在 5% 电压裕度内根本无解）。作为对照，另一组扫描——$p \in \{1.5, 2.0, 2.5\}$× 峰值

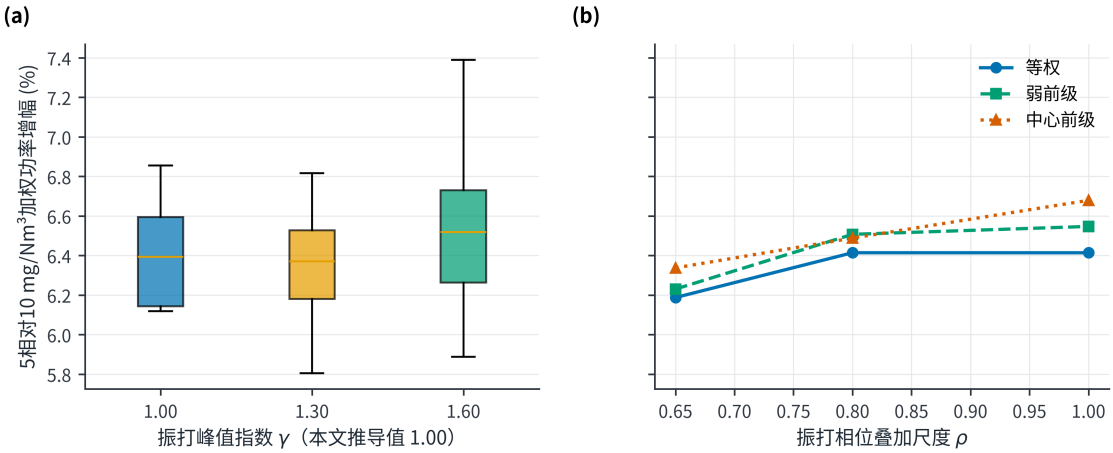

图 18　结构情景下的功率增幅分布

表 17　功率增幅对幂次 p 的依赖

p	物理含义	10→5 mg/Nm³ 加权增幅	5 mg/Nm³ 在 5% 电压裕度内是否可达
1.0	去除指数正比于电压	14.2%	否，需更大电压裕度
1.5	中间情形	9.2%	否，需更大电压裕度
2.0	**Deutsch 迁移速度形式（中心）**	**6.8%**	**是**
2.5	强电压响应	5.4%	是
3.0	极强电压响应	4.4%	是

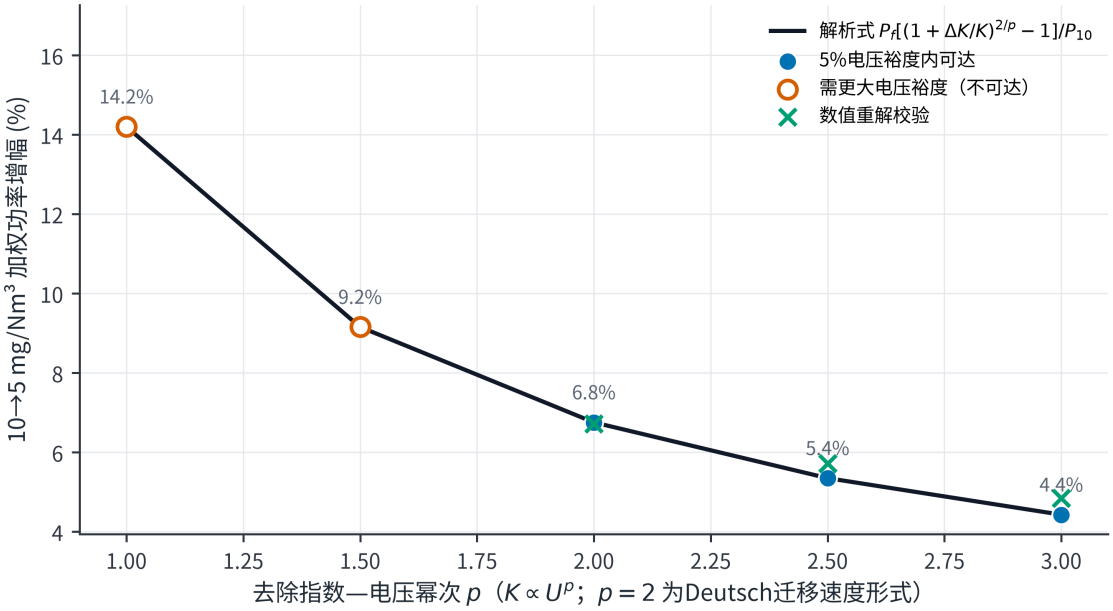

图 19　功率增幅对幂次 p 的依赖

尺度 $b_0 \in \{0.9, 1.2, 1.5\}\times$ 安全裕度 $s \in \{0, 0.08, 0.15\}$ 共 27 组——中，15 组在 5% 电压裕度内四类工况全部可行，不可行的 12 组也全部来自 $p = 1.5$；有解各组的加权增幅为 3.99%～7.04%、中位数 5.85%。

因此，**就 10→5 mg/Nm³ 的功率增幅与目标可达性而言，p 是主导性的待标定参数**，且一次电压阶跃试验即可确定：在工况稳定时把四个电场的电压同时抬高几个百分点，记录未截顶的出口浓度如何变化，再把 $\ln C_{out}$ 对 $\ln U$ 作回归，斜率即为 $-K_0 p$。但须明确，这**不意味着 p 是唯一需要标定的参数**：$\alpha_i, \beta_i, b_0, \gamma, \rho, s$ 同样未由附件识别，它们影响峰值结构与分场控制的具体分配，仍须由后续现场数据一并标定（§9.3）。

层次三：电场优先级是否依赖权重设定。第三问的判据含先验权重 α_i，需检验结论是否由该先验单独写入。

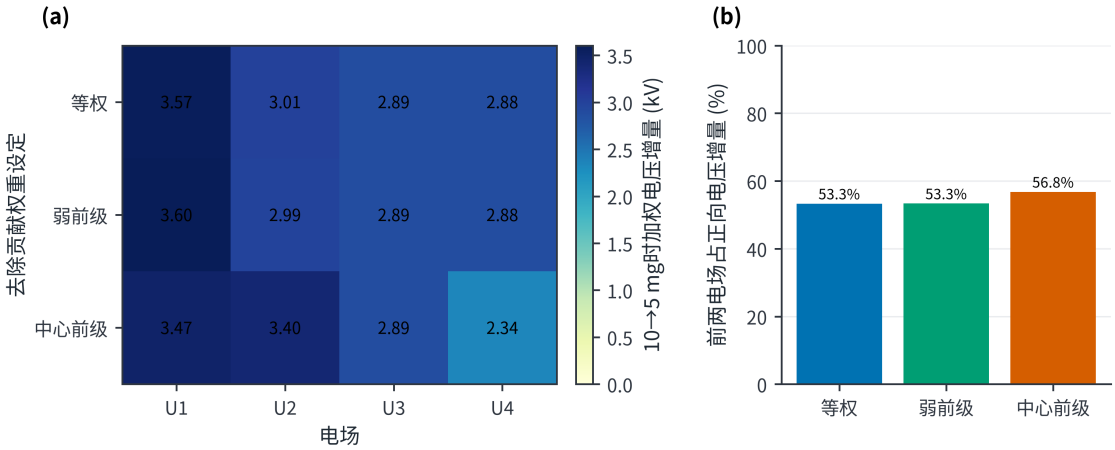

图 20　电场贡献权重消融结果

图 20 显示，在等权、弱前级与中心前级三种权重形状下，10→5 mg/Nm³ 所需的加权电压增量分布保持一致的格局，与按 $\alpha_i/(b_i U_i^{ref\,2})$ 排序的解析优先级相符。这说明优先级结论并非由中心权重单独决定，但其具体数值仍依赖同一灰箱结构与支持域。

需要强调，以上区间均为**先验参数网格上的情景范围，而非统计意义上的置信区间**。

8.3 稳健性结论

稳健性证据分四层。第一，轮廓系数支持把工况分成四类。第二，四种设定的比较与时间外推支持所选的功率模型，其中 $1/T$ 形式在**完全未参与拟合**的测试日上表现最好，这是本文最强的实证证据。第三，多起点收敛一致以及随机搜索的对照，支持数值解的质量。第四，解析估计与数值最优解互相验证，情景扫描与权重消融则检验了结论对建模选择的依赖程度。第五，截尾极大似然对截顶假设作出可证伪的确认，相位折叠对振打同步波动给出上界，二者把 §2.2 与 §4.5 由否定性陈述改写为可检验结论。

但稳健性的边界同样须写明：如 §5.6.1 所示，八个解中通常有六到七个控制量停在支持域边界上，其中四个振打周期全部取在上界。因此多起点收敛一致主要反映约束面对解的钉定作用，而非目标函数在内部具有良好的曲率；支持域一旦改变，解的位置即随之改变（表 14）。

但仍须强调：排放层的全部数值都是机理推演的结果。无论是误差余量还是情景范围，都不能把一个机理模型变成经过实测验证的模型。

9　模型评价、实施方案与局限

9.1 模型优点

第一，先逐列判定数据能够支持与不能支持的结论，既避免以复杂算法拟合截顶噪声，也未因一列不可用而放弃其余 13 列的信息。

第二，功率模型采用有物理含义的形式而非通用多项式，在未参与拟合的测试日上达到 $R^2 = 0.993$、偏差 −0.02 kW，并由此发现“振打占电耗近四成”这一可直接使用的结论。

第三，排放起算点直接取记录值，不引入未经证实的量纲假设，使第二问回答的是题目实际提出的问题。

第四，电场优先级由公式推出，而不是从某一次求解的现象里归纳。

第五，第四问给出解析式，把功率代价的主要不确定性归结到一个可现场标定的幂次 p，并同时说明其余灰箱参数仍待标定。

第六，把推荐解偏离历史运行经验的程度作为结论的一部分报告，并给出严格落在历史经验之内的保守方案及其电耗代价，而非回避该问题。

第七，对两项否定性判断分别补作可证伪检验：截尾极大似然预测的封顶比例与实测相差 0.0014，

相位折叠给出振打同步波动不超过 0.03 mg/Nm³。二者使否定性结论具备被推翻的可能，而不只是未能找到证据。

第八，逐解报告活跃约束，指明四个振打周期为支持域上界所决定的角点解，并以上界改取 P95 的重解量化该选择的影响。

9.2 模型局限

最主要的局限是，排放层的全部系数（$p, \alpha_i, \beta_i, b_0, \gamma, \rho, s$）都无法从附件中辨认，只能依据机理和文献设定；表 17 给出的区间是人为参数网格上的情景范围，不是统计置信区间。

其次，最优解高度依赖支持域的取法：八个解中六到七个控制量停在边界上，四个振打周期全部取在上界，故这四个量的数值由边界给定而非由权衡给定（§5.6.1）。与之相关，功率模型中 $\sum c_i/T_i$ 一项的量级远超振打机构本身（§4.1），本文只把它当作随振打频率变化的经验关系使用，其机理归属仍待现场分解厂用电后确认。

再次，历史运行范围并不等于设备的安全边界，5 mg/Nm³ 档所需的 5% 电压外推更须经高压电源容量与绝缘裕度核算。模型也没有二次电流、分场功率、火花率、极板积灰状态和联锁信息；分钟级数据不足以观察逐次振打峰值，因此相位叠加尺度 ρ 无法标定。四类工况的温度中心都没有覆盖 140～160 °C 的高比电阻区间，本文结论在该区间无效。此外，测试日的数据曾被前期处理流程接触过，不能算作完全未触碰的盲测。

因此，本文给出的控制参数只能作为现场试验的起点，**不得直接下发生产设备**。

9.3 上线与补采方案

建议分四步部署。

第一阶段：影子运行。仅在后台计算、不改变设备设定，用以验证工况识别与 TARGET-10/TARGET-5 参数表的加载逻辑；同期完成高压电源容量核算，确认 5 mg/Nm³ 所需的电压裕度是否存在。

第二阶段：小步阶跃标定。在每类工况内执行单因素小步阶跃，重点是**标定幂次** p（§8.2），并由现场确定电压与周期的实际变化速率上限。

第三阶段：重新标定。以 1～5 s 的未截顶出口浓度、实际振打事件、二次电流、火花率与分场功率，重新标定排放层参数及相位尺度 ρ，并单独补采 140～160 °C 高比电阻区间的数据。

第四阶段：独立盲测与闭环投用。冻结模型，在全新日期上盲测，通过后方允许投入闭环。任何超出取值范围、传感器失效或联锁事件均应触发 FALLBACK。EPA 技术资料与相关实验都强调气流分布、收尘面积和再飞扬对性能评价的重要性 [2-3,6]，因此补采数据时应优先补齐这几项。

10 结论

1. **问题一：分层作答**。可由数据确定的部分：总功率几乎完全由控制量决定，$P = 73.1 + 0.0850\sum U_i^2 + 5.39\times10^4\sum T_i^{-1}$（测试日 $R^2 = 0.993$），其中振打环节在中位工况下占总功率的 38.8%，而在现有数据精度下工况变量对总功率无额外解释力；历史操作规律中，入口浓度升高时缩短各场振打周期这一规律在四个时段方向一致，而入口浓度升高时抬升前级电压则仅在最后两天成立；附件还含两段持续异常工况，其中 05-06 的 158.2 °C 事件正落在粉尘比电阻的高值区。不可由数据确定的部分：出口浓度记录在 50 mg/Nm³ 附近呈现明显的上限截顶特征，与全部 12 个变量的相关系数绝对值均小于 0.017；截尾极大似然进一步确认该截顶假设——仅用未封顶记录估得潜变量 $N(50.036, 0.313^2)$，据此预测的封顶比例 0.5461 与实测 0.5475 相差 0.0014，而潜变量标准差仅为均值的 0.63%，故该列在信息量上不可能识别控制量对排放的作用。振打峰值则由积灰量守恒推得：单次峰值随周期近似线性放大，而其对小时均值的贡献与周期无关；相位折叠检验另给出与振打同步的排放波动上界约 0.03 mg/Nm³，但该上界不约束被分钟平均摊薄且被截顶抹去的单次尖峰。

2. **问题二：4 类工况与 10 mg/Nm³ 方案**。基于入口三变量识别出 4 类典型工况（轮廓系数 0.4084）。以记录值 50 mg/Nm³ 为锚点，四类工况满足 10 mg/Nm³ 的最低功率为 1851.15～2104.30 kW，加权 1973.0 kW，日电耗 47.35 MWh，较历史实际的 1783.2 kW 上升 10.6%。收紧排放必然增加电耗，同一排放水平下不存在额外的节能空间。该方案电压仍位于历史范围之内，但已明显偏离历史常规操作（马氏距离达经验 97.5% 水平的 2.4～5.4 倍）；若要求严格落在历史经验之内，四类工况仍均可行，加权功率升至 2026.5 kW，多耗 2.7%。须一并说明解的结构：八个解中通常有六到七个控制量停在支持域边界，其中四个振打周期全部恰好取在上界，属角点解，其数值由支持域上界给定而非由电耗与峰值的权衡给定；上界改取 P95 后八组仍全部可行，加权功率升至 2020.4 kW（多耗 2.4%）。

3. **问题三：优先级可由解析判据确定**。单位电功率的除尘收益为 $K_0\alpha_i/(\sum \alpha \cdot b_i U_i^{ref\,2})$。第三电场参考电压最低，该收益在四类工况中均居首；“前级优先”的结论实由排序约束 $U_1 \geq U_3 + 3$ 导出。振打侧按 c_i/T_i^2 排序，应优先延长周期较短的前级。低负荷工况须四场同步升压，高负荷工况则可择优升压，使单位功率收益最低的电场留在低位。

4. **问题四：功率代价与容量约束**。限值由 10 收紧至 5 mg/Nm³ 时，同比缩放假设下的解析估计 $\Delta P = P_{field}\ln[(10-B)/(5-B)]/K_{10}$ 给出 123.1～142.4 kW，与八变量数值最优解相差不足 1%；加权功率由 1973.0 升至 2105.5 kW，增幅 6.72%，日电耗由 47.35 增至 50.53 MWh。增幅与 $2/p$ 成正比；就功率增量与目标可达性而言，p 是主导性的待标定参数，但 $\alpha_i, \beta_i, b_0, \gamma, \rho, s$ 同样未由附件识别，仍须现场一并标定。更关键的约束不是电耗而是电源容量：5 mg/Nm³ 要求各电场电压超出历史最大值最多 5.07%，四类工况所需的最小裕度分别为 5%、4%、1%、0%。若既有电源无法提供该裕度，仅靠优化运行参数无法达标，应考虑电源改造、增设电场或前端减尘等措施。该增幅对支持域取法不敏感：振打周期上界改取 P95 后仍达 6.13%。以上排放数值均属机理推演，必须在补采未截顶的排放数据、振打事件与电气状态后重新标定。

参考文献

[1] 中华人民共和国生态环境部. 水泥工业大气污染物排放标准：GB 4915—2013（含 2025 年修改单）[EB/OL]. (2013-12-27)[2026-08-06]. https://www.mee.gov.cn/ywgz/fgbz/bz/bzwb/dqhjbh/dqgdwrywrwpfbz/201312/t20131227_265765.htm.

[2] OGLESBY S Jr, NICHOLS G B. *A Manual of Electrostatic Precipitator Technology: Part I—Fundamentals*[R/OL]. Birmingham: Southern Research Institute, 1970[2026-08-06]. https://nepis.epa.gov/Exe/ZyPURL.cgi?Dockey=9101FJFA.TXT.

[3] SZABO M F, SHAH Y M, SCHLIESSER S P. *Inspection Manual for Evaluation of Electrostatic Precipitator Performance*[R/OL]. Washington, DC: U.S. Environmental Protection Agency, 1981[2026-08-06]. https://nepis.epa.gov/Exe/ZyPURL.cgi?Dockey=9400034C.TXT.

[4] MIZUNO A. Electrostatic precipitation[J]. IEEE Transactions on Dielectrics and Electrical Insulation, 2000, 7(5): 615-624. DOI: 10.1109/94.879357.

[5] LI Z, SHAO C, AN Y, et al. Energy-saving optimal control for a factual electrostatic precipitator with multiple electric-field stages based on GA[J]. Journal of Process Control, 2013, 23(8): 1041-1051. DOI: 10.1016/j.jprocont.2013.06.007.

[6] NAVARRETE B, VILCHES L F, RODRIGUEZ-GALAN M, et al. A pilot scale study of the rapping reentrainment and fouling in electrostatic precipitation[J]. Environmental Progress & Sustainable Energy, 2015, 34(1): 7-14. DOI: 10.1002/ep.11934.

[7] ZHENG C, ZHAO Z, GUO Y, et al. A real-time optimization method for economic and effective operation of electrostatic precipitators[J]. Journal of the Air & Waste Management Association, 2020, 70(7): 708-720. DOI: 10.1080/10962247.2020.1767227.

[8] FU Y, SU Z. Deep neural network based concentration prediction model of dry electrostatic precipitator with a modified PID optimizer[J]. Journal of Physics: Conference Series, 2022, 2189: 012018. DOI: 10.1088/1742-6596/2189/1/012018.

[9] MACQUEEN J. Some methods for classification and analysis of multivariate observations[C]//Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability. Berkeley: University of California Press, 1967, 1: 281-297.

[10] ROUSSEEUW P J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis[J]. Journal of Computational and Applied Mathematics, 1987, 20: 53-65. DOI: 10.1016/0377-0427(87)90125-7.

[11] HOERL A E, KENNARD R W. Ridge regression: Biased estimation for nonorthogonal problems[J]. Technometrics, 1970, 12(1): 55-67. DOI: 10.1080/00401706.1970.10488634.

[12] WHITE H J. *Industrial Electrostatic Precipitation*[M]. Reading, MA: Addison-Wesley, 1963.

[13] PARKER K R. *Electrical Operation of Electrostatic Precipitators*[M]. London: Institution of Electrical Engineers, 2003.

[14] OpenAI Codex, GPT-5, OpenAI, 2026-08-05—2026-08-07.

[15] GLM, GLM-5.2, 智谱 AI, 2026-08-05—2026-08-07.

[16] Kimi, K3, 月之暗面, 2026-08-06—2026-08-07.

附录 A 与既有多电场节能优化研究的对照

本文与文献 [5]、[7] 同属“在排放约束下对多个电场分别优化”这一类工作，但两点不同：排放模型怎么标定，以及可行范围怎么划。对照见表 A1。

表 A1　与既有多电场节能优化研究的方法对照

维度	文献 [5]（GA 优化）	文献 [7]（实时优化）	本文
排放模型标定	由现场实测出口浓度回归	由在线仪表实时校正	出口浓度不可识别，改以记录值为锚点的 Deutsch 灰箱，全程标注为推演
功率模型	分场电源功率实测	分场功率与二次电流	仅有总功率，按 $U^2 + 1/T$ 物理设定辨识，测试 $R^2 = 0.993$
振打处理	未纳入优化	作为固定运行参数	纳入决策变量，并由积灰量守恒推导峰值标度
求解方法	遗传算法	滚动实时优化	多起点 SLSQP ＋解析优先级判据＋随机搜索对照
可行域	设备额定范围	在线安全约束	历史运行包络，越界量作为结论显式报告

上述工作的共同前提是排放模型可由现场数据标定。本文缺少这一条件，其处理方式是把不可标定的部分全部收敛到单一幂次 p，并给出具体的标定方案（§8.2）——这是本文与上述工作的主要差别。

附录 B 以出口浓度为直接目标的对照建模

为便于与其他方案交叉比较，这里给出直接以 `C_out_mgNm3` 为目标建模的结果（训练/验证仍按时间划分）：

1. 以是否等于 50 为标签作分类，验证集 AUC=0.5098，与随机猜测无异；
2. 只用未截顶的样本做回归，验证集 $R^2 = -0.0190$；
3. 用全部记录值做回归，验证集 $R^2 = -0.0219$。

后两项的 R^2 为负，说明模型表现不及常数预测。因此，任何在该列上报告高 R^2 的模型，其性能几乎必然源于以下两种情形之一：随机划分数据导致的时间泄漏，或仅复现了 5491 条恒为 50 的截顶值。

附录 C 结果适用性声明

本文的功率模型、工况划分、历史操作规律、异常工况识别以及数据质量统计，均直接来自附件，可用 src/中的程序复算。

排放层的各项系数、振打峰值系数，以及由此得到的 5/10 mg/Nm³ 控制参数和功率增幅，则属于机理推演。若数据提供方给出新的未截顶排放数据，或现场标定得到的幂次与 $p = 2$ 不同，应按 §8.2 重新计算，不得沿用本文的数值。5 mg/Nm³ 档所需的电压外推，须先经高压电源容量与绝缘裕度核算。

附录 D 支撑材料与 AI 工具使用声明

求解流程：

```
输入：分钟级数据、排放限值C_lim
1. 按时间划分训练/验证/回顾性测试；
2. 逐列可识别性审计；截尾极大似然检验截顶假设；按各场设定周期作相位折叠，给出振打同步波动上界；提取
   历史操作规律与异常工况；
3. 在训练期对Temp、C_in、Q标准化，K-means取k=4；
4. 比较四种功率设定，选定 P = a0 + sum b_i U_i^2 + sum c_i / T_i；
5. 以记录值50 mg/Nm³为锚点建立灰箱排放层；由积灰量守恒 gamma=1；
6. 对每个工况与限值：
   6.1 取该工况控制量的历史最小—最大范围为支持域；
   6.2 多起点SLSQP求解（5结构起点＋48随机起点）；
   6.3 舍入至0.1 kV / 1 s 后按解析优先级修复可行性；
   6.4 以4×10^5点随机搜索独立对照；
   6.5 记录活跃约束；将T上界改取P95后重解，量化支持域取法的影响；
7. 报告马氏距离外推审计与所需电压裕度；
8. 扫描27组尺度情景与81组结构情景，给出幂次p参考表；
输出：控制参数、功率代价、外推审计、敏感性、图表与运行摘要。
```

源程序清单：src/scenario_model.py(功率模型、灰箱排放情景、支持域优化、活跃约束与敏感性)、src/outlet_diagnostics.py（截顶假设的截尾似然检验与振打周期性检验）、src/paper_figures.py（20 幅正文图与图注追踪表）、src/plot_utils.py（统一绘图样式）、src/integrity_check.py（确定性核验）、src/compute_frontier.py（排放—功率前沿）、src/build_paper_pdf.py（提交稿装配）。生成本文全部结果与图表的完整可运行源程序见附录 E，与电子支撑材料 src/ 目录逐字一致。

支撑材料清单： data/处理数据包；results/下功率基线、滚动验证、最优控制、活跃约束、振打周期上界敏感性、截顶与周期性检验、外推审计、电压裕度需求、求解对照、结构情景、场权重消融、幂次参考表、动态控制表和运行摘要；figures_paper/下 20 幅正文图（19 幅 PNG/PDF 由绘图程序生成，图 1 为 TikZ 源文件独立编译）及 figure_manifest.csv；integrity/下确定性核验结果。

AI 工具使用声明（提交前由参赛队核验确认）：本稿坚持参赛队人工主导。题意分析、模型构建、公式推导、参数设定、程序审阅、结果复算和论文定稿由参赛队承担；AI 编码助手仅用于代码排错、图表版式检查、文字校对与格式核验。AI 输出均经人工筛选和复核，未替代现场数据，也未把不可识别的排放关系改写为实证结论。

附录 E　完整源程序

本附录给出生成本文全部结果与图表的完整可运行源程序，与电子支撑材料 `src/` 目录逐字一致。按依赖顺序排列：情景模型与优化、排放—功率前沿、绘图样式与图集生成、确定性核验、提交稿装配。

E.1　`scenario_model.py`

```python
#!/usr/bin/env python3
"""Reproducible scenario model for the cement ESP problem.

Two layers are kept strictly separate:

* the **power layer** is identified from the supplied data.  Total power is a
  near-deterministic function of the control settings alone, with the physically
  motivated form ``P = a0 + sum_i b_i U_i^2 + sum_i c_i / T_i`` (field power
  proportional to voltage squared, rapping power proportional to rapping
  frequency ``60/T_i``);
* the **emission layer** is an explicitly labelled engineering scenario,
  because the recorded outlet concentration is capped near 50 mg/Nm^3 and
  carries no learnable signal.

The historical emission anchor is the *recorded* outlet concentration; no
decimal-point rescaling is applied.  The control problem solved here is
therefore the path from the historical ~50 mg/Nm^3 level down to the 10 and
5 mg/Nm^3 targets set by the problem statement.
"""

from __future__ import annotations

import json
import math
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import minimize


SEED = 20260805
ROOT = Path(__file__).resolve().parents[1]
DATA_FILE = ROOT / "data" / "full_timeseries_with_flags.csv"
CLUSTER_EVAL_FILE = ROOT / "data" / "condition_cluster_evaluation.csv"
OUT_DIR = ROOT / "results"

U_COLS = [f"U{i}_kV" for i in range(1, 5)]
T_COLS = [f"T{i}_s" for i in range(1, 5)]
COND_COLS = ["Temp_C", "C_in_gNm3", "Q_Nm3h"]

# ---- central scenario constants (engineering priors, not fitted to C_out) ----
ALPHA_CENTRAL = np.array([1.15, 1.10, 0.95, 0.80])   # per-field removal weights
BETA_CENTRAL = np.array([0.30, 0.27, 0.20, 0.17])    # rapping-period deviation loss
PEAK_SHARES = np.array([0.30, 0.30, 0.22, 0.18])     # per-field share of the peak proxy
P_EXPONENT_CENTRAL = 2.0     # K ~ U^p ; p = 2 is the Deutsch drift-velocity form
PEAK_EXPONENT_CENTRAL = 1.0  # derived: cake mass per rap ~ L*T  =>  peak ~ T^1
PEAK_SCALE_CENTRAL = 1.2
PHASE_SCALE_CENTRAL = 1.0
SAFETY_INDEX_CENTRAL = 0.08
# 5 mg/Nm^3 needs voltage above every field's historical maximum; the margin is
# an explicitly declared extrapolation of the historical envelope, not a fact.
HEADROOM_10 = 0.00
```

```python
HEADROOM_5 = 0.05

MULTISTART = 48
SWEEP_MULTISTART = 16   # sensitivity grids: fewer starts, cross-checked against the central solve


def _r2(y: np.ndarray, pred: np.ndarray) -> float:
    return float(1.0 - np.sum((y - pred) ** 2) / np.sum((y - y.mean()) ** 2))


def _rmse(y: np.ndarray, pred: np.ndarray) -> float:
    return float(np.sqrt(np.mean((y - pred) ** 2)))


# ------------------------------------------------------------------ power models

def voltage_polynomial(u: np.ndarray) -> np.ndarray:
    """Equivalent to PolynomialFeatures(degree=2, include_bias=False) for 4 voltages."""
    cols = [u[:, i] for i in range(4)]
    for i in range(4):
        for j in range(i, 4):
            cols.append(u[:, i] * u[:, j])
    return np.column_stack(cols)


class _ScaledOLS:
    """Least squares with per-column scaling.

    The scaling matters: ``1/T`` is of order 4e-3 while ``U^2`` is of order
    3e3, and an unscaled ``lstsq`` truncates the rapping columns as though they
    were rank-deficient, which makes the rapping term look irrelevant.
    """

    def __init__(self) -> None:
        self.beta: np.ndarray | None = None

    def _fit_design(self, x: np.ndarray, y: np.ndarray) -> None:
        xx = np.column_stack([np.ones(len(x)), x])
        scale = np.abs(xx).max(axis=0)
        scale[scale == 0] = 1.0
        beta, *_ = np.linalg.lstsq(xx / scale, y, rcond=None)
        self.beta = beta / scale

    def _predict_design(self, x: np.ndarray) -> np.ndarray:
        if self.beta is None:
            raise RuntimeError("model not fitted")
        return self.beta[0] + x @ self.beta[1:]


class PhysicalPowerModel(_ScaledOLS):
    """P = a0 + sum_i b_i U_i^2 + sum_i c_i / T_i  (optionally + condition terms)."""

    name = "physical_voltage_square_plus_rapping_frequency"

    def __init__(self, with_conditions: bool = False) -> None:
        super().__init__()
        self.with_conditions = with_conditions

    def design(self, frame: pd.DataFrame) -> np.ndarray:
        u = frame[U_COLS].to_numpy(float)
        t = frame[T_COLS].to_numpy(float)
        blocks = [u ** 2, 1.0 / t]
```

```
        if self.with_conditions:
            blocks.append(frame[COND_COLS].to_numpy(float))
        return np.column_stack(blocks)

    def fit(self, frame: pd.DataFrame, y: np.ndarray) -> "PhysicalPowerModel":
        self._fit_design(self.design(frame), y)
        return self

    def predict(self, frame: pd.DataFrame) -> np.ndarray:
        return self._predict_design(self.design(frame))

    # -- fast path used inside the optimiser ------------------------------
    @property
    def intercept(self) -> float:
        return float(self.beta[0])

    @property
    def b(self) -> np.ndarray:
        return np.asarray(self.beta[1:5], float)

    @property
    def c(self) -> np.ndarray:
        return np.asarray(self.beta[5:9], float)

    def power(self, u: np.ndarray, t: np.ndarray) -> np.ndarray:
        """Total power for raw control arrays (last axis = 4 fields)."""
        return self.intercept + np.sum(self.b * u ** 2, axis=-1) + np.sum(self.c / t, axis=-1)

    def field_power(self, u: np.ndarray) -> np.ndarray:
        return np.sum(self.b * u ** 2, axis=-1)

    def rapping_power(self, t: np.ndarray) -> np.ndarray:
        return np.sum(self.c / t, axis=-1)


class RidgePowerModel:
    """Standardised ridge baseline with the rapping period entered *linearly*.

    Retained as the misspecified reference point: it is the specification the
    first version of this paper used, and the comparison in
    ``power_model_baselines.csv`` shows what treating ``T`` as a linear term
    costs on the retrospective test window.
    """

    def __init__(self, alpha: float = 1e-3, quadratic: bool = True):
        self.alpha = alpha
        self.quadratic = quadratic
        self.mean: np.ndarray | None = None
        self.std: np.ndarray | None = None
        self.beta: np.ndarray | None = None

    def design(self, frame: pd.DataFrame) -> np.ndarray:
        u = frame[U_COLS].to_numpy(float)
        remainder = frame[T_COLS + COND_COLS].to_numpy(float)
        voltage = voltage_polynomial(u) if self.quadratic else u
        return np.column_stack([voltage, remainder])

    def fit(self, frame: pd.DataFrame, y: np.ndarray) -> None:
        x = self.design(frame)
        self.mean = x.mean(axis=0)
        self.std = x.std(axis=0)
        self.std[self.std == 0] = 1.0
```

```
        z = (x - self.mean) / self.std
        zz = np.column_stack([np.ones(len(z)), z])
        penalty = np.eye(zz.shape[1]) * math.sqrt(self.alpha)
        penalty[0, 0] = 0.0
        augmented_x = np.vstack([zz, penalty])
        augmented_y = np.concatenate([y, np.zeros(zz.shape[1])])
        self.beta = np.linalg.lstsq(augmented_x, augmented_y, rcond=None)[0]

    def predict(self, frame: pd.DataFrame) -> np.ndarray:
        if self.mean is None or self.std is None or self.beta is None:
            raise RuntimeError("model not fitted")
        x = self.design(frame)
        z = (x - self.mean) / self.std
        zz = np.column_stack([np.ones(len(z)), z])
        return np.einsum("ij,j->i", zz, self.beta)


def evaluate_power_models(
    train: pd.DataFrame,
    validation: pd.DataFrame,
    test: pd.DataFrame,
) -> tuple[PhysicalPowerModel, dict[str, object], pd.DataFrame, pd.DataFrame]:
    """Fit every candidate specification on the training window only."""
    alpha_grid = (1e-4, 1e-3, 1e-2, 1e-1, 1.0, 10.0)
    n = len(train)
    rolling_bounds = ((0, int(0.40 * n), int(0.60 * n)),
                      (0, int(0.60 * n), int(0.80 * n)),
                      (0, int(0.80 * n), n))
    rolling_rows: list[dict[str, object]] = []
    selected: dict[str, float] = {}
    for model_name, quadratic in (("linear_ridge_T", False), ("quadratic_ridge_T", True)):
        alpha_scores: list[tuple[float, float]] = []
        for alpha in alpha_grid:
            fold_rmses = []
            for fold, (start, fit_end, eval_end) in enumerate(rolling_bounds, start=1):
                fit = train.iloc[start:fit_end]
                evaluate = train.iloc[fit_end:eval_end]
                candidate = RidgePowerModel(alpha=alpha, quadratic=quadratic)
                candidate.fit(fit, fit["P_total_kW"].to_numpy(float))
                pred = candidate.predict(evaluate)
                actual = evaluate["P_total_kW"].to_numpy(float)
                rmse = _rmse(actual, pred)
                fold_rmses.append(rmse)
                rolling_rows.append({
                    "model": model_name, "alpha": alpha, "fold": fold,
                    "fit_rows": len(fit), "evaluation_rows": len(evaluate),
                    "r2": _r2(actual, pred), "rmse_kW": rmse,
                    "bias_kW": float(np.mean(pred - actual)),
                })
            alpha_scores.append((float(np.mean(fold_rmses)), alpha))
        selected[model_name] = min(alpha_scores)[1]

    # the physical specification has no tuning parameter; still report it per fold
    for fold, (start, fit_end, eval_end) in enumerate(rolling_bounds, start=1):
        fit = train.iloc[start:fit_end]
        evaluate = train.iloc[fit_end:eval_end]
        candidate = PhysicalPowerModel().fit(fit, fit["P_total_kW"].to_numpy(float))
        pred = candidate.predict(evaluate)
        actual = evaluate["P_total_kW"].to_numpy(float)
        rolling_rows.append({
            "model": "physical_invT", "alpha": np.nan, "fold": fold,
            "fit_rows": len(fit), "evaluation_rows": len(evaluate),
```

```
            "r2": _r2(actual, pred), "rmse_kW": _rmse(actual, pred),
            "bias_kW": float(np.mean(pred - actual)),
        })

    y_train = train["P_total_kW"].to_numpy(float)
    fitted: dict[str, object] = {}
    for model_name, quadratic in (("linear_ridge_T", False), ("quadratic_ridge_T", True)):
        m = RidgePowerModel(alpha=selected[model_name], quadratic=quadratic)
        m.fit(train, y_train)
        fitted[model_name] = m
    fitted["physical_invT"] = PhysicalPowerModel().fit(train, y_train)
    fitted["physical_invT_plus_conditions"] = PhysicalPowerModel(with_conditions=True).fit(train, y_train)

    mean_power = float(y_train.mean())
    baseline_rows: list[dict[str, object]] = []
    for split_name, frame in (("train", train), ("validation", validation), ("retrospective_test", test)):
        actual = frame["P_total_kW"].to_numpy(float)
        for model_name in ("mean_predictor", "linear_ridge_T", "quadratic_ridge_T",
                           "physical_invT", "physical_invT_plus_conditions"):
            pred = (np.full(len(frame), mean_power) if model_name == "mean_predictor"
                    else fitted[model_name].predict(frame))
            baseline_rows.append({
                "model": model_name,
                "selected_alpha": selected.get(model_name, np.nan),
                "split": split_name,
                "r2": _r2(actual, pred),
                "rmse_kW": _rmse(actual, pred),
                "bias_kW": float(np.mean(pred - actual)),
            })

    chosen: PhysicalPowerModel = fitted["physical_invT"]  # type: ignore[assignment]
    val_actual = validation["P_total_kW"].to_numpy(float)
    val_pred = chosen.predict(validation)
    test_actual = test["P_total_kW"].to_numpy(float)
    test_pred = chosen.predict(test)
    guard = float(np.quantile(np.abs(val_pred - val_actual), 0.90))
    metrics: dict[str, object] = {
        "model": chosen.name,
        "formula": "P = a0 + sum_i b_i U_i^2 + sum_i c_i / T_i",
        "fit_scope": "train_only",
        "intercept_kW": chosen.intercept,
        "b_field_kW_per_kV2": chosen.b.tolist(),
        "c_rapping_kW_s": chosen.c.tolist(),
        "train_r2": _r2(y_train, chosen.predict(train)),
        "validation_r2": _r2(val_actual, val_pred),
        "validation_rmse_kW": _rmse(val_actual, val_pred),
        "validation_bias_kW": float(np.mean(val_pred - val_actual)),
        "validation_abs_error_q90_kW": guard,
        "retrospective_test_r2": _r2(test_actual, test_pred),
        "retrospective_test_rmse_kW": _rmse(test_actual, test_pred),
        "retrospective_test_bias_kW": float(np.mean(test_pred - test_actual)),
        "test_protocol": "retrospective_not_blind_due_to_prior_package_use",
        "reporting_guard_rule": "point_prediction_plus_validation_absolute_error_q90",
        "ridge_selected_alphas": selected,
    }
    return chosen, metrics, pd.DataFrame(rolling_rows), pd.DataFrame(baseline_rows)


# ------------------------------------------------------------ condition layer

def condition_profiles(df: pd.DataFrame) -> pd.DataFrame:
    train = df[df["split"] == "train"].copy()
```

```
    overall_load = train["dust_load_kg_h"].median()
    labels = {
        1: "低温低浓度高流量",
        2: "高温中浓度低流量",
        3: "高温高浓度高流量",
        4: "低温高浓度低流量",
    }
    rows: list[dict[str, float | int | str]] = []
    for cluster, group in train.groupby("condition_cluster"):
        row: dict[str, float | int | str] = {
            "condition_cluster": int(cluster),
            "condition_name": labels[int(cluster)],
            "count": int(len(group)),
            "share": float(len(group) / len(train)),
            "Temp_C": float(group["Temp_C"].mean()),
            "C_in_gNm3": float(group["C_in_gNm3"].mean()),
            "Q_Nm3h": float(group["Q_Nm3h"].mean()),
            "dust_load_kg_h": float(group["dust_load_kg_h"].mean()),
            "C_out_recorded_median": float(group["C_out_mgNm3"].dropna().median()),
            "historical_power_kW": float(group["P_total_kW"].mean()),
        }
        for col in U_COLS + T_COLS:
            row[f"{col}_min"] = float(group[col].min())
            row[f"{col}_p05"] = float(group[col].quantile(0.05))
            row[f"{col}_median"] = float(group[col].median())
            row[f"{col}_p95"] = float(group[col].quantile(0.95))
            row[f"{col}_max"] = float(group[col].max())
        row["load_ratio_to_train_median"] = float(row["dust_load_kg_h"] / overall_load)
        row["K0_anchor"] = math.log(1000.0 * row["C_in_gNm3"] / row["C_out_recorded_median"])
        rows.append(row)
    return pd.DataFrame(rows).sort_values("condition_cluster").reset_index(drop=True)


def scenario_t_star(profile: pd.Series, global_t_median: np.ndarray,
                    load_exponent: float = -0.18) -> np.ndarray:
    load_ratio = float(profile["load_ratio_to_train_median"])
    raw = global_t_median * load_ratio ** load_exponent
    lower = np.array([profile[f"{c}_min"] for c in T_COLS], float)
    upper = np.array([profile[f"{c}_max"] for c in T_COLS], float)
    return np.clip(raw, lower, upper)


def support_statistics(train_group: pd.DataFrame) -> tuple[np.ndarray, np.ndarray, float]:
    """Empirical control-support centre, inverse covariance and d^2 cut-off."""
    hist = train_group[U_COLS + T_COLS].to_numpy(float)
    mu = hist.mean(axis=0)
    inv_cov = np.linalg.pinv(np.cov(hist, rowvar=False))
    delta = hist - mu
    historical_d2 = np.einsum("ij,jk,ik->i", delta, inv_cov, delta)
    return mu, inv_cov, float(np.quantile(historical_d2, 0.975))


# ------------------------------------------------------------ emission layer

def scenario_emission(
    profile: pd.Series,
    u: np.ndarray,
    t: np.ndarray,
    t_star: np.ndarray,
    p_exponent: float = P_EXPONENT_CENTRAL,
    peak_scale: float = PEAK_SCALE_CENTRAL,
    safety_index: float = SAFETY_INDEX_CENTRAL,
```

```
    alpha_weights: np.ndarray | None = None,
    peak_exponent: float = PEAK_EXPONENT_CENTRAL,
    phase_scale: float = PHASE_SCALE_CENTRAL,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Return continuous base emission, rapping peak excess, and their sum.

    ``K = K0 * S(U) - [G(T) - G(T_ref)]`` with
    ``S(U) = sum_i alpha_i (U_i/U_ref_i)^p / sum_i alpha_i``.

    At the historical anchor ``U = U_ref`` and ``T = T_ref`` this returns
    ``K = K0 = ln(1000 C_in / C_out_recorded)``, i.e. the model reproduces the
    recorded operating point by construction.  Every non-control argument is a
    scenario input; none is estimated from the recorded outlet series.
    """
    u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
    t_ref = np.array([profile[f"{c}_median"] for c in T_COLS], float)
    cin = float(profile["C_in_gNm3"])
    k_anchor = float(profile["K0_anchor"])

    if alpha_weights is None:
        alpha_weights = ALPHA_CENTRAL
    alpha = np.asarray(alpha_weights, float)
    strength = np.sum(alpha * (u / u_ref) ** p_exponent, axis=-1) / alpha.sum()

    load_ratio = float(profile["load_ratio_to_train_median"])
    beta = BETA_CENTRAL * load_ratio ** 0.5

    def loss(period: np.ndarray) -> np.ndarray:
        ratio = period / t_star
        return np.sum(beta * (ratio + 1.0 / ratio - 2.0), axis=-1)

    ref_loss = float(loss(t_ref))
    k = k_anchor * strength - (loss(t) - ref_loss)
    base = cin * 1000.0 * np.exp(-k + safety_index)

    # Derived form: cake mass per rap ~ L * T, so the per-event peak grows
    # linearly with the period (peak_exponent = 1) while the contribution to
    # the hourly mean, ~ L*T*(1/T) = L, does not depend on the period at all.
    peak = (peak_scale * phase_scale * load_ratio
            * np.sum(PEAK_SHARES * (t / t_star) ** peak_exponent, axis=-1))
    return base, peak, base + peak


def field_priority(profile: pd.Series, model: PhysicalPowerModel,
                   alpha_weights: np.ndarray | None = None) -> np.ndarray:
    """Removal gain per unit of electrical power for each field.

    Differentiating ``K = K0 * sum_i alpha_i (U_i/U_ref_i)^2 / sum(alpha)`` and
    ``P = sum_i b_i U_i^2`` with respect to ``U_i^2`` gives

        dK/dP |_i = K0 * alpha_i / (sum(alpha) * b_i * U_ref_i^2),

    so the field to raise first is the one with the largest
    ``alpha_i / U_ref_i^2`` -- *not* the largest ``alpha_i``.
    """
    alpha = ALPHA_CENTRAL if alpha_weights is None else np.asarray(alpha_weights, float)
    u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
    return float(profile["K0_anchor"]) * alpha / (alpha.sum() * model.b * u_ref ** 2)


# ---------------------------------------------------------------- optimisation
```

```python
def _bounds(profile: pd.Series, voltage_headroom: float,
            period_upper: str = "max") -> tuple[np.ndarray, np.ndarray]:
    """Support box for the eight controls.

    ``period_upper`` selects the upper bound of the rapping periods: ``"max"``
    is the historical maximum used throughout the paper, ``"p95"`` the 95th
    percentile.  Every optimum places all four periods exactly on this bound,
    so the choice -- not any interior trade-off -- fixes the rapping half of
    the solution; ``"p95"`` quantifies what that choice is worth.
    """
    lo = np.concatenate([[profile[f"{c}_min"] for c in U_COLS],
                         [profile[f"{c}_min"] for c in T_COLS]])
    t_key = "_p95" if period_upper == "p95" else "_max"
    hi = np.concatenate([[profile[f"{c}_max"] * (1.0 + voltage_headroom) for c in U_COLS],
                         [profile[f"{c}{t_key}"] for c in T_COLS]])
    return lo.astype(float), hi.astype(float)


#: a control counts as sitting on a bound when it is within this distance of it
BOUND_TOL = np.array([0.05] * 4 + [0.5] * 4)


def active_bounds(z: np.ndarray, lo: np.ndarray, hi: np.ndarray) -> dict[str, object]:
    """Which controls sit on their support bound, and on which side."""
    at_lo = np.abs(z - lo) <= BOUND_TOL
    at_hi = np.abs(z - hi) <= BOUND_TOL
    names = U_COLS + T_COLS
    return {
        "controls_at_lower_bound": "|".join(n for n, f in zip(names, at_lo) if f),
        "controls_at_upper_bound": "|".join(n for n, f in zip(names, at_hi) if f),
        "n_controls_at_bound": int(np.sum(at_lo | at_hi)),
        "all_periods_at_upper_bound": bool(np.all(at_hi[4:])),
    }


def _ordering(z: np.ndarray) -> np.ndarray:
    return np.array([z[0] - z[2] - 3.0, z[1] - z[3] - 3.0,
                     z[6] - z[4] - 100.0, z[7] - z[5] - 100.0])


def _round_controls(z: np.ndarray) -> np.ndarray:
    return np.concatenate([np.round(z[:4], 1), np.round(z[4:])])


def solve_condition(
    profile: pd.Series,
    model: PhysicalPowerModel,
    t_star: np.ndarray,
    limit: float,
    voltage_headroom: float = 0.0,
    mahalanobis: tuple[np.ndarray, np.ndarray, float] | None = None,
    multistart: int = MULTISTART,
    seed: int = SEED,
    period_upper: str = "max",
    **emission_kwargs,
) -> dict[str, object]:
    """Deterministic multi-start SLSQP solve of one condition/limit pair.

    The feasible set is *not* convex in ``U``: the objective ``sum b_i U_i^2``
    is convex while the emission constraint requires a convex function of
    ``U`` to be large enough, which is a reverse-convex constraint.  A dense
    multi-start is therefore used, and ``verify_with_random_search`` re-checks
```

```
    the result against a large random sample of the same feasible set.
    """
    lo, hi = _bounds(profile, voltage_headroom, period_upper)

    def objective(z: np.ndarray) -> float:
        return float(model.power(z[:4], z[4:]))

    def emission_slack(z: np.ndarray) -> float:
        return float(limit - scenario_emission(profile, z[:4], z[4:], t_star, **emission_kwargs)[2])

    constraints: list[dict[str, object]] = [
        {"type": "ineq", "fun": emission_slack},
        {"type": "ineq", "fun": _ordering},
    ]
    if mahalanobis is not None:
        mu, inv_cov, threshold = mahalanobis
        constraints.append({
            "type": "ineq",
            "fun": lambda z: float(threshold - (z - mu) @ inv_cov @ (z - mu)),
        })

    u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
    t_ref = np.array([profile[f"{c}_median"] for c in T_COLS], float)
    rng = np.random.default_rng(seed)
    starts = [np.concatenate([u_ref, t_ref]), hi.copy(), lo.copy(), 0.5 * (lo + hi),
              np.concatenate([hi[:4], t_star])]
    starts += [lo + (hi - lo) * rng.random(8) for _ in range(multistart)]

    best_z: np.ndarray | None = None
    best_val = np.inf
    converged = 0
    values: list[float] = []
    for z0 in starts:
        result = minimize(objective, np.clip(z0, lo, hi), method="SLSQP",
                          bounds=list(zip(lo, hi)), constraints=constraints,
                          options={"maxiter": 600, "ftol": 1e-10})
        if not result.success:
            continue
        z = np.clip(result.x, lo, hi)
        if emission_slack(z) < -1e-7 or np.any(_ordering(z) < -1e-7):
            continue
        converged += 1
        values.append(float(result.fun))
        if result.fun < best_val:
            best_val, best_z = float(result.fun), z

    if best_z is None:
        return {
            "condition_cluster": int(profile["condition_cluster"]),
            "condition_name": str(profile["condition_name"]),
            "limit_mgNm3": float(limit),
            "voltage_headroom": float(voltage_headroom),
            "status": "INFEASIBLE_IN_DECLARED_SUPPORT",
        }

    # Round to the resolution a DCS can actually hold, then repair feasibility
    # by nudging the most power-efficient field upward.
    z = _round_controls(best_z)
    order = np.argsort(-field_priority(profile, model, emission_kwargs.get("alpha_weights")))
    for _ in range(400):
        if emission_slack(z) >= 0 and np.all(_ordering(z) >= -1e-9):
            break
```

```
        for i in order:
            if z[i] + 0.1 <= hi[i] + 1e-9:
                z[i] = round(z[i] + 0.1, 1)
                break
        else:
            break
    if emission_slack(z) < 0:
        z = best_z  # keep the unrounded solution rather than report an infeasible one

    base, peak, total = scenario_emission(profile, z[:4], z[4:], t_star, **emission_kwargs)
    row: dict[str, object] = {
        "condition_cluster": int(profile["condition_cluster"]),
        "condition_name": str(profile["condition_name"]),
        "limit_mgNm3": float(limit),
        "voltage_headroom": float(voltage_headroom),
        "status": "SCENARIO_FEASIBLE",
        "scenario_base_emission_mgNm3": float(base),
        "scenario_peak_excess_mgNm3": float(peak),
        "scenario_peak_total_mgNm3": float(total),
        "predicted_power_kW": float(model.power(z[:4], z[4:])),
        "field_power_kW": float(model.field_power(z[:4])),
        "rapping_power_kW": float(model.rapping_power(z[4:])),
        "historical_power_kW": float(profile["historical_power_kW"]),
        "multistart_converged": converged,
        "multistart_power_spread_kW": float(np.max(values) - np.min(values)) if values else np.nan,
        "period_upper_rule": period_upper,
        "emission_slack_mgNm3": float(emission_slack(z)),
        "min_ordering_slack": float(np.min(_ordering(z))),
    }
    row.update(active_bounds(z, lo, hi))
    row["power_change_vs_historical_pct"] = 100.0 * (
        row["predicted_power_kW"] / float(profile["historical_power_kW"]) - 1.0)
    for i, col in enumerate(U_COLS + T_COLS):
        row[col] = float(z[i])
    return row


def verify_with_random_search(
    profile: pd.Series,
    model: PhysicalPowerModel,
    t_star: np.ndarray,
    limit: float,
    voltage_headroom: float,
    n: int = 400_000,
    seed: int = SEED,
    **emission_kwargs,
) -> float:
    """Best feasible power found by uniform random sampling of the same set."""
    lo, hi = _bounds(profile, voltage_headroom)
    rng = np.random.default_rng(seed)
    z = lo + (hi - lo) * rng.random((n, 8))
    u = np.round(z[:, :4], 1)
    t = np.round(z[:, 4:])
    keep = ((u[:, 0] >= u[:, 2] + 3.0) & (u[:, 1] >= u[:, 3] + 3.0)
            & (t[:, 2] >= t[:, 0] + 100.0) & (t[:, 3] >= t[:, 1] + 100.0))
    u, t = u[keep], t[keep]
    if not len(u):
        return float("nan")
    _, _, total = scenario_emission(profile, u, t, t_star, **emission_kwargs)
    feasible = total <= limit
    if not np.any(feasible):
        return float("nan")
```

```
    return float(np.min(model.power(u[feasible], t[feasible])))


def aggregate(rows: list[dict[str, object]], key: str = "predicted_power_kW") -> float:
    weights = np.array([float(r["share"]) for r in rows])
    return float(np.average([float(r[key]) for r in rows], weights=weights))


def solve_all(profiles: pd.DataFrame, model: PhysicalPowerModel,
              t_stars: dict[int, np.ndarray], limit: float, headroom: float,
              multistart: int = MULTISTART, period_upper: str = "max",
              **emission_kwargs) -> list[dict[str, object]]:
    rows = []
    for _, profile in profiles.iterrows():
        cluster = int(profile["condition_cluster"])
        r = solve_condition(profile, model, t_stars[cluster], limit,
                            voltage_headroom=headroom, multistart=multistart,
                            period_upper=period_upper, **emission_kwargs)
        r["share"] = float(profile["share"])
        rows.append(r)
    return rows


# ---------------------------------------------------- data-side diagnostics

def historical_strategy_correlations(df: pd.DataFrame) -> pd.DataFrame:
    """Operator response of the control settings to the exogenous conditions.

    Unlike the outlet concentration, this relationship *is* identifiable, and
    it gives Question 3 an empirical baseline.  It is reported per window as
    well as over the whole record, because the two are not the same: the
    rapping-period rule holds throughout, while the voltage response to inlet
    concentration reverses sign between the first five days and the last two.
    """
    windows = [("full_record", df), ("train", df[df["split"] == "train"]),
               ("validation", df[df["split"] == "validation"]),
               ("retrospective_test", df[df["split"] == "test"])]
    rows = []
    for name, frame in windows:
        for control in U_COLS + T_COLS:
            row: dict[str, object] = {"window": name, "rows": int(len(frame)), "control": control}
            for cond in COND_COLS:
                row[f"r_{cond}"] = float(np.corrcoef(frame[cond], frame[control])[0, 1])
            rows.append(row)
    return pd.DataFrame(rows)


def anomaly_events(df: pd.DataFrame) -> pd.DataFrame:
    """The two sustained excursions planted in the attachment."""
    specs = [
        ("入口浓度尖峰", df["C_in_gNm3"] > 60.0),
        ("入口温度尖峰", df["Temp_C"] > 150.0),
    ]
    baseline = {c: float(df[c].median()) for c in COND_COLS + U_COLS + T_COLS + ["P_total_kW"]}
    rows = []
    for name, mask in specs:
        sub = df[mask]
        row: dict[str, object] = {
            "event": name,
            "start": str(sub["timestamp"].min()),
            "end": str(sub["timestamp"].max()),
            "minutes": int(len(sub)),
```

```
        }
        for col in COND_COLS + U_COLS + T_COLS + ["P_total_kW"]:
            row[f"{col}_event_mean"] = float(sub[col].mean())
            row[f"{col}_event_max"] = float(sub[col].max())
            row[f"{col}_event_min"] = float(sub[col].min())
            row[f"{col}_baseline_median"] = baseline[col]
        rows.append(row)
    return pd.DataFrame(rows)


def rapping_power_tradeoff(train: pd.DataFrame, model: PhysicalPowerModel) -> pd.DataFrame:
    """How much power the rapping term holds, and what extending it recovers."""
    u_med = train[U_COLS].median().to_numpy(float)
    t_med = train[T_COLS].median().to_numpy(float)
    field = float(model.field_power(u_med))
    base_rap = float(model.rapping_power(t_med))
    total = model.intercept + field + base_rap
    rows = [{
        "scenario": "历史中位周期",
        "T_setting": ";".join(f"{v:.0f}" for v in t_med),
        "field_power_kW": field,
        "rapping_power_kW": base_rap,
        "total_power_kW": total,
        "rapping_share_pct": 100.0 * base_rap / total,
        "power_recovered_kW": 0.0,
        "cake_mass_per_rap_change_pct": 0.0,
    }]
    for label, t_new in (("延长至历史P95", train[T_COLS].quantile(0.95).to_numpy(float)),
                         ("延长至历史最大值", train[T_COLS].max().to_numpy(float))):
        rap = float(model.rapping_power(t_new))
        rows.append({
            "scenario": label,
            "T_setting": ";".join(f"{v:.0f}" for v in t_new),
            "field_power_kW": field,
            "rapping_power_kW": rap,
            "total_power_kW": model.intercept + field + rap,
            "rapping_share_pct": 100.0 * rap / (model.intercept + field + rap),
            "power_recovered_kW": base_rap - rap,
            "cake_mass_per_rap_change_pct": 100.0 * (float(np.mean(t_new / t_med)) - 1.0),
        })
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------- driver

def main() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
    df["condition_cluster"] = df["condition_cluster"].astype(int)
    profiles = condition_profiles(df)
    profiles.to_csv(OUT_DIR / "condition_profiles.csv", index=False, encoding="utf-8-sig")

    train = df[df["split"] == "train"].copy()
    validation = df[df["split"] == "validation"].copy()
    test = df[df["split"] == "test"].copy()

    model, metrics, rolling_metrics, baseline_metrics = evaluate_power_models(train, validation, test)
    rolling_metrics.to_csv(OUT_DIR / "power_model_rolling_validation.csv", index=False, encoding="utf-8-sig")
    baseline_metrics.to_csv(OUT_DIR / "power_model_baselines.csv", index=False, encoding="utf-8-sig")
    (OUT_DIR / "power_model_metrics.json").write_text(
        json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8")
    guard = float(metrics["validation_abs_error_q90_kW"])
```

```
736
737     pd.read_csv(CLUSTER_EVAL_FILE).to_csv(
738         OUT_DIR / "cluster_selection_metrics.csv", index=False, encoding="utf-8-sig")
739
740     historical_strategy_correlations(df).to_csv(
741         OUT_DIR / "historical_strategy_correlations.csv", index=False, encoding="utf-8-sig")
742     anomaly_events(df).to_csv(OUT_DIR / "anomaly_events.csv", index=False, encoding="utf-8-sig")
743     rapping_power_tradeoff(train, model).to_csv(
744         OUT_DIR / "rapping_power_tradeoff.csv", index=False, encoding="utf-8-sig")
745
746     global_t_median = train[T_COLS].median().to_numpy(float)
747     t_stars = {int(p["condition_cluster"]): scenario_t_star(p, global_t_median)
748                for _, p in profiles.iterrows()}
749
750     # -- field priority criterion (analytic) ------------------------------
751     priority_rows = []
752     for _, profile in profiles.iterrows():
753         crit = field_priority(profile, model)
754         u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
755         row = {"condition_cluster": int(profile["condition_cluster"]),
756                "condition_name": profile["condition_name"]}
757         for i in range(4):
758             row[f"U{i+1}_ref_kV"] = float(u_ref[i])
759             row[f"alpha_{i+1}"] = float(ALPHA_CENTRAL[i])
760             row[f"criterion_{i+1}_per_kW"] = float(crit[i])
761         row["rank_order"] = ">".join(f"E{i+1}" for i in np.argsort(-crit))
762         priority_rows.append(row)
763     pd.DataFrame(priority_rows).to_csv(
764         OUT_DIR / "field_priority_criterion.csv", index=False, encoding="utf-8-sig")
765
766     # -- central solutions ------------------------------------------------
767     rows10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10)
768     rows5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5)
769     optimum = pd.DataFrame(rows10 + rows5)
770     optimum["validation_error_adjusted_power_kW"] = optimum["predicted_power_kW"] + guard
771     optimum["validation_error_guard_kW"] = guard
772     optimum.to_csv(OUT_DIR / "optimal_controls_central_scenario.csv",
773                    index=False, encoding="utf-8-sig")
774
775     # -- active-constraint report -----------------------------------------
776     # Which controls the optimum leaves sitting on the edge of the support box.
777     # All four periods do, in every one of the eight solves: lengthening a
778     # period buys rapping power directly while costing only a sliver of the
779     # emission headroom, so the period half of the solution is a corner set by
780     # the box rather than by an interior trade-off.
781     active_cols = ["condition_cluster", "condition_name", "limit_mgNm3",
782                    "controls_at_lower_bound", "controls_at_upper_bound",
783                    "n_controls_at_bound", "all_periods_at_upper_bound",
784                    "emission_slack_mgNm3", "min_ordering_slack", "predicted_power_kW"]
785     pd.DataFrame(rows10 + rows5)[active_cols].to_csv(
786         OUT_DIR / "active_constraints.csv", index=False, encoding="utf-8-sig")
787
788     # -- what the period bound itself is worth ----------------------------
789     # Re-solve with the rapping-period ceiling at P95 instead of the historical
790     # maximum.  The maximum is a single extreme minute of the training week, so
791     # this quantifies how much of the reported optimum rests on that one point.
792     period_rows = []
793     for limit, headroom, base_rows in ((10.0, HEADROOM_10, rows10), (5.0, HEADROOM_5, rows5)):
794         p95_rows = solve_all(profiles, model, t_stars, limit, headroom, period_upper="p95")
795         for base, alt in zip(base_rows, p95_rows):
796             if alt.get("status") != "SCENARIO_FEASIBLE":
797                 period_rows.append({"condition_cluster": base["condition_cluster"],
```

```
                                    "limit_mgNm3": limit, "status": alt.get("status")})
                continue
            period_rows.append({
                "condition_cluster": base["condition_cluster"],
                "condition_name": base["condition_name"],
                "limit_mgNm3": limit,
                "status": "SCENARIO_FEASIBLE",
                "power_bound_max_kW": base["predicted_power_kW"],
                "power_bound_p95_kW": alt["predicted_power_kW"],
                "power_penalty_kW": alt["predicted_power_kW"] - base["predicted_power_kW"],
                "power_penalty_pct": 100.0 * (alt["predicted_power_kW"]
                                              / base["predicted_power_kW"] - 1.0),
                "share": base["share"],
            })
    period_sensitivity = pd.DataFrame(period_rows)
    period_sensitivity.to_csv(OUT_DIR / "period_bound_sensitivity.csv",
                              index=False, encoding="utf-8-sig")

    # -- support-domain audit (reported, not imposed) ------------------------
    audit_rows = []
    for row in rows10 + rows5:
        cluster = int(row["condition_cluster"])
        group = train[train["condition_cluster"] == cluster]
        mu, inv_cov, threshold = support_statistics(group)
        z = np.array([float(row[c]) for c in U_COLS + T_COLS])
        d2 = float((z - mu) @ inv_cov @ (z - mu))
        u_max = np.array([group[c].max() for c in U_COLS], float)
        audit_rows.append({
            "condition_cluster": cluster,
            "condition_name": row["condition_name"],
            "limit_mgNm3": row["limit_mgNm3"],
            "mahalanobis_d2": d2,
            "empirical_q975": threshold,
            "ratio_to_q975": d2 / threshold,
            "max_voltage_over_historical_max_pct": float(
                100.0 * max((float(row[c]) / m - 1.0) for c, m in zip(U_COLS, u_max))),
            "verdict": "SUPPORTED" if d2 <= threshold else "EXTRAPOLATION",
        })
    pd.DataFrame(audit_rows).to_csv(OUT_DIR / "support_boundary_audit.csv",
                                    index=False, encoding="utf-8-sig")

    # Does the historical support ellipsoid admit any feasible point at all?
    restricted_rows = []
    for _, profile in profiles.iterrows():
        cluster = int(profile["condition_cluster"])
        stats = support_statistics(train[train["condition_cluster"] == cluster])
        for limit, headroom in ((10.0, HEADROOM_10), (5.0, HEADROOM_5)):
            r = solve_condition(profile, model, t_stars[cluster], limit,
                                voltage_headroom=headroom, mahalanobis=stats)
            restricted_rows.append({
                "condition_cluster": cluster, "limit_mgNm3": limit,
                "status_with_mahalanobis_constraint": r["status"],
                "predicted_power_kW": r.get("predicted_power_kW", np.nan),
            })
    pd.DataFrame(restricted_rows).to_csv(
        OUT_DIR / "mahalanobis_constrained_feasibility.csv", index=False, encoding="utf-8-sig")

    # -- minimum voltage headroom needed at each limit -----------------------
    headroom_rows = []
    for _, profile in profiles.iterrows():
        cluster = int(profile["condition_cluster"])
        for limit in (10.0, 5.0):
```

```
            need = np.nan
            for m in np.arange(0.0, 0.31, 0.01):
                if solve_condition(profile, model, t_stars[cluster], limit,
                                   voltage_headroom=float(m), multistart=16
                                   )["status"] == "SCENARIO_FEASIBLE":
                    need = float(m)
                    break
            headroom_rows.append({
                "condition_cluster": cluster,
                "condition_name": profile["condition_name"],
                "limit_mgNm3": limit,
                "min_voltage_headroom_pct": 100.0 * need,
            })
    pd.DataFrame(headroom_rows).to_csv(
        OUT_DIR / "voltage_headroom_requirement.csv", index=False, encoding="utf-8-sig")

    # -- optimiser verification against dense random search ------------------
    verify_rows = []
    for _, profile in profiles.iterrows():
        cluster = int(profile["condition_cluster"])
        for limit, headroom, rows in ((10.0, HEADROOM_10, rows10), (5.0, HEADROOM_5, rows5)):
            solved = next(r for r in rows if r["condition_cluster"] == cluster)
            if solved["status"] != "SCENARIO_FEASIBLE":
                continue
            rnd = verify_with_random_search(profile, model, t_stars[cluster], limit, headroom)
            verify_rows.append({
                "condition_cluster": cluster,
                "condition_name": profile["condition_name"],
                "limit_mgNm3": limit,
                "slsqp_power_kW": solved["predicted_power_kW"],
                "random_search_power_kW": rnd,
                "random_search_gap_pct": 100.0 * (rnd / float(solved["predicted_power_kW"]) - 1.0),
                "multistart_converged": solved["multistart_converged"],
                "multistart_power_spread_kW": solved["multistart_power_spread_kW"],
            })
    verification = pd.DataFrame(verify_rows)
    verification.to_csv(OUT_DIR / "optimization_verification.csv", index=False, encoding="utf-8-sig")

    # -- Question 4 ----------------------------------------------------------
    q4_rows = []
    for _, profile in profiles.iterrows():
        cluster = int(profile["condition_cluster"])
        a = next(r for r in rows10 if r["condition_cluster"] == cluster)
        b = next(r for r in rows5 if r["condition_cluster"] == cluster)
        k10 = float(profile["K0_anchor"]) * float(
            np.sum(ALPHA_CENTRAL * (np.array([a[c] for c in U_COLS], float)
                                    / np.array([profile[f"{c}_median"] for c in U_COLS], float)) ** 2)
            / ALPHA_CENTRAL.sum())
        delta_k = math.log((10.0 - float(a["scenario_peak_excess_mgNm3"]))
                           / (5.0 - float(a["scenario_peak_excess_mgNm3"])))
        q4_rows.append({
            "condition_cluster": cluster,
            "condition_name": profile["condition_name"],
            "share": float(profile["share"]),
            "power_10_kW": float(a["predicted_power_kW"]),
            "power_5_kW": float(b["predicted_power_kW"]),
            "field_power_10_kW": float(a["field_power_kW"]),
            "K_at_10": k10,
            "delta_K_10_to_5": delta_k,
            "analytic_delta_power_kW": float(a["field_power_kW"]) * delta_k / k10,
            "numeric_delta_power_kW": float(b["predicted_power_kW"]) - float(a["predicted_power_kW"]),
            "increase_pct": 100.0 * (float(b["predicted_power_kW"]) / float(a["predicted_power_kW"]) - 1.0),
```

```
        })
    q4 = pd.DataFrame(q4_rows)
    q4.to_csv(OUT_DIR / "question4_by_condition.csv", index=False, encoding="utf-8-sig")
    weighted_p10 = float(np.average(q4["power_10_kW"], weights=q4["share"]))
    weighted_p5 = float(np.average(q4["power_5_kW"], weights=q4["share"]))
    historical_weighted = float(np.average(profiles["historical_power_kW"], weights=profiles["share"]))
    q4_summary = {
        "weighted_power_10_kW": weighted_p10,
        "weighted_power_5_kW": weighted_p5,
        "weighted_increase_pct": 100.0 * (weighted_p5 / weighted_p10 - 1.0),
        "historical_weighted_actual_power_kW": historical_weighted,
        "power_10_vs_historical_pct": 100.0 * (weighted_p10 / historical_weighted - 1.0),
        "daily_energy_10_MWh": weighted_p10 * 24.0 / 1000.0,
        "daily_energy_5_MWh": weighted_p5 * 24.0 / 1000.0,
        "daily_energy_historical_MWh": historical_weighted * 24.0 / 1000.0,
        "validation_error_guard_kW": guard,
    }
    (OUT_DIR / "question4_summary.json").write_text(
        json.dumps(q4_summary, ensure_ascii=False, indent=2), encoding="utf-8")

    # -- p reference table: the single scalar that needs site calibration ----
    # Holding the 10 mg/Nm^3 operating point fixed, scaling every voltage by f
    # gives K = K_10 f^p and P_field = P_field,10 f^2, so halving the limit
    # (delta_K) costs exactly P_field,10 [(1 + delta_K/K_10)^{2/p} - 1]:
    # the penalty is governed by 2/p and nothing else.
    p_rows = []
    for p_exp in (1.0, 1.5, 2.0, 2.5, 3.0):
        analytic = []
        for row in q4_rows:
            ratio = 1.0 + row["delta_K_10_to_5"] / row["K_at_10"]
            analytic.append({"share": row["share"],
                             "delta": row["field_power_10_kW"] * (ratio ** (2.0 / p_exp) - 1.0),
                             "p10": row["power_10_kW"]})
        w = np.array([a["share"] for a in analytic])
        delta = float(np.average([a["delta"] for a in analytic], weights=w))
        p10 = float(np.average([a["p10"] for a in analytic], weights=w))
        # required voltage headroom above the historical maximum, same reasoning
        f = float(np.mean([(1.0 + r["delta_K_10_to_5"] / r["K_at_10"]) ** (1.0 / p_exp)
                           for r in q4_rows]))
        r10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10,
                        multistart=SWEEP_MULTISTART, p_exponent=p_exp)
        r5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5,
                       multistart=SWEEP_MULTISTART, p_exponent=p_exp)
        feasible = all(r["status"] == "SCENARIO_FEASIBLE" for r in r10 + r5)
        p_rows.append({
            "p_exponent": p_exp,
            "analytic_delta_power_kW": delta,
            "analytic_increase_pct": 100.0 * delta / p10,
            "implied_voltage_scaling_10_to_5": f,
            "feasible_at_5pct_headroom": feasible,
            "numeric_weighted_power_10_kW": aggregate(r10) if feasible else np.nan,
            "numeric_weighted_power_5_kW": aggregate(r5) if feasible else np.nan,
            "numeric_increase_pct": (100.0 * (aggregate(r5) / aggregate(r10) - 1.0)) if feasible else np.nan,
        })
    pd.DataFrame(p_rows).to_csv(OUT_DIR / "p_exponent_reference.csv",
                                index=False, encoding="utf-8-sig")

    # -- scale sensitivity: p x peak scale x safety index (27 combinations) --
    sensitivity_rows = []
    for p_exp in (1.5, 2.0, 2.5):
        for peak_scale in (0.9, 1.2, 1.5):
            for safety in (0.0, 0.08, 0.15):
```

```
                kw = dict(p_exponent=p_exp, peak_scale=peak_scale, safety_index=safety)
                r10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10,
                                multistart=SWEEP_MULTISTART, **kw)
                r5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5,
                               multistart=SWEEP_MULTISTART, **kw)
                ok = all(r["status"] == "SCENARIO_FEASIBLE" for r in r10 + r5)
                sensitivity_rows.append({
                    "p_exponent": p_exp, "peak_scale": peak_scale, "safety_index": safety,
                    "all_conditions_feasible": ok,
                    "weighted_power_10_kW": aggregate(r10) if ok else np.nan,
                    "weighted_power_5_kW": aggregate(r5) if ok else np.nan,
                    "weighted_power_increase_pct": (100.0 * (aggregate(r5) / aggregate(r10) - 1.0)) if ok else np.nan
    ↪ ,
                })
    sensitivity = pd.DataFrame(sensitivity_rows)
    sensitivity.to_csv(OUT_DIR / "question4_sensitivity.csv", index=False, encoding="utf-8-sig")

    # -- structural sensitivity: 81 grey-box shape combinations --------------
    alpha_profiles = {
        "equal": np.array([1.00, 1.00, 1.00, 1.00]),
        "weak_front": np.array([1.08, 1.04, 0.98, 0.90]),
        "central_front": ALPHA_CENTRAL,
    }
    structural_rows = []
    for alpha_name, alpha_weights in alpha_profiles.items():
        for peak_exponent in (1.00, 1.30, 1.60):
            for phase_scale in (0.65, 0.80, 1.00):
                for tstar_exponent in (-0.30, -0.18, -0.10):
                    stars = {int(p["condition_cluster"]): scenario_t_star(p, global_t_median, tstar_exponent)
                             for _, p in profiles.iterrows()}
                    kw = dict(alpha_weights=alpha_weights, peak_exponent=peak_exponent,
                              phase_scale=phase_scale)
                    r10 = solve_all(profiles, model, stars, 10.0, HEADROOM_10,
                                    multistart=SWEEP_MULTISTART, **kw)
                    r5 = solve_all(profiles, model, stars, 5.0, HEADROOM_5,
                                   multistart=SWEEP_MULTISTART, **kw)
                    ok = all(r["status"] == "SCENARIO_FEASIBLE" for r in r10 + r5)
                    structural_rows.append({
                        "alpha_profile": alpha_name,
                        "peak_exponent": peak_exponent,
                        "phase_scale": phase_scale,
                        "tstar_load_exponent": tstar_exponent,
                        "all_conditions_feasible": ok,
                        "weighted_power_10_kW": aggregate(r10) if ok else np.nan,
                        "weighted_power_5_kW": aggregate(r5) if ok else np.nan,
                        "weighted_power_increase_pct": (100.0 * (aggregate(r5) / aggregate(r10) - 1.0)) if ok else np
    ↪ .nan,
                    })
    structural = pd.DataFrame(structural_rows)
    structural.to_csv(OUT_DIR / "structural_sensitivity.csv", index=False, encoding="utf-8-sig")

    # -- field weight ablation ------------------------------------------------
    ablation_rows, ablation_summary_rows = [], []
    for alpha_name, alpha_weights in alpha_profiles.items():
        r10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10,
                        multistart=SWEEP_MULTISTART, alpha_weights=alpha_weights)
        r5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5,
                       multistart=SWEEP_MULTISTART, alpha_weights=alpha_weights)
        per_profile = []
        for a, b in zip(r10, r5):
            row = {"alpha_profile": alpha_name, "condition_cluster": a["condition_cluster"],
                   "condition_name": a["condition_name"], "share": a["share"]}
```

```
            for i, col in enumerate(U_COLS, start=1):
                row[f"delta_U{i}_kV"] = float(b[col]) - float(a[col])
            per_profile.append(row)
            ablation_rows.append(row)
        weights = np.array([x["share"] for x in per_profile])
        deltas = np.array([[x[f"delta_U{i}_kV"] for i in range(1, 5)] for x in per_profile])
        weighted = np.average(deltas, axis=0, weights=weights)
        positive = np.maximum(weighted, 0)
        ablation_summary_rows.append({
            "alpha_profile": alpha_name,
            **{f"weighted_delta_U{i}_kV": weighted[i - 1] for i in range(1, 5)},
            "front_two_share_of_positive_adjustment": float(positive[:2].sum() / positive.sum())
            if positive.sum() else np.nan,
        })
    pd.DataFrame(ablation_rows).to_csv(
        OUT_DIR / "field_priority_ablation_by_condition.csv", index=False, encoding="utf-8-sig")
    pd.DataFrame(ablation_summary_rows).to_csv(
        OUT_DIR / "field_priority_ablation_summary.csv", index=False, encoding="utf-8-sig")

    # -- historical comparison and online schedule --------------------------
    historical_rows, dynamic_rows = [], []
    for _, profile in profiles.iterrows():
        cluster = int(profile["condition_cluster"])
        u_med = np.array([profile[f"{c}_median"] for c in U_COLS], float)
        t_med = np.array([profile[f"{c}_median"] for c in T_COLS], float)
        median_power = float(model.power(u_med, t_med))
        a = next(r for r in rows10 if r["condition_cluster"] == cluster)
        b = next(r for r in rows5 if r["condition_cluster"] == cluster)
        historical_rows.append({
            "condition_cluster": cluster,
            "condition_name": profile["condition_name"],
            "share": float(profile["share"]),
            "historical_mean_actual_power_kW": float(profile["historical_power_kW"]),
            "model_power_at_historical_median_controls_kW": median_power,
            "scenario_optimal_10_power_kW": float(a["predicted_power_kW"]),
            "power_change_vs_model_median_pct": 100.0 * (float(a["predicted_power_kW"]) / median_power - 1.0),
        })
        dynamic_rows.append({
            "condition_cluster": cluster,
            "condition_name": profile["condition_name"],
            "mode_selection_rule": "EXTERNAL_TARGET_SELECTION",
            "available_modes": "TARGET-10/TARGET-5/FALLBACK",
            "automatic_emission_threshold_switching": False,
            "provisional_voltage_ramp_kV_per_min": 0.5,
            "provisional_period_ramp_s_per_min": 5.0,
            "phase_scenario_for_online_trial": "0.65/0.80/1.00",
            "implementation_status": "PROVISIONAL_REQUIRES_T/R_CAPACITY_CHECK_AND_SITE_INTERLOCK_LIMITS",
            **{f"target10_{c}": a[c] for c in U_COLS + T_COLS},
            **{f"target5_{c}": b[c] for c in U_COLS + T_COLS},
        })
    pd.DataFrame(historical_rows).to_csv(
        OUT_DIR / "historical_median_power_comparison.csv", index=False, encoding="utf-8-sig")
    pd.DataFrame(dynamic_rows).to_csv(
        OUT_DIR / "dynamic_control_schedule.csv", index=False, encoding="utf-8-sig")

    scenario_params = {
        "status": "EMISSION_LAYER_IS_SCENARIO_ONLY_C_OUT_NOT_IDENTIFIABLE",
        "emission_anchor": "recorded outlet median, no decimal rescaling",
        "p_exponent_central": P_EXPONENT_CENTRAL,
        "p_exponent_grid": [1.0, 1.5, 2.0, 2.5, 3.0],
        "peak_exponent_central": PEAK_EXPONENT_CENTRAL,
        "peak_exponent_grid": [1.00, 1.30, 1.60],
```

```
        "voltage_weight_profiles": {k: v.tolist() for k, v in alpha_profiles.items()},
        "rapping_phase_scales": [0.65, 0.80, 1.00],
        "tstar_load_exponents": [-0.30, -0.18, -0.10],
        "rapping_loss_coefficients": BETA_CENTRAL.tolist(),
        "peak_field_shares": PEAK_SHARES.tolist(),
        "support_rule": "cluster-specific full historical range plus engineering ordering; "
                        "Mahalanobis d2 reported as extrapolation audit, not imposed",
        "voltage_headroom_10": HEADROOM_10,
        "voltage_headroom_5": HEADROOM_5,
        "optimiser": "multi-start SLSQP (%d starts) verified against 400k random samples" % MULTISTART,
    }
    (OUT_DIR / "scenario_parameters.json").write_text(
        json.dumps(scenario_params, ensure_ascii=False, indent=2), encoding="utf-8")

    feasible_structural = structural[structural["all_conditions_feasible"]]
    feasible_scale = sensitivity[sensitivity["all_conditions_feasible"]]
    summary = {
        "power_model": metrics,
        "cluster_k": 4,
        "central_q4": q4_summary,
        "rapping_share_at_median_pct": float(
            rapping_power_tradeoff(train, model).iloc[0]["rapping_share_pct"]),
        "scale_sensitivity": {
            "feasible_combinations": int(len(feasible_scale)),
            "total_combinations": int(len(sensitivity)),
            "increase_pct_min": float(feasible_scale["weighted_power_increase_pct"].min()),
            "increase_pct_median": float(feasible_scale["weighted_power_increase_pct"].median()),
            "increase_pct_max": float(feasible_scale["weighted_power_increase_pct"].max()),
        },
        "structural_sensitivity": {
            "feasible_combinations": int(len(feasible_structural)),
            "total_combinations": int(len(structural)),
            "increase_pct_min": float(feasible_structural["weighted_power_increase_pct"].min()),
            "increase_pct_median": float(feasible_structural["weighted_power_increase_pct"].median()),
            "increase_pct_max": float(feasible_structural["weighted_power_increase_pct"].max()),
        },
        "optimiser_verification": {
            "max_random_search_gap_pct": float(verification["random_search_gap_pct"].max()),
            "max_multistart_spread_kW": float(verification["multistart_power_spread_kW"].max()),
        },
        "central_all_conditions_feasible": bool((optimum["status"] == "SCENARIO_FEASIBLE").all()),
    }
    (OUT_DIR / "model_run_summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(summary, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

E.2 `outlet_diagnostics.py`

```
#!/usr/bin/env python3
"""Falsifiable tests on the recorded outlet concentration.

Section 2.2 of the paper rules the outlet column unusable, and section 4.5
derives the rapping-peak scaling instead of measuring it.  Both statements are
negative, and a negative claim of the form "we looked and found nothing" is
weaker than it needs to be.  This module replaces each with a test that could
have failed and did not:

* **censoring.**  If the column really is a latent concentration clipped at the
```

```
  50 mg/Nm^3 range limit, then fitting the latent normal by censored maximum
  likelihood on the *uncensored* records must reproduce the *observed*
  proportion of records sitting exactly on the cap.  That proportion is not
  used in the fit, so agreement is a genuine out-of-sample prediction.

* **rapping periodicity.**  Individual rapping events cannot be attributed at
  one-minute resolution, but the set periods (T1, T2 ~ 3.9 min; T3, T4 ~ 7.4
  min) all sit above the 2-minute Nyquist limit, so a *periodic* component
  synchronised with them is detectable.  Epoch folding of the outlet residual
  bounds any such component; because upward excursions are the ones the cap
  hides, the censoring indicator is folded as well, which is the test that
  survives clipping.

Outputs ``results/outlet_censoring_periodicity.csv``.
"""

from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd
from scipy import optimize, stats

ROOT = Path(__file__).resolve().parents[1]
DATA_FILE = ROOT / "data" / "full_timeseries_with_flags.csv"
OUT_DIR = ROOT / "results"

CAP = 50.0
T_COLS = [f"T{i}_s" for i in range(1, 5)]
N_PHASE_BINS = 10


def censoring_test(outlet: np.ndarray) -> dict[str, float]:
    """Censored-normal MLE fitted without using the observed cap fraction."""
    uncensored = outlet[outlet < CAP]
    n_censored = int((outlet >= CAP).sum())

    def negative_log_likelihood(theta: np.ndarray) -> float:
        mu, log_sigma = theta
        sigma = float(np.exp(log_sigma))
        return -(stats.norm.logpdf(uncensored, mu, sigma).sum()
                 + n_censored * stats.norm.logsf(CAP, mu, sigma))

    fit = optimize.minimize(negative_log_likelihood, [CAP + 0.2, np.log(0.3)],
                            method="Nelder-Mead",
                            options={"xatol": 1e-8, "fatol": 1e-8, "maxiter": 4000})
    mu, sigma = float(fit.x[0]), float(np.exp(fit.x[1]))
    predicted = float(stats.norm.sf(CAP, mu, sigma))
    observed = n_censored / len(outlet)
    return {
        "latent_mu_mgNm3": mu,
        "latent_sigma_mgNm3": sigma,
        "predicted_censored_fraction": predicted,
        "observed_censored_fraction": observed,
        "absolute_error": abs(predicted - observed),
        # the latent spread as a share of the level: how much room the column
        # has to carry any information about the controls at all
        "latent_cv_pct": 100.0 * sigma / mu,
    }


```

```
def epoch_folding(values: np.ndarray, period_minutes: float) -> float:
    """Peak-to-peak amplitude of ``values`` folded on ``period_minutes``."""
    phase = (np.arange(len(values)) % period_minutes) / period_minutes
    binned = np.minimum((phase * N_PHASE_BINS).astype(int), N_PHASE_BINS - 1)
    means = np.array([values[binned == k].mean() for k in range(N_PHASE_BINS)])
    return float(means.max() - means.min())


def censoring_phase_test(censored: np.ndarray, period_minutes: float) -> float:
    """p-value for the censoring rate depending on rapping phase.

    A real rapping peak pushes the concentration *up*, and up is exactly what
    the 50 mg/Nm^3 cap removes.  So the sharp test is not on the recorded
    values but on which minutes are clipped: a periodic peak would clip
    preferentially at one phase.
    """
    phase = (np.arange(len(censored)) % period_minutes) / period_minutes
    binned = np.minimum((phase * N_PHASE_BINS).astype(int), N_PHASE_BINS - 1)
    table = np.array([[int(((binned == k) & (censored == 1)).sum()) for k in range(N_PHASE_BINS)],
                      [int(((binned == k) & (censored == 0)).sum()) for k in range(N_PHASE_BINS)]])
    return float(stats.chi2_contingency(table).pvalue)


def main() -> None:
    df = pd.read_csv(DATA_FILE).dropna(subset=["C_out_mgNm3"]).reset_index(drop=True)
    outlet = df["C_out_mgNm3"].to_numpy(float)

    rows: list[dict[str, object]] = []
    censoring = censoring_test(outlet)
    rows.append({
        "test": "censored_normal_mle",
        "target": "C_out_mgNm3",
        "statistic": censoring["predicted_censored_fraction"],
        "reference": censoring["observed_censored_fraction"],
        "detail": (f"latent N(mu={censoring['latent_mu_mgNm3']:.3f}, "
                   f"sigma={censoring['latent_sigma_mgNm3']:.3f}) censored at {CAP:g}; "
                   f"latent CV {censoring['latent_cv_pct']:.2f}% of level"),
        "verdict": ("censoring hypothesis reproduces the observed cap fraction to "
                    f"{censoring['absolute_error']:.4f}"),
    })

    residual = outlet - outlet.mean()
    censored_flag = (outlet >= CAP).astype(int)
    residual_sd = float(residual.std())
    for col in T_COLS:
        period_minutes = float(df[col].median()) / 60.0
        amplitude = epoch_folding(residual, period_minutes)
        p_value = censoring_phase_test(censored_flag, period_minutes)
        rows.append({
            "test": "epoch_folding",
            "target": col,
            "statistic": amplitude,
            "reference": residual_sd,
            "detail": (f"median period {period_minutes:.2f} min "
                       f"(Nyquist limit 2 min); censoring-rate chi2 p={p_value:.3f}"),
            "verdict": (f"no rapping-synchronous component above {amplitude:.3f} mg/Nm3 "
                        f"({100.0 * amplitude / residual_sd:.1f}% of residual sd)"),
        })

    out = pd.DataFrame(rows)
    OUT_DIR.mkdir(exist_ok=True)
    out.to_csv(OUT_DIR / "outlet_censoring_periodicity.csv",
```

```
                index=False, encoding="utf-8-sig")
    print(out.to_string(index=False))


if __name__ == "__main__":
    main()
```

E.3 paper_figures.py

```
#!/usr/bin/env python3
"""Generate the complete publication figure set for the cement ESP paper.

All empirical plots are regenerated from the supplied processed dataset and
model result tables. Scenario-only plots are explicitly labelled in captions.
Both PNG previews and vector PDF files are exported.
"""

from __future__ import annotations

import sys
import hashlib
from pathlib import Path

import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
import numpy as np
import pandas as pd

SRC_DIR = Path(__file__).resolve().parent
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))

from plot_utils import (GRAY, LIGHT_GRAY, NEUTRAL_FILL, PALETTE, add_subpanel_label,
                        figure_legend, finish_axes, headroom, note, save_figure, set_paper_style)
from scenario_model import (ALPHA_CENTRAL, PhysicalPowerModel, RidgePowerModel,
                            T_COLS, U_COLS)


ROOT = SRC_DIR.parent
DATA_FILE = ROOT / "data" / "full_timeseries_with_flags.csv"
RESULTS = ROOT / "results"
OUT = ROOT / "figures_paper"

COND_SHORT = {
    1: "工况1\n低温低浓高流",
    2: "工况2\n高温中浓低流",
    3: "工况3\n高温高浓高流",
    4: "工况4\n低温高浓低流",
}

CAPTIONS = {
    "01_outlet_data_diagnosis": "出口浓度记录存在显著截顶且不覆盖目标区间。有效记录全部位于48.74～50.00 mg/Nm³，其中5491条恰好等于50，5与10 mg/Nm³目标区间内没有任何训练样本，因而无法由该列直接回归出目标区间附近的控制—排放关系。",
    "02_process_timeseries": "首24小时入口温度、入口浓度、烟气流量与总功率的分钟级变化。入口变量与功率具有连续变化和工况迁移特征，与图2中被压缩在单点附近的出口浓度形成对照。",
    "22_technical_roadmap": "全文技术路线。可识别性审计把论证分为两条证据强度不同的路径：实证数据层（实线）可由附件复算，条件式灰箱层（虚线）须经现场标定后方可采信；两条路径仅在支持域优化阶段合并，四问结论按其所属路径分别标注。",
    "04b_operating_evidence": "附件中可识别的另一半信息。(a)(b) 为两段持续异常工况及控制端的同步响应：入口浓度尖峰时抬高$U_1$、功率随之上冲，入口温度尖峰（比电阻高值区）时反而降压降功率；(c) 为整周记录中工况变量与控制设定的相关矩阵，显示入口浓度升高时缩短各场振打周期这一稳定规律。全部为历史操作的相关关系，不构成排放侧的因果证据。",
```

```
    "14_rapping_peak_mechanism": "振打周期同时牵动电耗与瞬时峰值。(a) 由每周期积灰量$\\propto L\\cdot T$推得单次再飞扬峰值随周期近似线性放大，而在单位面积灰量对应的再飞扬比例近似不随周期变化的前提下，单位时间再飞扬量的一阶近似与周期无关（水平点线）；(b) 振打环节在中位工况下占总功率约四成，把周期由中位延长到历史极值可回收约109 kW，代价是单次积灰量增加约18%。",
    "03_cluster_selection": "两项内部指标并不一致：轮廓系数在$k=4$取最大值，Calinski-Harabasz指标却在$k=5$取最大值。因此$k=4$并非唯一的\"数学最优\"聚类数，本文取$k=4$的依据是轮廓系数最高、四类簇心构成清晰的温度×流量结构，且四类比五类更便于控制解释。",
    "04_condition_map": "四类工况在入口浓度—温度平面上形成清晰分区，点的大小表示烟气流量，从而同时呈现三维工况信息。",
    "07_power_prediction": "总功率由控制量近乎确定地决定。物理设定$P=a_0+\\sum b_iU_i^2+\\sum c_i/T_i$在回顾性测试段的$R^2$与RMSE均显著优于均值基线及把振打周期作为线性项的岭回归设定。",
    "08b_feature_importance": "振打周期必须以$1/T$进入功率模型。(a) 扣除电场功率后的剩余功率对$T_1$呈明显双曲形状，线性设定（虚线）在两端系统性偏离；(b) 线性$T$设定在回顾性测试段留下约−10.8 kW的平均偏差，改为$1/T$后偏差降至约0，说明该偏差来自设定误差而非数据漂移。",
    "08_power_residuals": "改用$1/T$设定后，回顾性测试残差围绕零对称分布，原先高功率区负偏差扩大的现象消失，确认修正后的误差不再含系统性结构。",
    "05b_boundary_proximity": "单变量幅值合规不等于组合落在历史联合运行经验之内。(a) 八组解的马氏距离平方为经验97.5%外壳的2.4〜7.0倍；(b) 10 mg/Nm³解的各场电压均未超出该工况历史最大值，5 mg/Nm³解则需超出最多约5%。",
    "19_emission_power_frontier": "排放—功率权衡前沿。限值由10逐档收紧至5 mg/Nm³时四类工况的最优功率单调上升，工况加权功率由约1973升至约2105 kW；两档均高于历史实际加权功率1783.2 kW，说明超低排放改造是以增电耗换减排，而非节能。",
    "11_optimal_voltage": "两档限值下的电压配置并非四场机械同步抬升：分配由单位功率去除收益$\\alpha_i/(b_iU_{i}^{ref\\,2})$与排序约束共同决定，工况3、4中$U_2$明显低于$U_1$，工况4的$U_4$被压至历史下界。",
    "13_emission_decomposition": "限值裕度由连续排放基值与振打峰值增量共同占用。柱顶虚线分别表示10和5 mg/Nm³情景限值；峰值项占用0.95〜1.61 mg/Nm³，故约束必须按$C_{peak}=C_{base}+B$整体施加，而不能只约束$C_{base}$。",
    "18_condition_energy_penalty": "闭式估计与八变量数值最优解的一致性检验。(a) 四类工况的点均落在$y=x$线附近；(b) 相对偏差为−0.04%〜+0.97%，全部落在±1%带内，说明按四场电压同比缩放导出的解析式可作为第四问功率代价的解析解释。",
    "15_search_convergence": "多起点SLSQP与同一可行域内4×10^5点随机搜索的对照。八个工况—限值组合中，47〜53个起点全部收敛到同一解（功率一致到10^-7 kW以内）；随机搜索均未找到更优解，其最优值反而高出0.97%〜5.17%，工况1的10 mg/Nm³组合更是一个可行样本都没采到。这构成数值稳定性的强证据，但不是全局最优的数学证明。",
    "16_sensitivity_distribution": "81组结构情景全部可行。峰值指数在本文推导值1.00附近变化几乎不改变功率增幅，相位叠加尺度与场权重形状的影响也有限；加权增幅为5.13%〜7.55%，中位数6.40%，说明核心结论对灰箱形状设定不敏感。",
    "20_eta_sensitivity": "功率增幅由幂次$p$主导。$K\\propto U^p$时增幅近似正比于$2/p$：$p=1$为14.2%，$p=2$（Deutsch迁移速度形式）为6.8%，$p=3$为4.4%。其中$p=1$与$p=1.5$在5%电压裕度内无解，说明$p$同时决定代价大小与目标可达性，是首要待标定参数。",
    "17_sensitivity_heatmap": "场权重消融结果。三种去除贡献权重形状下，10→5 mg/Nm³所需的加权电压增量分布与按$\\alpha_i/(b_iU_{i}^{ref\\,2})$排序的解析优先级一致，说明优先级结论不由中心权重单独写入。",
}

TRACE_META = {
    "01_outlet_data_diagnosis": (str(DATA_FILE), "fig01_outlet_diagnosis", "§2.2；图2", "出口记录为量程截顶；不代表真实排放分布"),
    "02_process_timeseries": (str(DATA_FILE), "fig02_timeseries", "§2.2；图3", "只展示首24 h，不代表全周期分布"),
    "22_technical_roadmap": (str(ROOT / "10_修订后完整论文_终稿.md"), "fig22_roadmap", "§3.3；图4", "概念流程图，不含新的经验数据"),
    "04b_operating_evidence": (str(RESULTS / "historical_strategy_correlations.csv"), "fig04b_operating_evidence", "§4.2—4.3；图5", "相关系数为同期线性关联，不等于因果；两段事件为个案，样本量有限"),
    "14_rapping_peak_mechanism": (str(RESULTS / "rapping_power_tradeoff.csv"), "fig14_peak", "§4.5；图6", "峰值尺度$b_0$仍为工程先验；小时均值结论依赖再飞扬比例近似恒定的前提"),
    "03_cluster_selection": (str(RESULTS / "cluster_selection_metrics.csv"), "fig03_cluster_selection", "§5.1；图7", "聚类指标只评价统计分离度，不等于物理工况唯一性"),
    "04_condition_map": (str(DATA_FILE), "fig04_condition_map", "§5.1；图8", "为可视化抽样点；聚类实际使用全部训练样本"),
    "07_power_prediction": (str(RESULTS / "power_model_baselines.csv"), "fig07_power_prediction + PhysicalPowerModel", "§5.2；图9", "回顾性测试并非真正未触碰盲测"),
    "08b_feature_importance": (str(RESULTS / "power_model_baselines.csv"), "fig08b_specification", "§5.2；图10", "残差图按$T_1$展开，其余三场周期固定在中位数"),
    "08_power_residuals": (str(DATA_FILE), "fig08_residuals + PhysicalPowerModel", "§5.2；图11", "残差接近零均值，但仍非全新日期盲测"),
    "05b_boundary_proximity": (str(RESULTS / "support_boundary_audit.csv"), "fig05b_boundary_proximity", "§5.6；图12", "经验支持域不是设备安全边界；超出部分必须由现场阶跃试验确认"),
```

```
    "19_emission_power_frontier": (str(RESULTS / "emission_power_frontier.csv"), "fig19_frontier", "§5.7；图13", "各限值下均为灰箱情景解，前沿形状依赖排放先验"),
    "11_optimal_voltage": (str(RESULTS / "optimal_controls_central_scenario.csv"), "grouped_controls(U_COLS)", "§6；图14", "最优电压是情景试验起点，不是设备安全设定"),
    "18_condition_energy_penalty": (str(RESULTS / "question4_by_condition.csv"), "fig18_penalty", "§7.2；图15", "一致性只说明解析近似可用，不改变灰箱先验的条件性"),
    "13_emission_decomposition": (str(RESULTS / "optimal_controls_central_scenario.csv"), "fig13_emission", "§7.4；图16", "全部排放值为情景推演，不是实测"),
    "15_search_convergence": (str(RESULTS / "optimization_verification.csv"), "fig15_optimizer_verification", "§8.1；图17", "随机搜索对照只能证伪，不构成全局最优的数学证明"),
    "16_sensitivity_distribution": (str(RESULTS / "structural_sensitivity.csv"), "fig16_structural_sensitivity", "§8.2；图18", "扫描范围仍由显式情景网格决定"),
    "20_eta_sensitivity": (str(RESULTS / "p_exponent_reference.csv"), "fig20_p_sensitivity", "§8.2；图19", "$p$的取值区间为先验网格；需由现场阶跃试验标定"),
    "17_sensitivity_heatmap": (str(RESULTS / "field_priority_ablation_summary.csv"), "fig17_field_ablation", "§8.2；图20", "消融仍依赖同一灰箱结构与支持域"),
}

MANUSCRIPT_FIGURE = {
    "01_outlet_data_diagnosis": 2,
    "02_process_timeseries": 3,
    "22_technical_roadmap": 4,
    "04b_operating_evidence": 5,
    "14_rapping_peak_mechanism": 6,
    "03_cluster_selection": 7,
    "04_condition_map": 8,
    "07_power_prediction": 9,
    "08b_feature_importance": 10,
    "08_power_residuals": 11,
    "05b_boundary_proximity": 12,
    "19_emission_power_frontier": 13,
    "11_optimal_voltage": 14,
    "18_condition_energy_penalty": 15,
    "13_emission_decomposition": 16,
    "15_search_convergence": 17,
    "16_sensitivity_distribution": 18,
    "20_eta_sensitivity": 19,
    "17_sensitivity_heatmap": 20,
}


def _label_bars(ax, bars, fmt="{:.0f}", dy=3.0):
    for bar in bars:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width() / 2, h + dy, fmt.format(h), ha="center", va="bottom", fontsize=7)


def fig01_outlet_diagnosis(df: pd.DataFrame) -> None:
    valid = df["C_out_mgNm3"].dropna()
    fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.2), gridspec_kw={"width_ratios": [1.15, 1]})
    ax = axes[0]
    add_subpanel_label(ax, "a")
    bins = np.linspace(valid.min() - 0.03, valid.max() + 0.03, 28)
    ax.hist(valid, bins=bins, color=PALETTE[0], edgecolor="white")
    ax.axvline(50, color=PALETTE[4], linestyle="--", label="记录上限 50")
    ax.set_xlim(valid.min() - 0.05, valid.max() + 0.05)
    ax.text(
        0.03,
        0.78,
        "5与10 mg/Nm³目标均在图示范围外",
        transform=ax.transAxes,
        color=PALETTE[2],
```

```
        fontsize=8,
        bbox={"boxstyle": "round", "facecolor": "white", "edgecolor": LIGHT_GRAY},
    )
    ax.set_xlabel("出口粉尘浓度记录 (mg/Nm³)")
    ax.set_ylabel("样本数")
    ax.legend(frameon=False, loc="upper left")
    finish_axes(ax)

    ax = axes[1]
    add_subpanel_label(ax, "b")
    total = len(df)
    categories = ["精确值", "50截顶", "缺失"]
    values = [int(df["C_out_exact_flag"].sum()), int(df["C_out_cap50_flag"].sum()), int(df["C_out_missing_flag"].sum
    ↪ ())]
    bars = ax.bar(categories, values, color=[PALETTE[0], PALETTE[4], GRAY])
    for bar, value in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width()/2, value + total*0.012, f"{value}\n({100*value/total:.1f}%)", ha="
    ↪ center", fontsize=8)
    ax.set_ylabel("样本数")
    ax.set_ylim(0, max(values) * 1.18)
    note(ax, "百分比以全部10080条样本为分母", loc="upper right")
    finish_axes(ax)
    fig.tight_layout(w_pad=2.0)
    save_figure(fig, OUT, "01_outlet_data_diagnosis")


def fig02_timeseries(df: pd.DataFrame) -> None:
    d = df.iloc[:1440].copy()
    hours = np.arange(len(d)) / 60
    fields = [
        ("Temp_C", "入口温度 (°C)", PALETTE[4]),
        ("C_in_gNm3", "入口浓度 (g/Nm³)", PALETTE[0]),
        ("Q_Nm3h", "烟气流量 (Nm³/h)", PALETTE[2]),
        ("P_total_kW", "总功率 (kW)", PALETTE[3]),
    ]
    fig, axes = plt.subplots(4, 1, figsize=(8.2, 5.5), sharex=True)
    for ax, (col, label, color), tag in zip(axes, fields, ["a", "b", "c", "d"]):
        add_subpanel_label(ax, tag, x=-0.08, y=1.02)
        ax.plot(hours, d[col], color=color, linewidth=0.8)
        ax.set_ylabel(label)
        finish_axes(ax)
    axes[-1].set_xlabel("首日运行时间 (h)")
    axes[-1].set_xlim(0, 24)
    fig.tight_layout(h_pad=0.5)
    save_figure(fig, OUT, "02_process_timeseries")


def fig03_cluster_selection(cluster_eval: pd.DataFrame) -> None:
    """The two internal indices disagree; the figure has to show that openly."""
    k = cluster_eval["k"].to_numpy(int)
    sil = cluster_eval["silhouette"].to_numpy(float)
    ch = cluster_eval["calinski_harabasz"].to_numpy(float)
    k_sil, k_ch = int(k[sil.argmax()]), int(k[ch.argmax()])

    fig, axes = plt.subplots(1, 2, figsize=(8.0, 3.2))
    for ax, y, best, label, color, marker in (
            (axes[0], sil, k_sil, "轮廓系数", PALETTE[0], "o"),
            (axes[1], ch / 1000.0, k_ch, "Calinski-Harabasz 指标 (×10³)", PALETTE[2], "s")):
        ax.plot(k, y, marker + "-", color=color, markersize=5.5, linewidth=1.7)
        ax.scatter([best], [y[k == best]], s=150, facecolor="none",
                   edgecolor=PALETTE[4], linewidth=1.8, zorder=4)
        ax.annotate("最大值 $k=%d$" % best, (best, y[k == best][0]),
```

```
                    textcoords="offset points", xytext=(0, 16), ha="center",
                    fontsize=8.6, color=PALETTE[4])
        ax.set_xlabel("聚类数 $k$")
        ax.set_ylabel(label)
        ax.set_xticks(k)
        headroom(ax, 0.24)
        finish_axes(ax, grid_axis="both")
    add_subpanel_label(axes[0], "a")
    add_subpanel_label(axes[1], "b")
    note(axes[0], "本文取 $k=4$", loc="lower right")
    note(axes[1], "该指标支持 $k=5$", loc="lower right")
    fig.tight_layout(w_pad=2.2)
    save_figure(fig, OUT, "03_cluster_selection")


def fig04_condition_map(df: pd.DataFrame) -> None:
    train = df[df["split"] == "train"]
    fig, ax = plt.subplots(figsize=(6.7, 4.4))
    for cluster, group in train.groupby("condition_cluster"):
        g = group.iloc[::5]
        sizes = 8 + 28 * (g["Q_Nm3h"] - train["Q_Nm3h"].min()) / (train["Q_Nm3h"].max() - train["Q_Nm3h"].min())
        ax.scatter(g["C_in_gNm3"], g["Temp_C"], s=sizes, alpha=0.38, color=PALETTE[int(cluster)-1], label=COND_SHORT[
    ↪ int(cluster)].replace("\n", " "))
    ax.set_xlabel("入口粉尘浓度 (g/Nm³)")
    ax.set_ylabel("入口温度 (°C)")
    ax.legend(frameon=False, ncol=2)
    finish_axes(ax, grid_axis="both")
    fig.tight_layout()
    save_figure(fig, OUT, "04_condition_map")



def fig04b_operating_evidence(df: pd.DataFrame, events: pd.DataFrame,
                              strategy: pd.DataFrame) -> None:
    """The identifiable half of Question 1.

    The outlet column carries no signal, but the operator's response to the
    inlet conditions is plainly visible: two sustained excursions (panels a, b)
    and a stable correlation structure between conditions and settings (c).
    """
    fig = plt.figure(figsize=(8.4, 5.6))
    gs = fig.add_gridspec(2, 2, height_ratios=[1, 1.05], hspace=0.60, wspace=0.62)

    windows = [
        ("2024-05-02 13:30", "2024-05-02 16:20", "C_in_gNm3", "入口浓度 (g/Nm³)", "a"),
        ("2024-05-06 12:30", "2024-05-06 15:30", "Temp_C", "入口温度 (°C)", "b"),
    ]
    for col_idx, (t0, t1, col, ylabel, tag) in enumerate(windows):
        ax = fig.add_subplot(gs[0, col_idx])
        add_subpanel_label(ax, tag, x=-0.14)
        win = df[(df["timestamp"] >= t0) & (df["timestamp"] <= t1)]
        minutes = (win["timestamp"] - win["timestamp"].iloc[0]).dt.total_seconds() / 60
        ax.plot(minutes, win[col], color=PALETTE[4], linewidth=1.4, label=ylabel.split(" ")[0])
        ax.set_ylabel(ylabel, color=PALETTE[4], fontsize=9)
        ax.tick_params(axis="y", labelcolor=PALETTE[4])
        ax.set_xlabel("事件窗口内时间 (min)")
        twin = ax.twinx()
        twin.plot(minutes, win["U1_kV"], color=PALETTE[0], linewidth=1.4, label="$U_1$")
        twin.plot(minutes, win["P_total_kW"] / 30.0, color=PALETTE[2],
                  linewidth=1.2, linestyle="--", label="总功率/30")
        twin.set_ylabel("$U_1$ (kV)  总功率/30 (kW)", fontsize=8.0)
        twin.spines["top"].set_visible(False)
```

```
        finish_axes(ax, grid_axis="none")
        handles = ax.get_lines() + twin.get_lines()
        ax.legend(handles, [h.get_label() for h in handles],
                  frameon=False, fontsize=7.4, ncol=3, loc="upper center",
                  bbox_to_anchor=(0.5, 1.30))

    ax = fig.add_subplot(gs[1, :])
    add_subpanel_label(ax, "c", x=-0.055)
    cond_labels = {"r_Temp_C": "入口温度 $H$", "r_C_in_gNm3": "入口浓度 $C_{in}$", "r_Q_Nm3h": "烟气流量 $Q$"}
    full = strategy[strategy["window"] == "full_record"].set_index("control")
    matrix = full.loc[U_COLS + T_COLS, list(cond_labels)].to_numpy(float).T
    im = ax.imshow(matrix, cmap="RdBu_r", aspect="auto", vmin=-0.30, vmax=0.30)
    ax.set_xticks(range(matrix.shape[1]),
                  [f"$U_{i}$" for i in range(1, 5)] + [f"$T_{i}$" for i in range(1, 5)])
    ax.set_yticks(range(3), list(cond_labels.values()))
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            ax.text(j, i, f"{matrix[i, j]:+.2f}", ha="center", va="center", fontsize=8.6,
                    color="white" if abs(matrix[i, j]) > 0.20 else "#1F2933")
    cbar = fig.colorbar(im, ax=ax, fraction=0.028, pad=0.015)
    cbar.set_label("Pearson 相关系数", fontsize=8.4)
    ax.set_title("全记录期工况变量与控制设定的历史响应（可由附件识别）", fontsize=9.2, pad=8)
    tr = strategy[strategy["window"] == "train"].set_index("control")
    te = strategy[strategy["window"] == "retrospective_test"].set_index("control")
    ax.text(1.0, -0.32,
            "注：振打周期规律（$C_{in}\\uparrow\\Rightarrow T\\downarrow$）在四个时窗内一致；"
            "电压规律并不平稳——$r(C_{in},U_1)$ 在前5天为 %+.2f，在末日为 %+.2f。"
            % (float(tr.loc["U1_kV", "r_C_in_gNm3"]), float(te.loc["U1_kV", "r_C_in_gNm3"])),
            transform=ax.transAxes, ha="right", va="top", fontsize=7.8, color=GRAY)
    save_figure(fig, OUT, "04b_operating_evidence")


def fig22_roadmap() -> None:
    """One-page technical roadmap.

    The paper's argument splits at the identifiability audit into an
    empirically identified branch and an explicitly conditional grey-box
    branch; the two are visually distinguished so a reviewer can see at a
    glance which conclusions carry which strength of evidence.
    """
    EMP, SCN = "#1F4E79", "#B45309"       # empirical / conditional
    fig, ax = plt.subplots(figsize=(7.4, 6.6))
    ax.set_xlim(0, 10); ax.set_ylim(0, 12.2); ax.axis("off")

    def box(x, y, w, h, text, color, dashed=False, fill="#FFFFFF", fs=9.0):
        ax.add_patch(FancyBboxPatch(
            (x - w / 2, y - h / 2), w, h, boxstyle="round,pad=0.10",
            linewidth=1.3, edgecolor=color, facecolor=fill,
            linestyle="--" if dashed else "-", zorder=2))
        ax.text(x, y, text, ha="center", va="center", fontsize=fs,
                color="#1F2933", zorder=3, linespacing=1.45)

    def arrow(x1, y1, x2, y2, color=GRAY):
        ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle="-|>",
                                     mutation_scale=13, linewidth=1.1,
                                     color=color, zorder=1,
                                     shrinkA=2, shrinkB=2))

    box(5, 11.5, 5.6, 0.78, "现场原始数据  10080×14", GRAY, fill="#F3F4F6")
    arrow(5, 11.11, 5, 10.62)
    box(5, 10.2, 4.6, 0.80, "§2  可识别性审计", "#111827", fill="#E5E7EB")

```

```
    arrow(4.0, 9.80, 2.6, 9.25); arrow(6.0, 9.80, 7.4, 9.25)
    box(2.6, 8.55, 4.0, 1.30,
        "实证数据层（可复算）\n§4.1  $P=a_0+\\sum b_iU_i^2+\\sum c_i/T_i$\n"
        "§4.2—4.3  历史操作规律与异常工况", EMP)
    box(7.4, 8.55, 4.0, 1.30,
        "条件式灰箱层（推演）\n§4.4—4.5  $C_{peak}(U,T)=C_{base}+B$\n"
        "Deutsch–Anderson  锚点 50 mg/Nm³", SCN, dashed=True)

    arrow(2.6, 7.90, 4.2, 7.35); arrow(7.4, 7.90, 5.8, 7.35)
    box(5, 6.95, 5.2, 0.80, "§5.1、§5.3  典型工况划分  +  历史支持域", "#111827")

    arrow(5, 6.55, 5, 6.05)
    box(5, 5.65, 5.4, 0.80, "问题二  §5.5  10 mg/Nm³ 最低功率控制参数", EMP)
    arrow(5, 5.25, 5, 4.75)
    box(5, 4.35, 5.4, 0.80, "问题三  §5.4、§6  电场调节优先级判据", EMP)
    arrow(5, 3.95, 5, 3.45)
    box(5, 3.05, 5.4, 0.80, "问题四  §7  10→5 mg/Nm³ 功率与容量代价", SCN, dashed=True)

    arrow(5, 2.65, 5, 2.15)
    box(5, 1.55, 6.6, 1.10,
        "§5.6 外推审计  ·  §8.2 敏感性分析\n§9.3 现场阶跃标定  ·  独立盲测后闭环",
        GRAY, fill="#F3F4F6", fs=8.6)

    handles = [
        plt.Line2D([], [], color=EMP, linewidth=1.6, label="实证辨识：可由附件复算"),
        plt.Line2D([], [], color=SCN, linewidth=1.6, linestyle="--",
                   label="条件式推演：须现场标定后方可采信"),
    ]
    ax.legend(handles=handles, loc="lower center", bbox_to_anchor=(0.5, -0.03),
              ncol=2, frameon=False, fontsize=8.4)
    fig.tight_layout()
    save_figure(fig, OUT, "22_technical_roadmap")


def fig05b_boundary_proximity(audit: pd.DataFrame) -> None:
    """How far outside historical joint operating experience each solution sits."""
    ordered = audit.sort_values(["condition_cluster", "limit_mgNm3"],
                                ascending=[True, False]).reset_index(drop=True)
    labels = [f"工况{int(c)}\n{int(l)} mg"
              for c, l in zip(ordered["condition_cluster"], ordered["limit_mgNm3"])]
    colors = [PALETTE[int(c) - 1] for c in ordered["condition_cluster"]]

    fig, axes = plt.subplots(1, 2, figsize=(8.4, 3.6), gridspec_kw={"width_ratios": [1.15, 1]})
    ax = axes[0]
    add_subpanel_label(ax, "a")
    x = np.arange(len(ordered))
    ax.vlines(x, 1.0, ordered["ratio_to_q975"], color=colors, linewidth=2.2, alpha=0.8)
    ax.scatter(x, ordered["ratio_to_q975"], color=colors, s=45,
               edgecolor="black", linewidth=0.5, zorder=3)
    ax.axhline(1.0, color=PALETTE[4], linestyle="--", linewidth=1.2, label="历史经验97.5%外壳")
    ax.set_xticks(x, labels, fontsize=7.6)
    ax.set_ylabel(r"$d_M^2/q_{0.975}$（倍）")
    ax.legend(frameon=False, loc="upper left", fontsize=8)
    headroom(ax, 0.16)
    finish_axes(ax)

    ax = axes[1]
    add_subpanel_label(ax, "b")
    bars = ax.bar(x, ordered["max_voltage_over_historical_max_pct"], color=colors, width=0.62)
    _label_bars(ax, bars, fmt="{:+.1f}%", dy=0.12)
    ax.axhline(0, color=PALETTE[4], linestyle="--", linewidth=1.2)
    ax.set_xticks(x, labels, fontsize=7.6)
```

```
    ax.set_ylabel("最高电压超出历史最大值 (%)")
    note(ax, "0%以下＝仍在历史电压包络内", loc="upper left")
    headroom(ax, 0.30)
    finish_axes(ax)
    fig.tight_layout(w_pad=1.9)
    save_figure(fig, OUT, "05b_boundary_proximity")


def fit_power_model(df: pd.DataFrame) -> PhysicalPowerModel:
    train = df[df["split"] == "train"]
    return PhysicalPowerModel().fit(train, train["P_total_kW"].to_numpy(float))


def fit_linear_period_model(df: pd.DataFrame) -> RidgePowerModel:
    """The misspecified reference: rapping period entered as a linear term."""
    train = df[df["split"] == "train"]
    metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
    alpha = float(metrics["ridge_selected_alphas"]["quadratic_ridge_T"])
    model = RidgePowerModel(alpha=alpha)
    model.fit(train, train["P_total_kW"].to_numpy(float))
    return model


def fig07_power_prediction(df: pd.DataFrame, model: RidgePowerModel, baselines: pd.DataFrame) -> None:
    test = df[df["split"] == "test"]
    actual = test["P_total_kW"].to_numpy(float)
    pred = model.predict(test)
    fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.5))
    ax = axes[0]
    add_subpanel_label(ax, "a")
    ax.scatter(actual, pred, s=10, alpha=0.42, color=PALETTE[0], edgecolors="none")
    lo, hi = min(actual.min(), pred.min()), max(actual.max(), pred.max())
    ax.plot([lo, hi], [lo, hi], "--", color=PALETTE[4], label="理想预测")
    metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
    ax.text(0.04, 0.94,
            "$R^2=%.3f$\nRMSE=%.2f kW\n偏差=%+.2f kW" % (float(metrics["retrospective_test_r2"]),
                                                        float(metrics["retrospective_test_rmse_kW"]),
                                                        float(metrics["retrospective_test_bias_kW"])),
            transform=ax.transAxes, va="top",
            bbox={"boxstyle":"round", "facecolor":"white", "edgecolor":LIGHT_GRAY})
    ax.set_xlabel("实际总功率 (kW)")
    ax.set_ylabel("预测总功率 (kW)")
    ax.legend(frameon=False, loc="lower right")
    finish_axes(ax, grid_axis="both")

    ax = axes[1]
    add_subpanel_label(ax, "b")
    labels = ["均值\n基线", "线性$U$\n线性$T$", "二次$U$\n线性$T$", "本文\n$U^2+1/T$"]
    model_order = ["mean_predictor", "linear_ridge_T", "quadratic_ridge_T", "physical_invT"]
    x = np.arange(len(model_order))
    val = baselines[baselines["split"] == "validation"].set_index("model").loc[model_order, "rmse_kW"]
    tst = baselines[baselines["split"] == "retrospective_test"].set_index("model").loc[model_order, "rmse_kW"]
    ax.bar(x-0.18, val, 0.36, color=PALETTE[5], label="验证集")
    ax.bar(x+0.18, tst, 0.36, color=PALETTE[0], label="回顾性测试")
    ax.set_xticks(x, labels, fontsize=7.6)
    ax.set_ylabel("RMSE (kW，对数刻度)")
    ax.set_yscale("log")
    ax.legend(frameon=False, loc="upper right")
    finish_axes(ax)
    fig.tight_layout()
    save_figure(fig, OUT, "07_power_prediction")

```

```python

def fig08_residuals(df: pd.DataFrame, model: RidgePowerModel) -> None:
    test = df[df["split"] == "test"]
    actual = test["P_total_kW"].to_numpy(float)
    pred = model.predict(test)
    res = pred - actual
    fig, axes = plt.subplots(1, 2, figsize=(7.7, 3.1))
    add_subpanel_label(axes[0], "a")
    axes[0].scatter(pred, res, s=9, alpha=0.38, color=PALETTE[0], edgecolors="none")
    axes[0].axhline(0, color=PALETTE[4], linestyle="--")
    axes[0].set_xlabel("预测总功率 (kW)")
    axes[0].set_ylabel("残差: 预测−实际 (kW)")
    finish_axes(axes[0], grid_axis="both")
    add_subpanel_label(axes[1], "b")
    axes[1].hist(res, bins=28, color=PALETTE[2], edgecolor="white")
    metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
    guard = float(metrics["validation_abs_error_q90_kW"])
    axes[1].axvline(res.mean(), color=PALETTE[4], linestyle="--", label=f"测试均值 {res.mean():.2f} kW")
    axes[1].axvline(guard, color=PALETTE[0], linestyle=":", label=f"验证|误差|P90 {guard:.2f} kW")
    axes[1].set_xlabel("残差 (kW)")
    axes[1].set_ylabel("样本数")
    axes[1].legend(frameon=False)
    finish_axes(axes[1])
    fig.tight_layout(w_pad=2.0)
    save_figure(fig, OUT, "08_power_residuals")


def fig08b_specification(df: pd.DataFrame, model: PhysicalPowerModel,
                         linear_model: RidgePowerModel, baselines: pd.DataFrame) -> None:
    """Why the rapping period must enter as 1/T rather than T.

    Panel (a) shows the residual of a voltage-only fit against the period:
    the curvature is the signature of a 1/T term.  Panel (b) contrasts the
    retrospective-test bias of both specifications.
    """
    train = df[df["split"] == "train"]
    test = df[df["split"] == "test"]
    fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.4), gridspec_kw={"width_ratios": [1.15, 1]})

    ax = axes[0]
    add_subpanel_label(ax, "a")
    u = train[U_COLS].to_numpy(float)
    resid = train["P_total_kW"].to_numpy(float) - model.field_power(u) - model.intercept
    t1 = train["T1_s"].to_numpy(float)
    order = np.argsort(t1)
    ax.scatter(t1[order][::7], resid[order][::7], s=7, alpha=0.22,
               color=PALETTE[5], edgecolors="none", label="扣除电场功率后的残差")
    grid = np.linspace(t1.min(), t1.max(), 120)
    rap_other = float(np.sum(model.c[1:] / train[T_COLS[1:]].median().to_numpy(float)))
    ax.plot(grid, model.c[0] / grid + rap_other, color=PALETTE[4], linewidth=2.0,
            label=r"拟合 $c_1/T_1+\mathrm{const}$")
    slope = np.polyfit(t1, resid, 1)
    ax.plot(grid, np.polyval(slope, grid), "--", color=PALETTE[0], linewidth=1.6,
            label=r"线性 $T_1$ 设定")
    ax.set_xlabel("第1电场振打周期 $T_1$ (s)")
    ax.set_ylabel("剩余功率 (kW)")
    ax.legend(frameon=False, fontsize=8, loc="upper right")
    finish_axes(ax, grid_axis="both")

    ax = axes[1]
    add_subpanel_label(ax, "b")
    names = {"quadratic_ridge_T": "线性 $T$ 设定", "physical_invT": "本文 $1/T$ 设定"}
```

```
    tst = baselines[baselines["split"] == "retrospective_test"].set_index("model")
    x = np.arange(2)
    bias = [float(tst.loc[k, "bias_kW"]) for k in names]
    rmse = [float(tst.loc[k, "rmse_kW"]) for k in names]
    ax.bar(x - 0.18, bias, 0.36, color=PALETTE[4], label="测试平均偏差")
    ax.bar(x + 0.18, rmse, 0.36, color=PALETTE[0], label="测试 RMSE")
    for xi, (b, r) in enumerate(zip(bias, rmse)):
        ax.text(xi - 0.18, b + (0.6 if b >= 0 else -1.6), f"{b:+.2f}", ha="center", fontsize=8)
        ax.text(xi + 0.18, r + 0.6, f"{r:.2f}", ha="center", fontsize=8)
    ax.axhline(0, color=GRAY, linewidth=0.8)
    ax.set_xticks(x, list(names.values()))
    ax.set_ylabel("回顾性测试 (kW)")
    ax.legend(frameon=False, fontsize=8)
    headroom(ax, 0.22)
    finish_axes(ax)
    fig.tight_layout(w_pad=2.0)
    save_figure(fig, OUT, "08b_feature_importance")


def grouped_controls(optimum: pd.DataFrame, cols: list[str], stem: str, ylabel: str) -> None:
    fig, axes = plt.subplots(2, 2, figsize=(8.2, 5.8), sharey=True)
    for ax, cluster, tag in zip(axes.flat, range(1,5), ["a", "b", "c", "d"]):
        add_subpanel_label(ax, tag)
        sub = optimum[optimum["condition_cluster"] == cluster].set_index("limit_mgNm3")
        x = np.arange(4)
        ax.bar(x-0.18, sub.loc[10.0, cols], 0.36, color=PALETTE[5], label="10 mg/Nm³")
        ax.bar(x+0.18, sub.loc[5.0, cols], 0.36, color=PALETTE[4], label="5 mg/Nm³")
        ax.set_xticks(x, [c.split("_")[0] for c in cols])
        ax.set_title(COND_SHORT[cluster].replace("\n", " "), fontsize=9)
        finish_axes(ax)
    axes[0,0].set_ylabel(ylabel); axes[1,0].set_ylabel(ylabel)
    handles, labels = axes[0,0].get_legend_handles_labels()
    fig.tight_layout(h_pad=2.0, rect=(0, 0.06, 1, 1))
    figure_legend(fig, handles, labels, ncol=2, bottom=0.0)
    save_figure(fig, OUT, stem)



def fig13_emission(optimum: pd.DataFrame) -> None:
    x = np.arange(8)
    ordered = optimum.sort_values(["limit_mgNm3", "condition_cluster"], ascending=[False, True])
    base = ordered["scenario_base_emission_mgNm3"].to_numpy()
    peak = ordered["scenario_peak_excess_mgNm3"].to_numpy()
    colors = [PALETTE[0]]*4 + [PALETTE[4]]*4
    fig, ax = plt.subplots(figsize=(8.0, 4.0))
    ax.bar(x, base, color=colors, alpha=0.82, label="连续排放基值")
    ax.bar(x, peak, bottom=base, color=colors, alpha=0.35, edgecolor=colors, label="振打峰值增量")
    ax.hlines(10, -0.5, 3.5, colors=GRAY, linestyles="--", linewidth=1.0)
    ax.hlines(5, 3.5, 7.5, colors=GRAY, linestyles="--", linewidth=1.0)
    ax.text(3.45, 10.15, "10 mg/Nm³", ha="right", fontsize=8, color=GRAY)
    ax.text(7.45, 5.15, "5 mg/Nm³", ha="right", fontsize=8, color=GRAY)
    ax.set_xticks(x, [f"工况{i}" for i in range(1,5)]*2)
    ax.set_ylabel("峰值总排放情景值 (mg/Nm³)")
    ax.legend(frameon=False, ncol=2)
    finish_axes(ax)
    fig.tight_layout()
    save_figure(fig, OUT, "13_emission_decomposition")


def fig14_peak(profiles: pd.DataFrame) -> None:
    """The rapping trade-off, derived rather than assumed.

```

```
    Cake mass per rap grows like L*T, so the per-event peak grows linearly with
    the period while the contribution to the hourly mean, L*T*(1/T) = L, does
    not depend on the period at all.  Panel (b) prices the same lever in power.
    """
    trade = pd.read_csv(RESULTS / "rapping_power_tradeoff.csv")
    ratio = np.linspace(0.75, 1.35, 200)
    fig, axes = plt.subplots(1, 2, figsize=(8.3, 3.5), gridspec_kw={"width_ratios": [1.1, 1]})

    ax = axes[0]
    add_subpanel_label(ax, "a")
    for cluster in (1, 3, 4):
        pr = profiles[profiles["condition_cluster"] == cluster].iloc[0]
        peak = 1.2 * float(pr["load_ratio_to_train_median"]) * ratio
        ax.plot(100 * (ratio - 1), peak, color=PALETTE[cluster - 1],
                marker=["o", "s", "^", "D"][cluster - 1], markevery=40,
                label=COND_SHORT[cluster].replace("\n", " "))
    ax.axhline(0, color=GRAY, linewidth=0.8)
    ax.axvline(0, color=GRAY, linewidth=0.8)
    ax.plot(100 * (ratio - 1), np.full_like(ratio, 1.2 * float(
        profiles[profiles["condition_cluster"] == 3].iloc[0]["load_ratio_to_train_median"])),
        ":", color="#111827", linewidth=1.5, label="对小时均值的贡献（工况3，与$T$无关）")
    ax.set_xlabel("振打周期相对参考周期变化 (%)")
    ax.set_ylabel("单次再飞扬峰值代理 (mg/Nm³)")
    ax.legend(frameon=False, fontsize=7.8, loc="upper left")
    headroom(ax, 0.18)
    finish_axes(ax, grid_axis="both")

    ax = axes[1]
    add_subpanel_label(ax, "b")
    x = np.arange(len(trade))
    bars = ax.bar(x, trade["rapping_power_kW"], color=[PALETTE[5], PALETTE[2], PALETTE[4]], width=0.6)
    _label_bars(ax, bars, fmt="{:.0f}", dy=6)
    for xi, row in trade.iterrows():
        if row["power_recovered_kW"] > 0:
            ax.text(xi, row["rapping_power_kW"] / 2,
                    "省%.0f kW\n积灰+%.0f%%" % (row["power_recovered_kW"],
                                                row["cake_mass_per_rap_change_pct"]),
                    ha="center", va="center", fontsize=7.6, color="white")
    ax.set_xticks(x, ["历史\n中位周期", "延长至\n历史P95", "延长至\n历史最大"], fontsize=8)
    ax.set_ylabel("振打环节功率 (kW)")
    note(ax, "中位工况下振打占总功率 %.1f%%" % float(trade.iloc[0]["rapping_share_pct"]),
         loc="upper right")
    headroom(ax, 0.24)
    finish_axes(ax)
    fig.tight_layout(w_pad=2.0)
    save_figure(fig, OUT, "14_rapping_peak_mechanism")


def fig15_optimizer_verification(verification: pd.DataFrame) -> None:
    """Multi-start SLSQP against a 4x10^5-point random search of the same set."""
    ordered = verification.sort_values(["condition_cluster", "limit_mgNm3"],
                                       ascending=[True, False]).reset_index(drop=True)
    labels = [f"工况{int(c)}\n{int(l)} mg"
              for c, l in zip(ordered["condition_cluster"], ordered["limit_mgNm3"])]
    x = np.arange(len(ordered))
    slsqp = ordered["slsqp_power_kW"].to_numpy(float)
    rnd = ordered["random_search_power_kW"].to_numpy(float)
    gaps = ordered["random_search_gap_pct"].to_numpy(float)

    fig, axes = plt.subplots(1, 2, figsize=(8.4, 3.3), gridspec_kw={"width_ratios": [1.2, 1]})

    ax = axes[0]
```

```
    add_subpanel_label(ax, "a")
    lo = float(np.nanmin(slsqp)) * 0.94
    hi = float(np.nanmax([np.nanmax(slsqp), np.nanmax(rnd)])) * 1.04
    ax.set_ylim(lo, hi)
    ax.bar(x - 0.18, slsqp, 0.36, color=PALETTE[0], label="多起点SLSQP")
    ax.bar(x + 0.18, np.where(np.isfinite(rnd), rnd, lo), 0.36,
           color=LIGHT_GRAY, label="随机搜索最优")
    for xi, value in enumerate(rnd):
        if not np.isfinite(value):
            ax.annotate("随机搜索\n无可行样本", xy=(xi + 0.18, lo),
                        xytext=(xi + 0.55, lo + 0.30 * (hi - lo)),
                        fontsize=7.2, color=PALETTE[4], ha="left",
                        arrowprops={"arrowstyle": "->", "color": PALETTE[4], "linewidth": 0.9})
    ax.set_xticks(x, labels, fontsize=7.4)
    ax.set_ylabel("最优总功率 (kW)")
    ax.legend(frameon=False, fontsize=8, loc="upper left", ncol=2)
    finish_axes(ax)

    ax = axes[1]
    add_subpanel_label(ax, "b")
    finite = np.isfinite(gaps)
    ax.bar(x[finite], gaps[finite], color=PALETTE[2], width=0.6)
    for xi, value in zip(x[finite], gaps[finite]):
        ax.text(xi, value + 0.12, f"{value:.2f}", ha="center", va="bottom", fontsize=7.4)
    ax.set_ylim(0, float(np.nanmax(gaps)) * 1.42)
    ax.set_xticks(x, labels, fontsize=7.4)
    ax.set_ylabel("随机搜索相对SLSQP的劣势 (%)")
    note(ax, "全部为正：随机搜索未找到更优解", loc="upper right")
    finish_axes(ax)
    fig.tight_layout(w_pad=2.2)
    save_figure(fig, OUT, "15_search_convergence")


def fig16_structural_sensitivity(structural: pd.DataFrame) -> None:
    valid = structural[structural["all_conditions_feasible"] == True]
    fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.5), sharey=True)
    ax = axes[0]
    add_subpanel_label(ax, "a")
    exps = sorted(valid["peak_exponent"].unique())
    data = [valid[valid["peak_exponent"] == x]["weighted_power_increase_pct"] for x in exps]
    boxes = ax.boxplot(data, tick_labels=[f"{x:.2f}" for x in exps],
                       patch_artist=True, showfliers=False)
    for i, box in enumerate(boxes["boxes"]):
        box.set_facecolor(PALETTE[i % len(PALETTE)]); box.set_alpha(0.72)
    ax.set_xlabel(r"振打峰值指数 $\gamma$（本文推导值 1.00）")
    ax.set_ylabel("5相对10 mg/Nm³加权功率增幅 (%)")
    finish_axes(ax)

    ax = axes[1]
    add_subpanel_label(ax, "b")
    phases = sorted(valid["phase_scale"].unique())
    for profile, style, label, color in zip(
            ["equal", "weak_front", "central_front"], ["o-", "s--", "^:"],
            ["等权", "弱前级", "中心前级"], [PALETTE[0], PALETTE[2], PALETTE[4]]):
        medians = [valid[(valid["phase_scale"] == ph) & (valid["alpha_profile"] == profile)]
                   ["weighted_power_increase_pct"].median() for ph in phases]
        ax.plot(phases, medians, style, color=color, label=label)
    ax.set_xlabel(r"振打相位叠加尺度 $\rho$")
    ax.legend(frameon=False)
    finish_axes(ax, grid_axis="both")
    fig.tight_layout(w_pad=2.0)
    save_figure(fig, OUT, "16_sensitivity_distribution")
```

```python


def fig17_field_ablation(ablation: pd.DataFrame) -> None:
    profiles = ["equal", "weak_front", "central_front"]
    labels = ["等权", "弱前级", "中心前级"]
    cols = [f"weighted_delta_U{i}_kV" for i in range(1,5)]
    d = ablation.set_index("alpha_profile").loc[profiles, cols]
    fig, axes = plt.subplots(1, 2, figsize=(7.8, 3.3), gridspec_kw={"width_ratios":[1.45,1]})
    ax = axes[0]
    add_subpanel_label(ax, "a")
    im = ax.imshow(d.to_numpy(), cmap="YlGnBu", aspect="auto", vmin=0)
    ax.set_xticks(range(4), [f"U{i}" for i in range(1,5)])
    ax.set_yticks(range(3), labels)
    ax.set_xlabel("电场")
    ax.set_ylabel("去除贡献权重设定")
    for i in range(3):
        for j in range(4):
            ax.text(j, i, f"{d.iloc[i,j]:.2f}", ha="center", va="center", color="white" if d.iloc[i,j]>5 else "black"
    ↪ , fontsize=8)
    cbar = fig.colorbar(im, ax=ax, fraction=0.05, pad=0.04)
    cbar.set_label("10→5 mg时加权电压增量 (kV)")

    ax = axes[1]
    add_subpanel_label(ax, "b")
    shares = 100*ablation.set_index("alpha_profile").loc[profiles,"front_two_share_of_positive_adjustment"]
    bars = ax.bar(labels, shares, color=[PALETTE[0],PALETTE[2],PALETTE[4]])
    _label_bars(ax, bars, fmt="{:.1f}%", dy=0.7)
    ax.set_ylabel("前两电场占正向电压增量 (%)")
    ax.set_ylim(0,100)
    finish_axes(ax)
    fig.tight_layout(w_pad=2.0)
    save_figure(fig, OUT, "17_sensitivity_heatmap")


def fig18_penalty(q4: pd.DataFrame) -> None:
    """Does the closed-form estimate reproduce the full eight-variable optimum?

    Four near-equal bars of ~6.7% carry little information; what is worth
    showing is that the analytical approximation and the numerical optimum
    agree to within 1%.
    """
    q4 = q4.sort_values("condition_cluster")
    ana = q4["analytic_delta_power_kW"].to_numpy(float)
    num = q4["numeric_delta_power_kW"].to_numpy(float)
    rel = 100.0 * (ana - num) / num

    fig, axes = plt.subplots(1, 2, figsize=(8.3, 3.5), gridspec_kw={"width_ratios": [1, 1.05]})

    ax = axes[0]
    add_subpanel_label(ax, "a")
    lo, hi = min(ana.min(), num.min()) - 8, max(ana.max(), num.max()) + 8
    ax.plot([lo, hi], [lo, hi], "--", color=GRAY, linewidth=1.2, label="$y=x$")
    for idx in range(len(q4)):
        ax.scatter(num[idx], ana[idx], s=70, color=PALETTE[idx], zorder=3,
                   edgecolor="white", linewidth=0.8,
                   label=COND_SHORT[int(q4["condition_cluster"].iloc[idx])].replace("\n", " "))
    ax.set_xlim(lo, hi); ax.set_ylim(lo, hi)
    ax.set_xlabel("数值最优解的 $\\Delta P$ (kW)")
    ax.set_ylabel("闭式估计的 $\\Delta P$ (kW)")
    ax.legend(frameon=False, fontsize=7.4, loc="upper left")
    finish_axes(ax, grid_axis="both")

```

```
    ax = axes[1]
    add_subpanel_label(ax, "b")
    x = np.arange(len(q4))
    bars = ax.bar(x, rel, color=[PALETTE[i] for i in range(len(q4))], width=0.58)
    for bar, value in zip(bars, rel):
        ax.text(bar.get_x() + bar.get_width() / 2,
                value + (0.04 if value >= 0 else -0.09),
                f"{value:+.2f}%", ha="center",
                va="bottom" if value >= 0 else "top", fontsize=8)
    ax.axhline(0, color=GRAY, linewidth=0.8)
    ax.axhspan(-1, 1, color=NEUTRAL_FILL, alpha=0.45, zorder=0)
    ax.text(len(q4) - 0.5, 0.86, "±1% 带", ha="right", va="top", fontsize=7.8, color=GRAY)
    ax.set_xticks(x, [COND_SHORT[int(c)] for c in q4["condition_cluster"]], fontsize=7.4)
    ax.set_ylabel("闭式估计相对数值解的偏差 (%)")
    ax.set_ylim(-1.2, 1.2)
    finish_axes(ax)
    fig.tight_layout(w_pad=2.2)
    save_figure(fig, OUT, "18_condition_energy_penalty")


def _relpath(path: str) -> str:
    """交付物中一律使用相对路径，避免写入本机绝对路径。"""
    try:
        return Path(path).resolve().relative_to(ROOT).as_posix()
    except ValueError:
        return Path(path).name


TIKZ_ROWS = [{
    "figure_id": "01", "manuscript_figure": 1, "stem": "00_esp_schematic",
    "caption": "四电场电除尘器结构与变量定义",
    "png": "figures_paper/00_esp_schematic.pdf", "pdf": "figures_paper/00_esp_schematic.pdf",
    "source_data": "figures_paper/00_esp_schematic.tex",
    "transformation": "按赛题给定的设备结构示意以TikZ矢量重绘，xelatex独立编译",
    "supported_manuscript_claims": "§1.1；图1",
    "limitations": "结构示意图，不含实测数据；比例非工程真实尺寸",
}]


def fig19_frontier(frontier: pd.DataFrame, weighted: pd.DataFrame) -> None:
    """排放—功率权衡前沿：说明放宽限值换来的功率下降不是等排放下的节能。"""
    fig, ax = plt.subplots(figsize=(7.4, 4.0))
    for idx, (cluster, grp) in enumerate(frontier.groupby("condition_cluster")):
        grp = grp.sort_values("limit_mgNm3")
        ax.plot(grp["limit_mgNm3"], grp["predicted_power_kW"], "o-",
                color=PALETTE[idx], markersize=4.5, linewidth=1.5,
                label=COND_SHORT[int(cluster)].replace("\n", " "))
    w = weighted.sort_values("limit_mgNm3")
    ax.plot(w["limit_mgNm3"], w["weighted_power_kW"], "s--", color="#111827",
            markersize=5, linewidth=2.0, label="工况加权")
    hist = float(frontier["historical_power_kW"].iloc[0]) if "historical_power_kW" in frontier else np.nan
    p10 = float(w[w["limit_mgNm3"] == 10.0]["weighted_power_kW"].iloc[0])
    p5 = float(w[w["limit_mgNm3"] == 5.0]["weighted_power_kW"].iloc[0])
    if np.isfinite(hist):
        ax.axhline(hist, color=GRAY, linestyle=":", linewidth=1.3)
        ax.text(9.9, hist - 14, f"历史实际加权功率 {hist:.0f} kW（排放约50 mg/Nm³）",
                fontsize=8, color=GRAY, ha="right")
    ax.annotate("", xy=(10.0, p10), xytext=(5.0, p5),
                arrowprops={"arrowstyle": "<->", "color": PALETTE[4], "linewidth": 1.3})
    ax.text(7.5, (p5 + p10) / 2 + 10,
            f"限值 10→5 mg/Nm³\n加权功率增 {p5 - p10:.0f} kW（{100*(p5-p10)/p10:.1f}%）",
            fontsize=8.2, color=PALETTE[4], ha="center")
```

```
    ax.set_xlabel("情景排放限值 (mg/Nm³)")
    ax.set_ylabel("支持域内最佳采样预测功率 (kW)")
    ax.set_xticks(sorted(frontier["limit_mgNm3"].unique()))
    ax.legend(frameon=False, ncol=2, fontsize=8.4, loc="upper right")
    headroom(ax, 0.16)
    finish_axes(ax, grid_axis="both")
    fig.tight_layout()
    save_figure(fig, OUT, "19_emission_power_frontier")


def fig20_p_sensitivity(p_ref: pd.DataFrame, prior: pd.DataFrame) -> None:
    """The increase is set by one calibratable scalar: the exponent p in K ~ U^p."""
    ref = p_ref.sort_values("p_exponent")
    fig, ax = plt.subplots(figsize=(6.9, 3.9))
    ax.plot(ref["p_exponent"], ref["analytic_increase_pct"], "-", color="#111827",
            linewidth=1.8, zorder=2, label=r"解析式 $P_f[(1+\Delta K/K)^{2/p}-1]/P_{10}$")
    feas = ref[ref["feasible_at_5pct_headroom"] == True]
    infeas = ref[ref["feasible_at_5pct_headroom"] != True]
    ax.scatter(feas["p_exponent"], feas["analytic_increase_pct"], s=62, color=PALETTE[0],
               edgecolor="white", linewidth=0.8, zorder=4, label="5%电压裕度内可达")
    ax.scatter(infeas["p_exponent"], infeas["analytic_increase_pct"], s=62,
               facecolor="white", edgecolor=PALETTE[4], linewidth=1.6, zorder=4,
               label="需更大电压裕度（不可达）")
    num = ref.dropna(subset=["numeric_increase_pct"])
    ax.scatter(num["p_exponent"], num["numeric_increase_pct"], marker="x", s=52,
               color=PALETTE[2], zorder=5, label="数值重解校验")
    for _, row in ref.iterrows():
        ax.annotate(f"{row['analytic_increase_pct']:.1f}%",
                    (row["p_exponent"], row["analytic_increase_pct"]),
                    textcoords="offset points", xytext=(0, 10), ha="center",
                    fontsize=8.2, color=GRAY)
    ax.set_xlabel(r"去除指数—电压幂次 $p$（$K\propto U^p$；$p=2$ 为Deutsch迁移速度形式）")
    ax.set_ylabel("10→5 mg/Nm³ 加权功率增幅 (%)")
    ax.legend(frameon=False, fontsize=8, loc="upper right")
    headroom(ax, 0.24)
    finish_axes(ax, grid_axis="both")
    fig.tight_layout()
    save_figure(fig, OUT, "20_eta_sensitivity")


def write_manifest() -> None:
    script_hash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
    model_hash = hashlib.sha256((SRC_DIR / "scenario_model.py").read_bytes()).hexdigest()
    rows = []
    for stem, caption in CAPTIONS.items():
        source_data, function_name, claims, limitations = TRACE_META[stem]
        source_data = _relpath(source_data)
        rows.append({"figure_id": stem.split("_")[0], "manuscript_figure": MANUSCRIPT_FIGURE[stem], "stem": stem, "
    ↪ caption": caption,
                     "png": f"figures_paper/{stem}.png", "pdf": f"figures_paper/{stem}.pdf",
                     "source_data": source_data,
                     "transformation": f"paper_figures.py::{function_name}; plot_sha256={script_hash}; model_sha256={
    ↪ model_hash}",
                     "supported_manuscript_claims": claims,
                     "limitations": limitations})
    rows = TIKZ_ROWS + rows
    rows.sort(key=lambda r: r["manuscript_figure"])
    pd.DataFrame(rows).to_csv(OUT / "figure_manifest.csv", index=False, encoding="utf-8-sig")
    lines = ["# 论文图组与图注", "", "所有图均提供 PNG 预览和 PDF 矢量版本。", ""]
    for row in sorted(rows, key=lambda x: x["manuscript_figure"]):
        lines.extend([
```

```
            f"## 正文图{row['manuscript_figure']}（文件编号{row['figure_id']}）{row['stem']}", "", row["caption"], ""
    ↪ ,
            f"- 源数据：`{row['source_data']}`",
            f"- 变换：`{row['transformation']}`",
            f"- 支持的正文论断：{row['supported_manuscript_claims']}",
            f"- 局限：{row['limitations']}",
            f"- PNG：`{row['png']}`", f"- PDF：`{row['pdf']}`", ""])
    (OUT / "图注清单.md").write_text("\n".join(lines), encoding="utf-8")


def main() -> None:
    set_paper_style()
    df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
    profiles = pd.read_csv(RESULTS / "condition_profiles.csv")
    cluster_eval = pd.read_csv(RESULTS / "cluster_selection_metrics.csv")
    optimum = pd.read_csv(RESULTS / "optimal_controls_central_scenario.csv")
    audit = pd.read_csv(RESULTS / "support_boundary_audit.csv")
    verification = pd.read_csv(RESULTS / "optimization_verification.csv")
    structural = pd.read_csv(RESULTS / "structural_sensitivity.csv")
    ablation = pd.read_csv(RESULTS / "field_priority_ablation_summary.csv")
    baselines = pd.read_csv(RESULTS / "power_model_baselines.csv")
    q4 = pd.read_csv(RESULTS / "question4_by_condition.csv")
    frontier = pd.read_csv(RESULTS / "emission_power_frontier.csv")
    frontier_w = pd.read_csv(RESULTS / "emission_power_frontier_weighted.csv")
    prior = pd.read_csv(RESULTS / "question4_sensitivity.csv")
    p_ref = pd.read_csv(RESULTS / "p_exponent_reference.csv")
    events = pd.read_csv(RESULTS / "anomaly_events.csv")
    strategy = pd.read_csv(RESULTS / "historical_strategy_correlations.csv")
    model = fit_power_model(df)
    linear_model = fit_linear_period_model(df)

    fig01_outlet_diagnosis(df)
    fig02_timeseries(df)
    fig22_roadmap()
    fig04b_operating_evidence(df, events, strategy)
    fig14_peak(profiles)
    fig03_cluster_selection(cluster_eval)
    fig04_condition_map(df)
    fig07_power_prediction(df, model, baselines)
    fig08b_specification(df, model, linear_model, baselines)
    fig08_residuals(df, model)
    fig05b_boundary_proximity(audit)
    fig19_frontier(frontier, frontier_w)
    grouped_controls(optimum, U_COLS, "11_optimal_voltage", "最优电压 (kV)")
    fig18_penalty(q4)
    fig13_emission(optimum)
    fig15_optimizer_verification(verification)
    fig16_structural_sensitivity(structural)
    fig20_p_sensitivity(p_ref, prior)
    fig17_field_ablation(ablation)
    write_manifest()
    print(f"Generated {len(CAPTIONS)} figures in {OUT}")


if __name__ == "__main__":
    main()
```

E.4 plot_utils.py

```
"""Shared publication plotting style for the cement ESP paper."""

from __future__ import annotations
```

```python

from pathlib import Path

import matplotlib.pyplot as plt
from cycler import cycler


# Okabe-Ito 色序：色觉障碍友好，灰度打印仍可区分
PALETTE = ["#0072B2", "#E69F00", "#009E73", "#CC79A7", "#D55E00", "#56B4E9"]
GRAY = "#5B6472"
LIGHT_GRAY = "#D1D5DB"
NEUTRAL_FILL = "#DFE3E8"


def set_paper_style() -> None:
    plt.rcParams.update(
        {
            "font.family": "sans-serif",
            # Linux 优先 Noto，macOS 回落到系统中文字体
            "font.sans-serif": [
                "Noto Sans CJK SC",
                "Source Han Sans SC",
                "PingFang SC",
                "Arial Unicode MS",
                "WenQuanYi Zen Hei",
                "DejaVu Sans",
            ],
            "axes.unicode_minus": False,
            "font.size": 10.0,
            "axes.labelsize": 10.0,
            "axes.titlesize": 10.0,
            "xtick.labelsize": 9.0,
            "ytick.labelsize": 9.0,
            "legend.fontsize": 9.0,
            "legend.frameon": False,
            "legend.handlelength": 1.8,
            "legend.columnspacing": 1.4,
            "legend.borderaxespad": 0.4,
            "axes.linewidth": 0.9,
            "axes.edgecolor": "#2F3640",
            "axes.labelcolor": "#1F2933",
            "axes.prop_cycle": cycler(color=PALETTE),
            "lines.linewidth": 1.7,
            "lines.markersize": 5.0,
            "xtick.direction": "out",
            "ytick.direction": "out",
            "xtick.major.width": 0.9,
            "ytick.major.width": 0.9,
            "xtick.color": "#2F3640",
            "ytick.color": "#2F3640",
            "grid.color": "#E5E7EB",
            "grid.linewidth": 0.6,
            "grid.alpha": 0.9,
            "figure.dpi": 160,
            "savefig.dpi": 400,
            "savefig.bbox": "tight",
            "savefig.pad_inches": 0.03,
            "pdf.fonttype": 42,
            "ps.fonttype": 42,
        }
    )

```

```

def finish_axes(ax, grid_axis: str = "y") -> None:
    if grid_axis != "none":
        ax.grid(True, axis=grid_axis)
    ax.set_axisbelow(True)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)


def add_subpanel_label(ax, label: str, x: float = -0.12, y: float = 1.05) -> None:
    """Add a compact (a), (b), ... identifier to a multi-panel figure."""
    ax.text(
        x,
        y,
        f"({label})",
        transform=ax.transAxes,
        fontsize=10.5,
        fontweight="bold",
        va="bottom",
        ha="right",
    )


def headroom(ax, frac: float = 0.18) -> None:
    """在数据上方留出空白，避免图例或注释压住柱顶与折线。"""
    lo, hi = ax.get_ylim()
    ax.set_ylim(lo, lo + (hi - lo) * (1.0 + frac))


def figure_legend(fig, handles, labels, ncol: int, bottom: float = -0.02) -> None:
    """把多面板共用图例统一放到画布下方，杜绝与数据重叠。"""
    fig.legend(
        handles,
        labels,
        loc="lower center",
        ncol=ncol,
        bbox_to_anchor=(0.5, bottom),
        frameon=False,
    )


def note(ax, text: str, loc: str = "upper left", color: str | None = None) -> None:
    """角落说明文字，带白底以免压住网格与数据。"""
    anchor = {
        "upper left": (0.02, 0.98, "left", "top"),
        "upper right": (0.98, 0.98, "right", "top"),
        "lower left": (0.02, 0.03, "left", "bottom"),
        "lower right": (0.98, 0.03, "right", "bottom"),
    }[loc]
    ax.text(
        anchor[0],
        anchor[1],
        text,
        transform=ax.transAxes,
        ha=anchor[2],
        va=anchor[3],
        fontsize=8.0,
        color=color or GRAY,
        bbox={
            "boxstyle": "round,pad=0.28",
            "facecolor": "white",
            "edgecolor": LIGHT_GRAY,
```

```
            "linewidth": 0.6,
            "alpha": 0.92,
        },
    )


def save_figure(fig, out_dir: Path, stem: str) -> None:
    out_dir.mkdir(parents=True, exist_ok=True)
    fig.savefig(out_dir / f"{stem}.png")
    fig.savefig(out_dir / f"{stem}.pdf")
    plt.close(fig)
```

E.5 compute_frontier.py

```
#!/usr/bin/env python3
"""Compute the scenario emission-power trade-off frontier.

The support-domain search is repeated at a range of emission limits, so the
paper can show the whole trade-off curve rather than only the 10 and
5 mg/Nm^3 end points.  Because the historical anchor is ~50 mg/Nm^3, every
point on this curve costs power relative to historical operation; the curve
also records how much voltage headroom above the historical maximum each
limit needs before it becomes reachable at all.

Usage:  python3 src/compute_frontier.py
"""

from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd

from scenario_model import (
    DATA_FILE,
    HEADROOM_5,
    T_COLS,
    condition_profiles,
    evaluate_power_models,
    scenario_t_star,
    solve_condition,
)

ROOT = Path(__file__).resolve().parents[1]
OUT_DIR = ROOT / "results"
LIMITS = [5.0, 6.0, 7.0, 8.0, 9.0, 10.0]


def main() -> None:
    df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
    df["condition_cluster"] = df["condition_cluster"].astype(int)
    profiles = condition_profiles(df)

    train = df[df["split"] == "train"].copy()
    validation = df[df["split"] == "validation"].copy()
    test = df[df["split"] == "test"].copy()
    model, metrics, _, _ = evaluate_power_models(train, validation, test)
    guard = float(metrics["validation_abs_error_q90_kW"])

    global_t_median = train[T_COLS].median().to_numpy(float)
    t_stars = {int(p["condition_cluster"]): scenario_t_star(p, global_t_median)
```

```
                for _, p in profiles.iterrows()}
    historical_weighted = float(np.average(profiles["historical_power_kW"],
                                           weights=profiles["share"]))

    rows: list[dict[str, object]] = []
    weighted_rows: list[dict[str, object]] = []
    for limit in LIMITS:
        # smallest declared headroom that makes every condition reachable
        headroom = 0.0
        while headroom <= HEADROOM_5 + 1e-9:
            trial = [solve_condition(p, model, t_stars[int(p["condition_cluster"])], limit,
                                     voltage_headroom=headroom, multistart=12)
                     for _, p in profiles.iterrows()]
            if all(r["status"] == "SCENARIO_FEASIBLE" for r in trial):
                break
            headroom = round(headroom + 0.01, 2)

        solved = [solve_condition(p, model, t_stars[int(p["condition_cluster"])], limit,
                                  voltage_headroom=headroom)
                  for _, p in profiles.iterrows()]
        for profile_row, r in zip(profiles.itertuples(), solved):
            rows.append({
                "condition_cluster": r["condition_cluster"],
                "condition_name": r["condition_name"],
                "limit_mgNm3": limit,
                "voltage_headroom_pct": 100.0 * headroom,
                "status": r["status"],
                "predicted_power_kW": r.get("predicted_power_kW", np.nan),
                "validation_error_adjusted_power_kW": r.get("predicted_power_kW", np.nan) + guard,
                "share": float(profile_row.share),
                "historical_power_kW": historical_weighted,
            })
        weights = np.array([float(p.share) for p in profiles.itertuples()])
        powers = np.array([float(r.get("predicted_power_kW", np.nan)) for r in solved])
        weighted_rows.append({
            "limit_mgNm3": limit,
            "voltage_headroom_pct": 100.0 * headroom,
            "weighted_power_kW": float(np.average(powers, weights=weights)),
            "weighted_daily_energy_MWh": float(np.average(powers, weights=weights)) * 24.0 / 1000.0,
            "historical_power_kW": historical_weighted,
        })

    pd.DataFrame(rows).to_csv(OUT_DIR / "emission_power_frontier.csv",
                              index=False, encoding="utf-8-sig")
    pd.DataFrame(weighted_rows).to_csv(OUT_DIR / "emission_power_frontier_weighted.csv",
                                       index=False, encoding="utf-8-sig")
    print(pd.DataFrame(weighted_rows).to_string(index=False))


if __name__ == "__main__":
    main()
```

E.6 integrity_check.py

```
#!/usr/bin/env python3
"""Deterministic integrity checks for manuscript numbers and artifacts."""

from __future__ import annotations

import json
import re
import sys
```

```python
from pathlib import Path

import numpy as np
import pandas as pd

SRC = Path(__file__).resolve().parent
if str(SRC) not in sys.path:
    sys.path.insert(0, str(SRC))

from scenario_model import PhysicalPowerModel, _r2, _rmse


ROOT = SRC.parent
DATA = ROOT / "data" / "full_timeseries_with_flags.csv"
RESULTS = ROOT / "results"
FIGURES = ROOT / "figures_paper"
MANUSCRIPT = ROOT / "10_修订后完整论文_终稿.md"
OUT = ROOT / "integrity"


def close(a: float, b: float, tol: float = 1e-6) -> bool:
    return bool(abs(a - b) <= tol)


def main() -> None:
    checks: list[dict[str, object]] = []

    def check(name: str, passed: bool, actual: object, expected: object) -> None:
        checks.append({"check": name, "passed": bool(passed), "actual": actual, "expected": expected})

    df = pd.read_csv(DATA)
    check("raw_row_count", len(df) == 10080, len(df), 10080)
    check("outlet_missing_count", int(df["C_out_missing_flag"].sum()) == 50, int(df["C_out_missing_flag"].sum()), 50)
    valid = df["C_out_mgNm3"].dropna()
    check("outlet_valid_count", len(valid) == 10030, len(valid), 10030)
    check("outlet_min", close(float(valid.min()), 48.74), float(valid.min()), 48.74)
    check("outlet_max", close(float(valid.max()), 50.0), float(valid.max()), 50.0)
    check("outlet_cap50_count", int((valid == 50).sum()) == 5491, int((valid == 50).sum()), 5491)
    check("outlet_le10_count", int((valid <= 10).sum()) == 0, int((valid <= 10).sum()), 0)
    check("outlet_le5_count", int((valid <= 5).sum()) == 0, int((valid <= 5).sum()), 0)

    metrics = json.loads((RESULTS / "power_model_metrics.json").read_text())
    train, val, test = (df[df["split"] == s] for s in ("train", "validation", "test"))
    model = PhysicalPowerModel().fit(train, train["P_total_kW"].to_numpy(float))
    vp, tp = model.predict(val), model.predict(test)
    recomputed = {
        "validation_r2": _r2(val["P_total_kW"].to_numpy(float), vp),
        "validation_rmse_kW": _rmse(val["P_total_kW"].to_numpy(float), vp),
        "retrospective_test_r2": _r2(test["P_total_kW"].to_numpy(float), tp),
        "retrospective_test_rmse_kW": _rmse(test["P_total_kW"].to_numpy(float), tp),
        "retrospective_test_bias_kW": float(np.mean(tp - test["P_total_kW"].to_numpy(float))),
    }
    for key, value in recomputed.items():
        check(f"power_metric_{key}", close(value, float(metrics[key]), 1e-9), value, metrics[key])
    guard = float(np.quantile(np.abs(vp-val["P_total_kW"].to_numpy(float)), 0.90))
    check("validation_error_guard_q90", close(guard, float(metrics["validation_abs_error_q90_kW"]), 1e-9), guard,
    ↪ metrics["validation_abs_error_q90_kW"])
    check(
        "reporting_guard_rule_named_consistently",
        metrics.get("reporting_guard_rule") == "point_prediction_plus_validation_absolute_error_q90"
        and "conservative_reporting_rule" not in metrics,
        {k: metrics[k] for k in metrics if "reporting" in k or "guard" in k},
```

```
        {"reporting_guard_rule": "point_prediction_plus_validation_absolute_error_q90"},
    )
    check(
        "power_model_is_physical_specification",
        metrics["model"] == "physical_voltage_square_plus_rapping_frequency",
        metrics["model"], "physical_voltage_square_plus_rapping_frequency",
    )
    # the four field coefficients must be near-identical: the fields share a design
    b = np.asarray(metrics["b_field_kW_per_kV2"], float)
    check("field_coefficients_consistent", float(b.std() / b.mean()) < 0.01,
          float(b.std() / b.mean()), "<1% relative spread")

    baselines = pd.read_csv(RESULTS / "power_model_baselines.csv")
    tst_base = baselines[baselines["split"] == "retrospective_test"].set_index("model")
    check(
        "invT_beats_linear_T_on_test",
        float(tst_base.loc["physical_invT", "rmse_kW"]) < float(tst_base.loc["quadratic_ridge_T", "rmse_kW"]),
        tst_base["rmse_kW"].to_dict(), "physical_invT < quadratic_ridge_T",
    )
    check(
        "invT_removes_test_bias",
        abs(float(tst_base.loc["physical_invT", "bias_kW"])) < 1.0
        and abs(float(tst_base.loc["quadratic_ridge_T", "bias_kW"])) > 5.0,
        {"physical_invT": float(tst_base.loc["physical_invT", "bias_kW"]),
         "quadratic_ridge_T": float(tst_base.loc["quadratic_ridge_T", "bias_kW"])},
        "|bias| < 1 kW with 1/T, > 5 kW with linear T",
    )
    check(
        "condition_variables_add_nothing",
        abs(float(tst_base.loc["physical_invT_plus_conditions", "r2"])
            - float(tst_base.loc["physical_invT", "r2"])) < 1e-3,
        {"with": float(tst_base.loc["physical_invT_plus_conditions", "r2"]),
         "without": float(tst_base.loc["physical_invT", "r2"])},
        "delta R2 < 1e-3",
    )
    rolling = pd.read_csv(RESULTS / "power_model_rolling_validation.csv")
    check("rolling_fold_count", rolling["fold"].nunique() == 3, int(rolling["fold"].nunique()), 3)

    cluster = pd.read_csv(RESULTS / "cluster_selection_metrics.csv")
    selected = cluster[cluster["selected"]].iloc[0]
    check("selected_cluster_k", int(selected["k"]) == 4, int(selected["k"]), 4)
    check("silhouette_k4", close(float(selected["silhouette"]), 0.4083689, 1e-6), float(selected["silhouette"]),
    ↪ 0.4083689)

    optimum = pd.read_csv(RESULTS / "optimal_controls_central_scenario.csv")
    check(
        "central_parameter_table_complete",
        len(optimum) == 8
        and set(map(tuple, optimum[["condition_cluster", "limit_mgNm3"]].to_numpy()))
        == {(cluster, limit) for cluster in range(1, 5) for limit in (10.0, 5.0)},
        optimum[["condition_cluster", "limit_mgNm3"]].values.tolist(),
        "4 conditions × 2 targets",
    )
    check(
        "validation_error_columns_named_consistently",
        {"validation_error_adjusted_power_kW", "validation_error_guard_kW"}.issubset(optimum.columns)
        and {"conservative_power_kW", "power_guard_kW"}.isdisjoint(optimum.columns),
        sorted(optimum.columns),
        "new validation-error names present; old conservative names absent",
    )
    check("central_all_feasible", bool((optimum["status"] == "SCENARIO_FEASIBLE").all()), optimum["status"].unique().
    ↪ tolist(), ["SCENARIO_FEASIBLE"])
```

```
    check("central_limits_respected", bool((optimum["scenario_peak_total_mgNm3"] <= optimum["limit_mgNm3"] + 1e-9).
    ↪ all()), float((optimum["scenario_peak_total_mgNm3"] - optimum["limit_mgNm3"]).max()), "<=0")
    check(
        "emission_anchor_not_rescaled",
        bool((pd.read_csv(RESULTS / "condition_profiles.csv")["C_out_recorded_median"] == 50.0).all()),
        pd.read_csv(RESULTS / "condition_profiles.csv")["C_out_recorded_median"].tolist(),
        "all 50.0 mg/Nm3 (recorded, unscaled)",
    )
    scenario = json.loads((RESULTS / "scenario_parameters.json").read_text())
    check("no_outlet_rescaling_parameter", "central_outlet_scale" not in scenario,
          sorted(scenario), "no central_outlet_scale key")

    audit = pd.read_csv(RESULTS / "support_boundary_audit.csv")
    check("support_audit_rows", len(audit) == 8, len(audit), 8)
    check(
        "targets_leave_historical_experience",
        bool((audit["ratio_to_q975"] > 1.0).all()),
        float(audit["ratio_to_q975"].min()), ">1 for all eight solutions",
    )
    check(
        "10mg_inside_voltage_envelope",
        bool((audit[audit["limit_mgNm3"] == 10.0]["max_voltage_over_historical_max_pct"] <= 1e-6).all()),
        float(audit[audit["limit_mgNm3"] == 10.0]["max_voltage_over_historical_max_pct"].max()),
        "<=0% over the historical maximum",
    )
    restricted = pd.read_csv(RESULTS / "mahalanobis_constrained_feasibility.csv")
    merged = restricted.merge(
        optimum[["condition_cluster", "limit_mgNm3", "predicted_power_kW"]],
        on=["condition_cluster", "limit_mgNm3"], suffixes=("_shell", "_free"))
    feasible_shell = merged["status_with_mahalanobis_constraint"] == "SCENARIO_FEASIBLE"
    check(
        "mahalanobis_shell_variant_mostly_feasible",
        int(feasible_shell.sum()) == 7,
        int(feasible_shell.sum()), "7 of 8 combinations feasible inside the shell",
    )
    penalty = 100.0 * (merged.loc[feasible_shell, "predicted_power_kW_shell"]
                       / merged.loc[feasible_shell, "predicted_power_kW_free"] - 1.0)
    check(
        "shell_variant_costs_power",
        bool((penalty > 0).all()),
        [round(float(penalty.min()), 2), round(float(penalty.max()), 2)],
        "staying inside the shell is never cheaper than the free optimum",
    )
    headroom = pd.read_csv(RESULTS / "voltage_headroom_requirement.csv")
    check(
        "headroom_10mg_is_zero",
        bool((headroom[headroom["limit_mgNm3"] == 10.0]["min_voltage_headroom_pct"] == 0.0).all()),
        headroom[headroom["limit_mgNm3"] == 10.0]["min_voltage_headroom_pct"].tolist(), "all 0%",
    )
    check(
        "headroom_5mg_within_declared_margin",
        float(headroom[headroom["limit_mgNm3"] == 5.0]["min_voltage_headroom_pct"].max()) <= 5.0,
        float(headroom[headroom["limit_mgNm3"] == 5.0]["min_voltage_headroom_pct"].max()), "<=5%",
    )

    q4 = pd.read_csv(RESULTS / "question4_by_condition.csv")
    p10 = float(np.sum(q4["share"] * q4["power_10_kW"]) / q4["share"].sum())
    p5 = float(np.sum(q4["share"] * q4["power_5_kW"]) / q4["share"].sum())
    increase = 100 * (p5 / p10 - 1)
    summary = json.loads((RESULTS / "question4_summary.json").read_text())
    check("weighted_power_10", close(p10, summary["weighted_power_10_kW"], 1e-9), p10, summary["weighted_power_10_kW"
    ↪ ])
```

```
    check("weighted_power_5", close(p5, summary["weighted_power_5_kW"], 1e-9), p5, summary["weighted_power_5_kW"])
    check("weighted_increase", close(increase, summary["weighted_increase_pct"], 1e-9), increase, summary["
    ↪ weighted_increase_pct"])
    check(
        "tightening_costs_power_vs_history",
        summary["power_10_vs_historical_pct"] > 0 and summary["weighted_increase_pct"] > 0,
        {"10mg_vs_history_pct": summary["power_10_vs_historical_pct"],
         "increase_pct": summary["weighted_increase_pct"]},
        "both positive: tightening the limit costs power",
    )
    rel = np.abs(q4["analytic_delta_power_kW"] - q4["numeric_delta_power_kW"]) / q4["numeric_delta_power_kW"]
    check(
        "analytic_matches_numeric_delta_power",
        float(rel.max()) < 0.02, float(rel.max()), "<2% relative difference",
    )

    verification = pd.read_csv(RESULTS / "optimization_verification.csv")
    check("verification_rows", len(verification) == 8, len(verification), 8)
    check(
        "multistart_agrees_on_optimum",
        float(verification["multistart_power_spread_kW"].max()) < 1.0,
        float(verification["multistart_power_spread_kW"].max()), "<1 kW across converged starts",
    )
    gaps = verification["random_search_gap_pct"].dropna()
    check(
        "random_search_finds_nothing_better",
        bool((gaps >= -1e-9).all()), float(gaps.min()), ">=0 for every combination",
    )

    sens = pd.read_csv(RESULTS / "question4_sensitivity.csv")
    check("sensitivity_grid_size", len(sens) == 27, len(sens), 27)
    p_ref = pd.read_csv(RESULTS / "p_exponent_reference.csv")
    check("p_reference_rows", len(p_ref) == 5, len(p_ref), 5)
    check(
        "scale_sensitivity_feasible_count",
        int(sens["all_conditions_feasible"].sum()) == 15,
        int(sens["all_conditions_feasible"].sum()), 15,
    )
    numeric = p_ref.dropna(subset=["numeric_increase_pct"])
    gap = (numeric["analytic_increase_pct"] - numeric["numeric_increase_pct"]).abs().max()
    check(
        "analytic_vs_numeric_p_gap",
        float(gap) < 0.5, round(float(gap), 3), "<0.5 percentage points",
    )
    check(
        "p_reference_monotone_decreasing",
        bool((p_ref.sort_values("p_exponent")["analytic_increase_pct"].diff().dropna() < 0).all()),
        p_ref.sort_values("p_exponent")["analytic_increase_pct"].round(3).tolist(),
        "increase falls as p rises (proportional to 2/p)",
    )

    structural = pd.read_csv(RESULTS / "structural_sensitivity.csv")
    structural_vals = structural.loc[structural["all_conditions_feasible"] == True, "weighted_power_increase_pct"]
    check("structural_total_count", len(structural) == 81, len(structural), 81)
    check("structural_all_feasible", len(structural_vals) == 81, len(structural_vals), 81)

    rapping = pd.read_csv(RESULTS / "rapping_power_tradeoff.csv")
    check(
        "rapping_share_reported",
        30.0 < float(rapping.iloc[0]["rapping_share_pct"]) < 45.0,
        float(rapping.iloc[0]["rapping_share_pct"]), "between 30% and 45% at median settings",
    )
```

```
    strategy = pd.read_csv(RESULTS / "historical_strategy_correlations.csv")
    check(
        "strategy_windows_reported",
        set(strategy["window"]) == {"full_record", "train", "validation", "retrospective_test"},
        sorted(set(strategy["window"])),
        "four evaluation windows",
    )
    period_rule = strategy[strategy["control"].str.startswith("T")]
    check(
        "rapping_period_rule_stable_across_windows",
        bool((period_rule["r_C_in_gNm3"] < 0).all()),
        float(period_rule["r_C_in_gNm3"].max()), "<0 in every window and field",
    )
    events = pd.read_csv(RESULTS / "anomaly_events.csv")
    check("anomaly_events_documented", len(events) == 2, len(events), 2)

    manuscript = MANUSCRIPT.read_text(encoding="utf-8")
    # A final submission may print this verifier itself in Appendix D. Restrict
    # manuscript-content checks to the paper and non-code appendices so string
    # literals inside the printed source do not become false positives.
    review_manuscript = manuscript.split("## 附录D 完整源程序", 1)[0]
    ref_body = review_manuscript.split("## 参考文献", 1)[0]
    cited = set()
    for token in re.findall(r"\[([0-9,-]+)\]", ref_body):
        for part in token.split(","):
            if "-" in part:
                a, b = map(int, part.split("-"))
                cited.update(range(a, b + 1))
            else:
                cited.add(int(part))
    listed = set(map(int, re.findall(r"^\[([0-9]+)\] ", review_manuscript.split("## 参考文献", 1)[1], flags=re.M)))
    # [1]-[13] are literature: every one must be both listed and cited in the
    # text.  [14]-[16] are the AI-tool disclosure entries the competition rules
    # require in the reference list; they are declarations, not sources, so
    # they are listed without an inline citation.
    literature, ai_tools = set(range(1, 14)), set(range(14, 17))
    check("all_references_cited", cited == literature, sorted(cited), sorted(literature))
    check("reference_list_complete", listed == literature | ai_tools,
          sorted(listed), sorted(literature | ai_tools))
    ai_block = review_manuscript.split("## 参考文献", 1)[1]
    check("ai_tool_references_listed",
          all(re.search(r"^\[%d\] .+\d{4}-\d{2}-\d{2}" % n, ai_block, flags=re.M) for n in ai_tools),
          [l for l in ai_block.splitlines() if re.match(r"^\[1[4-6]\] ", l)],
          "three dated AI-tool entries")
    clean_manuscript = re.sub(r"<!--.*?-->\n?", "", review_manuscript)
    table6_segment = clean_manuscript.split("表12\u3000四工况两档目标", 1)[1].split("表13", 1)[0]
    table6_rows: dict[tuple[int, float], list[float]] = {}
    for line in table6_segment.splitlines():
        if not re.match(r"^\| [1-4] \| (?:10|5) \|", line):
            continue
        cells = [cell.strip() for cell in line.strip().strip("|").split("|")]
        table6_rows[(int(cells[0]), float(cells[1]))] = [float(x) for x in cells[2:]]
    table6_matches = len(table6_rows) == 8
    for key, displayed in table6_rows.items():
        expected_row = optimum[
            (optimum["condition_cluster"] == key[0]) & (optimum["limit_mgNm3"] == key[1])
        ]
        if len(expected_row) != 1:
            table6_matches = False
            continue
        row = expected_row.iloc[0]
        expected = [
```

```
            *[round(float(row[f"U{i}_kV"]), 1) for i in range(1, 5)],
            *[round(float(row[f"T{i}_s"]), 0) for i in range(1, 5)],
            round(float(row["scenario_peak_total_mgNm3"]), 3),
            round(float(row["predicted_power_kW"]), 2),
        ]
        table6_matches &= bool(np.allclose(displayed, expected, atol=5e-3))
    check(
        "table6_complete_and_matches_results",
        table6_matches,
        {f"condition_{key[0]}_target_{key[1]:g}": value for key, value in table6_rows.items()},
        "8 rows matching optimal_controls_central_scenario.csv after displayed rounding",
    )
    p_table_segment = clean_manuscript.split("表17\u3000功率增幅对幂次", 1)[1].split("\n\n", 3)[1]
    printed = {}
    for line in p_table_segment.splitlines():
        cells = [c.strip().strip("*") for c in line.strip().strip("|").split("|")]
        if len(cells) == 4 and re.fullmatch(r"[\d.]+", cells[0]):
            printed[float(cells[0])] = cells[2]
    expected = {float(r["p_exponent"]): f"{r['analytic_increase_pct']:.1f}%"
                for _, r in p_ref.iterrows()}
    check(
        "p_table_matches_source_at_1dp",
        printed == expected, printed, expected,
    )

    abstract = clean_manuscript.split("## 摘要", 1)[1].split("**关键词", 1)[0]
    abstract_chars = len(re.sub(r"\s+", "", abstract))
    # Range recalibrated for the "针对问题x，本文建立了……模型" abstract: naming the
    # model and method for each of the four questions costs roughly 500 more
    # characters than the previous compressed wording.  Verified against the
    # rendered PDF -- at 1870 chars the abstract fills page 1 and page 2 still
    # opens with section 1, so the upper bound is where it would start to spill.
    check("abstract_fills_one_page", 1400 <= abstract_chars <= 2000, abstract_chars, "1400-2000 chars (one page)")
    check("anonymous_manuscript", "作者：" not in clean_manuscript and "学校：" not in clean_manuscript, ["作者：" in
    ↪  clean_manuscript, "学校：" in clean_manuscript], [False, False])
    check("no_english_abstract", "## Abstract" not in clean_manuscript, "## Abstract" in clean_manuscript, False)
    check("outlet_rescaling_claim_removed", "乘0.10" not in clean_manuscript and "0.1缩放" not in clean_manuscript, [
    ↪ "乘0.10" in clean_manuscript, "0.1缩放" in clean_manuscript], [False, False])
    check("table_caption_count", len(re.findall(r"^表\d+", clean_manuscript, flags=re.M)) == 17, len(re.findall(r"^表
    ↪ \d+", clean_manuscript, flags=re.M)), 17)
    check("manuscript_figure_count", len(re.findall(r"^!\[图", clean_manuscript, flags=re.M)) == 20, len(re.findall(r
    ↪ "^!\[图", clean_manuscript, flags=re.M)), 20)
    # -- the outlet column is ruled unusable on a falsifiable test, not on a
    # -- failure to find a signal; the cap fraction is predicted, not fitted
    diag = pd.read_csv(RESULTS / "outlet_censoring_periodicity.csv")
    censoring = diag[diag["test"] == "censored_normal_mle"].iloc[0]
    check("censoring_test_present", len(diag) == 5, len(diag), 5)
    check(
        "censoring_prediction_matches_observed",
        abs(float(censoring["statistic"]) - float(censoring["reference"])) < 0.01,
        abs(float(censoring["statistic"]) - float(censoring["reference"])),
        "< 0.01",
    )
    folding = diag[diag["test"] == "epoch_folding"]
    check("rapping_periodicity_bounded", float(folding["statistic"].max()) < 0.05,
          float(folding["statistic"].max()), "< 0.05 mg/Nm3")

    # -- every optimum is a corner solution in the rapping periods; the paper
    # -- has to say so, and the P95 re-solve has to remain feasible
    active = pd.read_csv(RESULTS / "active_constraints.csv")
    check("active_constraints_reported", len(active) == 8, len(active), 8)
    check("all_solutions_corner_in_periods",
```

```
          bool(active["all_periods_at_upper_bound"].all()),
          active["all_periods_at_upper_bound"].tolist(), "all True")
    period = pd.read_csv(RESULTS / "period_bound_sensitivity.csv")
    check("period_bound_sensitivity_feasible",
          len(period) == 8 and (period["status"] == "SCENARIO_FEASIBLE").all(),
          period["status"].tolist(), "8 x SCENARIO_FEASIBLE")
    weighted = lambda frame, col: float((frame[col] * frame["share"]).sum() / frame["share"].sum())
    p10, p5 = period[period["limit_mgNm3"] == 10.0], period[period["limit_mgNm3"] == 5.0]
    p95_rise = 100.0 * (weighted(p5, "power_bound_p95_kW") / weighted(p10, "power_bound_p95_kW") - 1.0)
    check("q4_increase_robust_to_period_bound", 5.0 < p95_rise < 8.0, round(p95_rise, 2), "5-8 % under the P95 bound"
    ↪ )

    check("ai_disclosure_present", "AI工具使用声明" in clean_manuscript, "AI工具使用声明" in clean_manuscript, True)
    check("support_list_exists", (ROOT / "支撑材料文件清单.md").exists(), (ROOT / "支撑材料文件清单.md").exists(),
    ↪ True)
    ai_detail = next(ROOT.parent.glob("*/AI工具使用详情*.md"), None)
    check("ai_detail_exists", ai_detail is not None, str(ai_detail), "AI工具使用详情*.md present in the process
    ↪ directory")

    manifest = pd.read_csv(FIGURES / "figure_manifest.csv")
    check("figure_manifest_count", len(manifest) == 20, len(manifest), 20)
    required_trace = {"source_data", "transformation", "caption", "supported_manuscript_claims", "limitations"}
    check("figure_trace_fields", required_trace.issubset(manifest.columns), sorted(manifest.columns), sorted(
    ↪ required_trace))
    missing_artifacts = []
    for _, row in manifest.iterrows():
        for col in ("png", "pdf", "source_data"):
            artifact = Path(row[col])
            if not artifact.is_absolute():
                artifact = ROOT / artifact
            if not artifact.exists():
                missing_artifacts.append(f"{row['stem']}:{col}")
    check("figure_artifacts_and_sources_exist", not missing_artifacts, missing_artifacts, [])

    OUT.mkdir(parents=True, exist_ok=True)
    failed = [x for x in checks if not x["passed"]]
    report = {"status": "PASS" if not failed else "FAIL", "total_checks": len(checks), "passed": len(checks)-len(
    ↪ failed), "failed": len(failed), "checks": checks}
    (OUT / "deterministic_integrity_results.json").write_text(json.dumps(report, ensure_ascii=False, indent=2),
    ↪ encoding="utf-8")
    print(json.dumps({k: report[k] for k in ("status", "total_checks", "passed", "failed")}, ensure_ascii=False,
    ↪ indent=2))
    if failed:
        for item in failed:
            print("FAILED", item)
        raise SystemExit(1)


if __name__ == "__main__":
    main()
```

E.7 build_paper_pdf.py

```
#!/usr/bin/env python3
"""Build the CUMCM submission PDF from the final Markdown manuscript.

Converts ``10_修订后完整论文_终稿.md`` into XeLaTeX and compiles it.
Layout follows the CUMCM paper format specification: A4, >=2.5 cm margins,
abstract alone on page 1, continuous arabic page numbers centred in the footer.

Usage:  python3 src/build_paper_pdf.py [--no-code | --core-code]
"""
```

```python

from __future__ import annotations

import re
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
MANUSCRIPT = ROOT / "10_修订后完整论文_终稿.md"
BUILD = ROOT / "build"
SRC = ROOT / "src"
CODE_MODULES = ["scenario_model.py", "outlet_diagnostics.py", "paper_figures.py",
                "plot_utils.py", "compute_frontier.py", "integrity_check.py",
                "build_paper_pdf.py"]

# ---------------------------------------------------------------- inline text

_SPECIALS = {
    "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$",
    "#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}",
    "~": r"\textasciitilde{}", "^": r"\textasciicircum{}",
    # symbols the CJK/serif fonts do not carry -> render them as maths
    "\u2103": r"$^\circ$C", "\u221d": r"$\propto$", "\u2260": r"$\neq$",
    "\u2264": r"$\le$", "\u2265": r"$\ge$", "\u2248": r"$\approx$",
    "\u207b": r"$^-$", "\u00b9": r"$^1$", "\u2070": r"$^0$",
    "\u2192": r"$\rightarrow$", "\u2191": r"$\uparrow$", "\u2193": r"$\downarrow$",
}


def esc(text: str) -> str:
    return "".join(_SPECIALS.get(ch, ch) for ch in text)


#: bare links and DOIs are single unbreakable words for TeX; \url can break them
_URL_RE = re.compile(r"(https?://\S+?)(?=[\s\u3002\uff0c]|$)|(?<=DOI: )(10\.\S+?)(?=[\s\u3002]|$)")


def _urlify(piece: str) -> str:
    r"""Escape text, but wrap any bare URL or DOI in a breakable url command."""
    out: list[str] = []
    last = 0
    for m in _URL_RE.finditer(piece):
        out.append(esc(piece[last:m.start()]))
        link = m.group(0).rstrip(".")
        trailing = m.group(0)[len(link):]
        out.append(r"\url{" + link + "}" + esc(trailing))
        last = m.end()
    out.append(esc(piece[last:]))
    return "".join(out)


#: Markdown bold may wrap a maths span ("**若把$x$作为约束**"), so the markers
#: cannot be resolved inside a single plain-text piece. They are replaced by a
#: sentinel first and paired up after every segment has been rendered.
_BOLD = "\x00"
#: single "*" is italic. Maths is split off before this runs, so the "*" in
#: spans like $C^*$ is never seen here, and list bullets are stripped by the
#: block parser, so the only "*" reaching this point is emphasis.
_ITAL = "\x01"


```

```
def _smart_quotes(piece: str, state: list[bool]) -> str:
    """Straight double quotes in Chinese prose -> paired curly quotes.

    ``state`` carries the open/close flag across the whole line, so a quoted
    phrase that wraps a maths span still gets a closing quote at its end.
    """
    out = []
    for ch in piece:
        if ch == '"':
            out.append("\u201c" if state[0] else "\u201d")
            state[0] = not state[0]
        else:
            out.append(ch)
    return "".join(out)


def inline(text: str) -> str:
    """Convert inline Markdown to LaTeX, leaving $...$ math untouched."""
    out: list[str] = []
    quote_open = [True]
    for i, seg in enumerate(re.split(r"(\$[^$]*\$)", text)):
        if i % 2:                       # math segment - pass through verbatim
            out.append(seg)
            continue
        # code spans next: their content must be escaped but not styled
        for j, piece in enumerate(re.split(r"(`[^`]*`)", seg)):
            if j % 2:
                out.append(r"\texttt{" + esc(piece[1:-1]) + "}")
            else:
                piece = piece.replace("**", _BOLD).replace("*", _ITAL)
                out.append(_urlify(_smart_quotes(piece, quote_open)))
    rendered = "".join(out)
    return _pair(_pair(rendered, _BOLD, "textbf"), _ITAL, "textit")


def _pair(text: str, sentinel: str, command: str) -> str:
    """Turn alternating sentinels into a LaTeX command; drop an odd trailing one."""
    parts = text.split(sentinel)
    if len(parts) == 1:
        return text
    result = parts[0]
    for k, chunk in enumerate(parts[1:], start=1):
        result += ("\\" + command + "{" if k % 2 else "}") + chunk
    if len(parts) % 2 == 0:             # odd number of markers
        result += "}"
    return result


# ---------------------------------------------------------------- block level

def _display_width(text: str) -> int:
    """Rough typeset width in en-units (CJK glyphs are double width)."""
    text = re.sub(r"\$[^$]*\$", "xx", text)          # math sets compactly
    text = re.sub(r"[*`]", "", text)
    return sum(2 if ord(ch) > 0x2E80 else 1 for ch in text)


# A row of ``l`` columns cannot wrap, so any table whose natural width exceeds
# the text block silently runs off the page. Text-heavy columns therefore
# become wrapping ``X`` columns and the table is set to exactly \textwidth.
_LINE_BUDGET = 84          # en-units that fit across the 16 cm text block
_WRAP_THRESHOLD = 16       # a column wider than this must be allowed to wrap
```

```


def convert_table(header: list[str], align: list[str], rows: list[list[str]],
                  number: str, title: str) -> str:
    ncol = len(header)
    widths = [max([_display_width(header[j])]
                  + [_display_width(r[j]) for r in rows if j < len(r)])
              for j in range(ncol)]
    natural = sum(widths) + 2 * ncol

    wrappable = [j for j in range(ncol)
                 if align[j] == "l" and widths[j] > _WRAP_THRESHOLD]
    use_tabularx = natural > _LINE_BUDGET and bool(wrappable)

    if use_tabularx:
        # Share the wrapping width between the text columns in proportion to
        # how much text each of them actually carries.
        # Proportional-to-length shares starve short-but-not-short-enough
        # columns, so damp the ratio with a square root and floor it.
        raw = {j: max(widths[j], 1) ** 0.5 for j in wrappable}
        total_wrap = sum(raw.values())
        shares = {j: max(len(wrappable) * raw[j] / total_wrap, 0.55) for j in wrappable}
        scale = len(wrappable) / sum(shares.values())
        parts = []
        for j in range(ncol):
            if j in wrappable:
                parts.append(r">{\hsize=%.3f\hsize\raggedright\arraybackslash}X"
                             % (shares[j] * scale))
            else:
                parts.append({"r": "r", "c": "c"}.get(align[j], "l"))
        open_env = r"\begin{tabularx}{\textwidth}{" + "".join(parts) + "}"
        close_env = r"\end{tabularx}"
        shrink = False
    else:
        spec = "".join({"r": "r", "c": "c"}.get(a, "l") for a in align)
        open_env = r"\begin{tabular}{" + spec + "}"
        close_env = r"\end{tabular}"
        shrink = natural > _LINE_BUDGET        # all-numeric wide table: scale it

    lines = [r"\begin{table}[htb]", r"\centering",
             r"\caption*{\normalsize 表" + number + "\u3000" + inline(title) + "}",
             r"\vspace{-2pt}", r"\zihao{-5}"]
    if shrink:
        lines.append(r"\resizebox{\textwidth}{!}{%")
    lines += [open_env, r"\toprule",
              " & ".join(inline(h) for h in header) + r" \\", r"\midrule"]
    for row in rows:
        lines.append(" & ".join(inline(c) for c in row) + r" \\")
    lines += [r"\bottomrule", close_env]
    if shrink:
        lines.append("}")
    lines.append(r"\end{table}")
    return "\n".join(lines)


def parse_align(sep_row: list[str]) -> list[str]:
    out = []
    for cell in sep_row:
        cell = cell.strip()
        if cell.startswith(":") and cell.endswith(":"):
            out.append("c")
        elif cell.endswith(":"):
```

```
            out.append("r")
        else:
            out.append("l")
    return out


def split_row(line: str) -> list[str]:
    return [c.strip() for c in line.strip().strip("|").split("|")]


def convert(md: str) -> tuple[str, str, str]:
    """Return (title, abstract_block, body)."""
    md = re.sub(r"<!--.*?-->", "", md, flags=re.S)
    lines = md.split("\n")

    title = ""
    abstract: list[str] = []
    body: list[str] = []
    target = body

    i = 0
    pending_table: tuple[str, str] | None = None   # (number, title)
    while i < len(lines):
        line = lines[i].rstrip()

        # ---- headings
        m = re.match(r"^# (.+)$", line)
        if m:
            title = m.group(1).strip()
            i += 1
            continue
        m = re.match(r"^## (.+)$", line)
        if m:
            head = m.group(1).strip()
            if head == "摘要":
                target = abstract
                i += 1
                continue
            target = body
            hm = re.match(r"^(\d+)\s+(.*)$", head)
            if hm:
                target.append(r"\section{" + inline(hm.group(2)) + "}")
            else:
                target.append(r"\section*{" + inline(head) + "}")
                target.append(r"\addcontentsline{toc}{section}{" + inline(head) + "}")
            i += 1
            continue
        m = re.match(r"^### (.+)$", line)
        if m:
            head = m.group(1).strip()
            hm = re.match(r"^([\d.]+|[A-Z]\.\d+)\s+(.*)$", head)
            target.append(r"\subsection*{" + inline(head if not hm else head) + "}")
            i += 1
            continue

        # ---- fenced code
        if line.startswith("```"):
            j = i + 1
            buf = []
            while j < len(lines) and not lines[j].startswith("```"):
                buf.append(lines[j])
                j += 1
```

```
            target.append(r"\begin{lstlisting}[style=pseudo]")
            target.extend(buf)
            target.append(r"\end{lstlisting}")
            i = j + 1
            continue

        # ---- display math
        if line.strip() == "$$":
            j = i + 1
            buf = []
            while j < len(lines) and lines[j].strip() != "$$":
                buf.append(lines[j])
                j += 1
            expr = "\n".join(buf).strip().rstrip(",.")
            target.append(r"\begin{equation}")
            target.append(expr)
            target.append(r"\end{equation}")
            i = j + 1
            continue

        # ---- image  ->  figure float
        m = re.match(r"^!\[图(\d+)\s*([^\]]*)\]\(([^)]+)\)\s*$", line)
        if m:
            num, alt, path = m.group(1), m.group(2), m.group(3)
            cap = alt
            k = i + 1
            while k < len(lines) and not lines[k].strip():
                k += 1
            cm = re.match(r"^图%s\u3000(.+)$" % num, lines[k].strip()) if k < len(lines) else None
            if cm:
                cap = cm.group(1)
                i = k
            width = "0.96" if path.endswith("00_esp_schematic.pdf") else "0.92"
            target += [r"\begin{figure}[htb]", r"\centering",
                       r"\includegraphics[width=%s\textwidth]{%s}" % (width, path),
                       r"\caption*{\normalsize 图" + num + "\u3000" + inline(cap) + "}",
                       r"\end{figure}"]
            i += 1
            continue

        # ---- standalone figure caption already consumed above; skip strays
        if re.match(r"^图\d+\u3000", line.strip()):
            i += 1
            continue

        # ---- table title line
        m = re.match(r"^表(\d+|A\d+)\u3000(.+)$", line.strip())
        if m:
            pending_table = (m.group(1), m.group(2))
            i += 1
            continue

        # ---- table body
        if line.strip().startswith("|"):
            header = split_row(line)
            align = parse_align(split_row(lines[i + 1]))
            rows = []
            j = i + 2
            while j < len(lines) and lines[j].strip().startswith("|"):
                rows.append(split_row(lines[j]))
                j += 1
            num, ttl = pending_table if pending_table else ("", "")
```

```
            target.append(convert_table(header, align, rows, num, ttl))
            pending_table = None
            i = j
            continue

        # ---- blockquote
        if line.startswith(">"):
            buf = [line.lstrip("> ").strip()]
            j = i + 1
            while j < len(lines) and lines[j].startswith(">"):
                buf.append(lines[j].lstrip("> ").strip())
                j += 1
            target.append(r"\begin{quote}\small " + inline(" ".join(buf)) + r"\end{quote}")
            i = j
            continue

        # ---- ordered / unordered list
        if re.match(r"^\d+\.\s+", line) or re.match(r"^[-*]\s+", line):
            ordered = bool(re.match(r"^\d+\.\s+", line))
            env = "enumerate" if ordered else "itemize"
            items = []
            j = i
            while j < len(lines) and (re.match(r"^\d+\.\s+", lines[j]) or re.match(r"^[-*]\s+", lines[j])):
                items.append(re.sub(r"^(\d+\.|[-*])\s+", "", lines[j].rstrip()))
                j += 1
            target.append(r"\begin{%s}[itemsep=1pt,topsep=3pt]" % env)
            target += [r"\item " + inline(it) for it in items]
            target.append(r"\end{%s}" % env)
            i = j
            continue

        # ---- blank
        if not line.strip():
            target.append("")
            i += 1
            continue

        # ---- plain paragraph
        target.append(inline(line.strip()))
        i += 1

    return title, "\n".join(abstract).strip(), "\n".join(body).strip()


# ---------------------------------------------------------------- preamble

PREAMBLE = r"""
\documentclass[UTF8,zihao=5,a4paper]{ctexart}
\usepackage[a4paper,top=2.5cm,bottom=2.5cm,left=2.5cm,right=2.5cm]{geometry}
\usepackage{amsmath,amssymb,bm}
\usepackage{graphicx}
\usepackage{booktabs}
\usepackage{caption}
\usepackage{enumitem}
\usepackage{listings}
\usepackage{xcolor}
\usepackage{setspace}
\usepackage{array}
\usepackage{tabularx}
\usepackage{fancyhdr}
\usepackage{xurl}

```

```
%%FONTS%%

\onehalfspacing
\setlength{\parindent}{2em}
\captionsetup{skip=4pt}

% The body has 20 figures, 17 tables and a dozen display equations, and at
% 1.5 line spacing LaTeX's default separations around them are generous. The
% competition caps the body at 30 pages, so the float and display gaps are
% tightened here. Margins (2.5 cm), body size (5号) and 1.5 spacing are fixed
% by the format specification and are left alone.
\setlength{\textfloatsep}{8pt plus 2pt minus 2pt}
\setlength{\intextsep}{8pt plus 2pt minus 2pt}
\setlength{\floatsep}{8pt plus 2pt minus 2pt}
\setlength{\abovedisplayskip}{5pt plus 2pt minus 2pt}
\setlength{\belowdisplayskip}{5pt plus 2pt minus 2pt}
\setlength{\abovedisplayshortskip}{3pt plus 1pt minus 1pt}
\setlength{\belowdisplayshortskip}{3pt plus 1pt minus 1pt}

% No page may carry a float and nothing else. The "p" placement is what lets
% LaTeX build such a page, so figures and tables are placed [htb] only, and
% \textfraction then guarantees every page keeps at least a little running
% text. The counterpart is that a float must be allowed to take almost the
% whole of a top or bottom area, or it would have nowhere left to go.
\renewcommand{\topfraction}{0.92}
\renewcommand{\bottomfraction}{0.75}
\renewcommand{\textfraction}{0.06}
\setcounter{topnumber}{3}
\setcounter{bottomnumber}{2}
\setcounter{totalnumber}{5}

% Line breaking.  \sloppy (tolerance 9999) let TeX stretch the inter-word and
% inter-CJK glue without bound, so a line holding a long unbreakable inline
% formula was padded until the Chinese characters visibly drifted apart.  Use a
% permissive but finite tolerance instead, let long inline maths wrap at its
% relations and binary operators, and cap how far the CJK glue may stretch.
\tolerance=3000
\emergencystretch=2em
\relpenalty=200
\binoppenalty=400
\hfuzz=2pt
\hbadness=3000
\xeCJKsetup{CJKglue=\hskip 0pt plus 0.04\baselineskip}

\pagestyle{fancy}
\fancyhf{}
\renewcommand{\headrulewidth}{0pt}
\fancyfoot[C]{\thepage}

\ctexset{
  section/format = {\centering\zihao{4}\bfseries},
  subsection/format = {\raggedright\zihao{5}\bfseries},
  section/beforeskip = 9pt plus 2pt minus 2pt,
  section/afterskip = 6pt,
  subsection/beforeskip = 7pt plus 2pt minus 2pt,
  subsection/afterskip = 4pt,
}

\lstdefinestyle{pseudo}{
  basicstyle=\ttfamily\zihao{-5},
  breaklines=true, breakatwhitespace=false,
  columns=fullflexible, keepspaces=true,
```

```
  frame=single, rulecolor=\color{gray!55},
  framesep=4pt, xleftmargin=6pt, xrightmargin=2pt,
  showstringspaces=false, extendedchars=true,
}
\lstdefinestyle{pycode}{
  language=Python,
  basicstyle=\ttfamily\zihao{6},
  keywordstyle=\color{blue!65!black},
  commentstyle=\color{green!42!black}\itshape,
  stringstyle=\color{orange!75!black},
  breaklines=true, breakatwhitespace=false,
  breakindent=12pt, postbreak=\mbox{\textcolor{gray}{$\hookrightarrow$}\space},
  columns=fullflexible, keepspaces=true,
  showstringspaces=false, extendedchars=true,
  numbers=left, numberstyle=\tiny\color{gray}, numbersep=6pt,
  frame=leftline, rulecolor=\color{gray!45}, framesep=5pt,
  xleftmargin=17pt, tabsize=4, upquote=true,
}

% 手工编号的图表题注（与正文交叉引用一致）
\captionsetup[figure]{labelformat=empty,justification=centering}
\captionsetup[table]{labelformat=empty,justification=centering}
"""



# ------------------------------------------------------------- fonts

#: Preferred first, portable fallback second. The submission machine normally
#: has TeX Gyre Termes and Noto CJK; a bare TeX Live may not.
FONT_CHOICES = {
    "main": ["TeX Gyre Termes", "Liberation Serif", "DejaVu Serif"],
    # The Latin mono must NOT be a CJK font. "Noto Sans Mono CJK SC" renders
    # ASCII correctly but tags its hyphen and pipe glyphs as U+2011 and U+FFE8,
    # so the appendix listings looked right yet copied out as Python that will
    # not parse. Chinese inside listings is covered by "cjkmono" below.
    "mono": ["DejaVu Sans Mono", "Noto Sans Mono", "Liberation Mono"],
    "cjkmain": ["Noto Serif CJK SC", "Fandol Song", "Noto Sans CJK SC"],
    "cjksans": ["Noto Sans CJK SC", "Fandol Hei"],
    "cjkmono": ["Noto Sans Mono CJK SC", "Noto Sans CJK SC"],
}


def _installed_families() -> set[str]:
    try:
        out = subprocess.run(["fc-list", ":", "family"], capture_output=True,
                             text=True, check=True).stdout
    except (OSError, subprocess.CalledProcessError):
        return set()
    return {name.strip() for line in out.splitlines() for name in line.split(",")}


def font_block() -> str:
    available = _installed_families()

    def pick(key: str) -> str:
        for name in FONT_CHOICES[key]:
            if not available or name in available:
                return name
        return FONT_CHOICES[key][-1]

    return "\n".join([
```

```
        r"\setCJKmainfont{%s}" % pick("cjkmain"),
        r"\setCJKsansfont{%s}" % pick("cjksans"),
        r"\setCJKmonofont{%s}" % pick("cjkmono"),
        r"\setmainfont{%s}" % pick("main"),
        r"\setmonofont{%s}[Scale=0.82]" % pick("mono"),
    ])


def build_tex(title: str, abstract: str, body: str, code: str) -> str:
    parts = [PREAMBLE.replace("%%FONTS%%", font_block()), r"\begin{document}", ""]
    # --- page 1: title + abstract only
    parts += [
        r"\begin{center}",
        r"{\zihao{3}\bfseries " + inline(title) + r"}",
        r"\end{center}",
        r"\vspace{6pt}",
        r"\begin{center}{\zihao{4}\bfseries 摘\quad 要}\end{center}",
        r"\vspace{2pt}",
    ]
    kw = ""
    keep = []
    for l in abstract.split("\n"):          # 保留空行以维持摘要的分段结构
        if l.startswith(r"\textbf{关键词："):
            kw = l
        else:
            keep.append(l)
    while keep and not keep[-1].strip():
        keep.pop()
    parts += keep
    if kw:
        parts += ["", r"\vspace{8pt}", "", r"\noindent " + kw]
    parts += [r"\clearpage", ""]
    parts.append(body)
    if code:
        parts.append(code)
    parts += ["", r"\end{document}", ""]
    return "\n".join(parts)


def _extract(module: str, names: list[str]) -> str:
    """Pull whole top-level definitions out of a source file, in order."""
    text = (SRC / module).read_text(encoding="utf-8").split("\n")
    out: list[str] = []
    for name in names:
        try:
            i = next(k for k, l in enumerate(text)
                     if l.startswith(("def " + name + "(", "class " + name)))
        except StopIteration:
            continue
        j = i + 1
        while j < len(text) and (not text[j].strip() or text[j][:1] in " )]}\t"):
            j += 1
        while j > i and not text[j - 1].strip():
            j -= 1
        out.extend(text[i:j] + [""])
    return "\n".join(out).rstrip() + "\n"


#: Appendix E carries the full listing by default; ``--core-code`` prints only
#: the two definitions that determine every number in the paper.
CORE_EXTRACTS = [
    ("scenario_model.py", ["PhysicalPowerModel"]),
```

```
    ("scenario_model.py", ["scenario_emission", "field_priority"]),
]


def code_appendix(full: bool = True) -> str:
    title = "\u9644\u5f55E\\quad " + ("\u5b8c\u6574\u6e90\u7a0b\u5e8f" if full else "\u6838\u5fc3\u6e90\u7a0b\u5e8f\
    ↪ u8282\u9009")
    out = [r"\clearpage",
           r"\section*{%s}" % title,
           r"\addcontentsline{toc}{section}{%s}" % title]
    if full:
        out.append("\u672c\u9644\u5f55\u7ed9\u51fa\u751f\u6210\u672c\u6587\u5168\u90e8\u7ed3\u679c\u4e0e\u56fe\u8868\
    ↪ u7684\u5b8c\u6574\u53ef\u8fd0\u884c\u6e90\u7a0b\u5e8f\uff0c\u4e0e\u7535\u5b50\u652f\u6491\u6750\u6599 \\texttt{
    ↪ src/} \u76ee\u5f55\u9010\u5b57\u4e00\u81f4\u3002"
                   "\u6309\u4f9d\u8d56\u987a\u5e8f\u6392\u5217\uff1a\u60c5\u666f\u6a21\u578b\u4e0e\u4f18\u5316\u3001\
    ↪ u6392\u653e\u2014\u529f\u7387\u524d\u6cbf\u3001\u7ed8\u56fe\u6837\u5f0f\u4e0e\u56fe\u96c6\u751f\u6210\u3001\
    ↪ u786e\u5b9a\u6027\u6838\u9a8c\u3001\u63d0\u4ea4\u7a3f\u88c5\u914d\u3002")
        for n, name in enumerate(CODE_MODULES, start=1):
            path = SRC / name
            if not path.exists():
                continue
            out += [r"\subsection*{E.%d\quad \texttt{%s}}" % (n, esc(name)),
                    r"\lstinputlisting[style=pycode]{%s}" % (path.as_posix())]
        return "\n".join(out)

    out.append("\u4e3a\u63a7\u5236\u7bc7\u5e45\uff0c\u672c\u9644\u5f55\u53ea\u7ed9\u51fa\u51b3\u5b9a\u5168\u6587\
    ↪ u6570\u503c\u7ed3\u679c\u7684\u4e24\u6bb5\u6838\u5fc3\u5b9a\u4e49\uff1b"
               "\u5b8c\u6574\u53ef\u8fd0\u884c\u6e90\u7801\u968f\u7535\u5b50\u652f\u6491\u6750\u6599\u63d0\u4ea4\
    ↪ u3002")
    BUILD.mkdir(exist_ok=True)
    for n, (module, names) in enumerate(CORE_EXTRACTS, start=1):
        snippet = BUILD / ("code_excerpt_%d.py" % n)
        snippet.write_text(_extract(module, names), encoding="utf-8")
        # the format string is raw, so the separator cannot be written as an
        # escape inside it -- \uff1a would reach XeLaTeX as a control sequence
        out += [r"\subsection*{E.%d\quad \texttt{%s}%s%s}"
                % (n, esc(module), "\uff1a", "\u3001".join(esc(x) for x in names)),
                r"\lstinputlisting[style=pycode]{%s}" % snippet.as_posix()]
    return "\n".join(out)


def main() -> None:
    include_code = "--no-code" not in sys.argv
    full_code = "--core-code" not in sys.argv
    md = MANUSCRIPT.read_text(encoding="utf-8")
    title, abstract, body = convert(md)
    tex = build_tex(title, abstract, body, code_appendix(full_code) if include_code else "")

    BUILD.mkdir(exist_ok=True)
    tex_path = BUILD / "paper.tex"
    tex_path.write_text(tex, encoding="utf-8")

    for run in range(2):
        proc = subprocess.run(
            ["xelatex", "-interaction=nonstopmode", "-halt-on-error",
             "-output-directory", str(BUILD), str(tex_path)],
            cwd=ROOT, capture_output=True, text=True)
        if proc.returncode != 0:
            log = (BUILD / "paper.log").read_text(encoding="utf-8", errors="replace")
            errs = [l for l in log.split("\n") if l.startswith("!")][:12]
            print("XeLaTeX 失败：\n" + "\n".join(errs))
            sys.exit(1)
```

```
    print("已生成", BUILD / "paper.pdf")


if __name__ == "__main__":
    main()
```